



UTPL

UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA

La Universidad Católica de Loja

FACULTAD DE INGENIERÍAS Y ARQUITECTURA

CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

**Sistema IoT para el monitoreo inteligente de la calidad del
aire en zonas urbanas de Quito**

Trabajo de Integración Curricular previo a la obtención del título de:

Ingeniero en Tecnologías de la Información

Autor: Mejia Dueñas Farith Anselmo

Director: Elizalde Solano René Rolando

Quito
Agosto 2026

Comentado [1]: Nombres completos,
Apellidos - Nombre

Aprobación del director del Trabajo de Integración Curricular

Loja, 31 de agosto del 2026

Doctor

Jorge Afranio López Vargas

Director de la carrera de Tecnologías de la Información

Ciudad. -

De mi consideración:

Me permito comunicar que, en calidad de director del presente Trabajo de Integración Curricular denominado: Sistema IoT para el monitoreo inteligente de la calidad del aire en zonas urbanas de Quito, realizado por Farith Anselmo Mejia Dueñas ha sido orientado y revisado durante su ejecución, así mismo ha sido verificado a través de la herramienta de similitud académica institucional, y cuenta con un porcentaje de coincidencia aceptable. En virtud de ello, y por considerar que el mismo cumple con todos los parámetros establecidos por la Universidad, doy mi aprobación a fin de continuar con el proceso académico correspondiente.

Particular que comunico para los fines pertinentes.

Atentamente,

Magister René Rolando Elizalde Solano

Director del Trabajo de Integración Curricular

C.I.: 1104126030

Correo electrónico: rrelizalde@utpl.edu.ec

Declaración de autoría y cesión de derechos

Yo, Farith Anselmo Mejía Dueñas, declaro y acepto en forma expresa lo siguiente:

Ser autor (a) del Trabajo de Integración Curricular denominado Sistema IoT para el monitoreo inteligente de la calidad del aire en zonas urbanas de Quito , de la carrera de Tecnologías de la Información, específicamente de los contenidos comprendidos en: (se debe colocar los nombres de los capítulos elaborados en el Trabajo de Integración Curricular, siendo (nombres y apellidos completos), director (a) del presente trabajo; también declaro que la presente investigación no vulnera derechos de terceros ni utiliza fraudulentamente obras preexistentes. Además, ratifico que las ideas, criterios, opiniones, procedimientos y resultados vertidos en el presente trabajo investigativo, son de mi exclusiva responsabilidad. Eximo expresamente a la Universidad Técnica Particular de Loja y a sus representantes legales de posibles reclamos o acciones judiciales o administrativas, con relación a la propiedad intelectual de este trabajo.

Que la presente obra, producto de mis actividades académicas y de investigación, forma parte del patrimonio de la Universidad Técnica Particular de Loja, de conformidad con el artículo 20, literal j), de la Ley Orgánica de Educación Superior; y, artículo 91 del Estatuto Orgánico de la UTPL, que establece: "Forman parte del patrimonio de la Universidad la propiedad intelectual de investigaciones, trabajos científicos o técnicos y tesis de grado que se realicen a través, o con el apoyo financiero, académico o institucional (operativo) de la Universidad", en tal virtud, cedo a favor de la Universidad Técnica Particular de Loja la titularidad de los derechos patrimoniales que me corresponden en calidad de autor/a, de forma incondicional, completa, exclusiva y por todo el tiempo de su vigencia.

La Universidad Técnica Particular de Loja queda facultada para ingresar el presente trabajo al Sistema Nacional de Información de la Educación Superior del Ecuador para su difusión pública, en cumplimiento del artículo 144 de la Ley Orgánica de Educación Superior.

.....
Farith Anselmo Mejia Dueñas

Autor del Trabajo de Integración Curricular

C.I.: 1726974635

Correo electrónico: famejia3@utpl.edu.ec

Dedicatoria

Tu apoyo siempre ha sido fundamental, has estado conmigo incluso en los momentos más turbulentos y oscuros. Llegar hasta acá no fue fácil, pero tu motivación y tu ejemplo hizo que vuelva a creer en mí nuevamente.

Te lo agradezco Petalita, desde lo más profundo de mi ser, por eso mismo, este trabajo te lo dedico, uno de muchísimos logros individuales que lograremos, juntos.

Agradecimiento

Quiero expresar mi sincero agradecimiento al Ing. Solano por su guía, paciencia y apoyo técnico en la dirección de este proyecto, así como a los docentes de la UTPL por toda la formación profesional recibida.

A mi familia adoptiva, les quedo completamente agradecidos por su apoyo incondicional y darme las fuerzas por continuar, de igual forma para mi novia al ser mi motivación durante este último tramo de mi carrera profesional.

Finalmente, a mi gato Oreo, por su fiel y constante compañía durante todas las noches y días más frustrantes.

Índice de contenido

Carátula	I
Aprobación del director del Trabajo de Integración Curricular	II
Declaración de autoría y cesión de derechos	III
Dedicatoria	V
Agradecimiento	VI
Índice de contenido	VII
Resumen	1
Abstract	2
1. Capítulo uno	3
1.1. Introducción	3
1.2. Antecedentes	5
1.3. Planteamiento del Problema	7
1.4. Justificación	7
1.5. Objetivos	9
Capítulo dos	10
2. Marco Teórico	10
2.1. Contaminación Atmosférica y Parámetros de Calidad del Aire	10
2.1.1. Monóxido de Carbono (CO)	10
2.1.2. Dióxido de Carbono (CO ₂)	11
2.1.3. Material Particulado (PM _{2.5} y PM ₁₀)	13
2.1.4. Dióxido de Nitrógeno (NO ₂)	15
2.1.5. Ozono Troposférico (O ₃)	16
2.2. Fundamentos del Internet de las Cosas (IoT) y Redes de Sensores	17
1.3.3. Estructura y dinámica operativa del protocolo MQTT	29
1.3.4. Infraestructura de Transporte y Estructuración de Datos	32

1.3.5. Protocolo de Control de Transmisión (TCP)	32
Capítulo tres	53
Análisis	53
3.1. Análisis de requisitos	53
3.1.2. Clasificación y Roles de los actores del sistema.....	55
3.1.3. Historias de Usuario	55
3.1.4. Casos de Uso del Sistema.....	57
3.1.4.1. Casos de uso Específicos	58
3.1.4.2. Diagrama de Clases	71
3.1.5. Planificación de Sprints.....	73
3.1.6. Planificación en base al análisis	75
3.1.6.1. Priorización de requerimientos	75
3.1.6.2. Estrategia de Mitigación de Riesgos Técnicos	75
3.1.6.3. Estimación de Recursos y Esfuerzo	76
Capítulo Cuatro	78
Desarrollo de la Solución	78
4.1. Sprint 0: Diseño de la solución (Modelo C4).....	78
4.1.1. Diseño lógico de la Base de Datos (Persistencia Híbrida)	78
4.1.2. Diagrama de Componentes	80
4.1.3. Arquitectura lógica y física.....	81
4.1.3. Arquitectura lógica y física.....	83
4.1.4.1. Sistema de Diseño Y Paleta Semántica	83
4.2. Sprint 1: Arquitectura Base, Seguridad y Conectividad Edge-to-Cloud	87
4.2.1. Backlog y Planificación del Sprint 1	88
4.3. Desarrollo de las Tareas dentro del Sprint 1.....	89
4.4. Desarrollo de las Tareas dentro del Sprint 2.....	97
4.4.1. Backlog y Planificación del Sprint 2	97
4.5. Desarrollo de las Tareas dentro del Sprint 3.....	105

4.5.1. Backlog y Planificación del Sprint 3	105
Capítulo Cinco.....	114
5.1. Pruebas de Usabilidad.....	114
5.1.1. Escala de Usabilidad del Sistema (SUS) como instrumento de evaluación	114
5.1.2. Diseño del Escenario de Prueba	114
5.1.3. Procesamiento y Resultados	115
5.1.4. Discusión de Resultados.....	117
5.2. Pruebas de Accesibilidad.....	118
5.3.1. Análisis de Hallazgos y Oportunidades de Mejora	119
5.3. Pruebas de Seguridad y Análisis de Vulnerabilidades.....	120
5.5.1. Prevención de inyecciones de código.....	120
5.5.2. Mitigación de fuga de información y vulnerabilidades XSS.....	121
5.5.3. Resultados de Seguridad	123
Conclusiones	125
Recomendaciones	127
Referencias	128
Apéndice.....	130
Apéndice 1. Usuarios en Pruebas de Usabilidad.....	130

Índice de tablas

Índice de figuras

Resumen

El propósito central de este estudio radica en facilitar el acceso público e instantáneo a datos ambientales, superando las restricciones de alcance hiperlocal y los elevados gastos de sostenimiento de las estaciones de vigilancia convencionales. En el ámbito metodológico, el desarrollo siguió el enfoque ágil Scrum a través de una estructura desacoplada en tres niveles funcionales. En la fase de captura física, la placa ESP32 contó con una interconexión con sensores económicos los cuales recolectaron registros de partículas suspendidas, Dióxido de Carbono y parámetros climáticos, enviando las tramas por Wi-Fi bajo el protocolo hacia la nube de AWS IoT Core. Por su parte, el procesamiento y almacenamiento se articularon con una API asíncrona en FastAPI y un esquema dual de datos, empleando MongoDB para la gestión administrativa e InfluxDB para el flujo de datos ininterrumpidos. La interfaz de usuario se implementó en React.js, incorporando mecanismos de alerta configurados a partir de los límites sugeridos por la Organización Mundial de la Salud (OMS). Las evaluaciones experimentales demostraron un comportamiento óptimo del sistema, obteniendo 82.3 puntos en la escala de usabilidad SUS, estándares certificados de accesibilidad web y alta tolerancia frente a vulnerabilidades como inyecciones No SQL y XSS. Así, la plataforma valida su utilidad como un recurso técnico confiable y escalable para impulsar la participación comunitaria y respaldar medidas de cuidado respiratorio.

Palabras clave: Monitoreo Ambiental, Telemetría MQTT, Series Temporales, Arquitectura IoT, Microcontrolador ESP32.

Abstract

The central purpose of this study is to facilitate instantaneous public access to environmental data, overcoming the restrictions of hyperlocal scope and the high maintenance costs of conventional monitoring stations. From a methodological standpoint, the development followed the Scrum agile approach through a decoupled structure across three functional levels. In the data acquisition phase, the ESP32 board was connected to low-cost sensors that collected readings of particulate matter, carbon dioxide, and climate parameters, sending data packets via Wi-Fi using the protocol to the AWS IoT Core cloud. Meanwhile, processing and storage were coordinated using an asynchronous API in FastAPI and a dual data schema, employing MongoDB for administrative management and InfluxDB for uninterrupted data flow. The user interface was implemented in React.js, incorporating alert mechanisms configured based on the thresholds suggested by the World Health Organization (WHO). Experimental evaluations demonstrated optimal system performance, scoring 82.3 points on the SUS usability scale, meeting certified web accessibility standards, and exhibiting high resilience against vulnerabilities such as SQL injection and XSS. Thus, the platform validates its usefulness as a reliable and scalable technical resource to promote community participation and support respiratory care measures.

Keywords: Internet of Things (IoT), Air quality, Low-cost sensors, FastAPI, React.js, InfluxDB, MQTT.

1. Capítulo uno

1.1. Introducción

La calidad del aire representa un desafío crítico para la salud pública y la sostenibilidad en las ciudades modernas, especialmente en entornos como Quito, donde la topografía irregular y la densidad de fuentes móviles limitan la dispersión de contaminantes. Hoy en día la ciudad depende de la Red Metropolitana de Monitoreo de la Calidad del Aire (REMMAQ), la cual, si bien proporciona datos valiosos, se limita a estaciones fijas de alto costo que no lograrán capturar las variaciones micro espaciales en zonas específicas. Frente a este escenario restrictivo, la investigación plantea la creación de una plataforma fundamentada en el internet de las cosas (IoT) mediante microcontroladores e instrumentación de costo asequible. La propuesta tiene como fin aperturar y difundir las métricas ambientales valiéndose de un esquema distribuido en tres niveles funcionales: la adquisición, tratamiento de los datos y la presentación interactiva hacia la población, estimulando de esta forma la ciencia comunitaria y la participación social informada.

El núcleo de la problemática reside en la escasa resolución espacial e intermitencia con las que registran las partículas finas PM_{2.5} Y PM₁₀ y los gases nocivos en áreas urbanas alejadas de la infraestructura de monitoreo oficial. Particularmente en el sector de "El Rosario", la falta de mediciones continuas e instantáneas produce una brecha de información que obstaculiza la toma oportuna de acciones preventivas frente a picos de polución, llegando a perjudicar de forma directa a grupos vulnerables como infantes y adultos de la tercera edad. De igual manera, las estaciones fijas estándar demandan presupuestos elevados de su instalación y mantenimiento, lo que limita su expansión y hace que los ciudadanos se encuentren privados de información para comprender las condiciones ambientales de su entorno cercano. Este panorama exige que exista un despliegue técnico de un sistema de nodos interconectados de forma inalámbrica que pueda romper los límites en cobertura y logre registrar focos críticos de emisión que están habitualmente inadvertidos por los mecanismos tradicionales.

El propósito fundamental de esta investigación es diseñar e implementar un sistema inteligente basado en el Internet de las Cosas (IoT) para el monitoreo de la calidad del aire en el barrio “El Rosario”, facilitando el acceso a datos en tiempo real para poder apoyar la gestión ambiental y la concienciación social. Para concretar esta meta, el proyecto se estructura en objetivos específicos que inician con el análisis de los niveles de contaminación y la identificación de puntos críticos en la zona de estudio, seguidos por el diseño técnico y ensamblaje de una red de nodos sensores de bajo costo para medición de variables ambientales y meteorológicas. De igual manera, se prevé la construcción de un entorno de software que logre integrar los repositorios en la nube y una interfaz dinámica de consulta, junto con un componente analítico orientado a la emisión automatizada de advertencias.

Esta plataforma se planea para justificar la necesidad de robustecer el monitoreo municipal formal mediante herramientas de alta granularidad espacial que optimicen la capacidad de acción frente al deterioro ambiental. En el plano técnico, la integración de estas herramientas facilita el despliegue de una arquitectura económica y modular, apta para extenderse hacia otros cuadrantes urbanos y capacitada para el tratamiento eficiente de flujos cronológicos masivos. En el ámbito social, la iniciativa impulsa la ciencia comunitaria al transparentar los índices de polución y fomentar la concienciación ecológica. Para sincronizarse con las políticas nacionales de adaptación climática, la solución no solo introduce innovación y tecnología, sino que aporta a la consolidación de espacios urbanos sostenibles y seguros para los habitantes de Quito.

El alcance de este trabajo comprende el desarrollo integral de una solución tecnológica que abarca desde el diseño de hardware hasta la implementación de servicios en la nube para el monitoreo ambiental. Se incluye el ensamblaje de nodos sensores funcionales con capacidad para medir contaminantes como $PM_{2.5}$, PM_{10} , $PM_{1.0}$, CO_2 y CO , garantizando la conectividad inalámbrica de dichos dispositivos hacia una infraestructura backend donde los datos serán procesados y almacenados. El proyecto contempla también la creación de una interfaz de usuario interactiva con gráficas, además de la lógica para el

envío de alertas automáticas basadas en los límites de la OMS. El despliegue final se limitará a una fase piloto en el barrio “El Rosario”, donde se validará la precisión, resiliencia y estabilidad del sistema bajo condiciones urbanas reales durante el periodo de prueba establecido.

1.2. Antecedentes

El monitoreo de la calidad del aire se ha fortalecido como una prioridad mundial debido al aumento de enfermedades asociadas a la exposición prolongada a los contaminantes atmosféricos. La Organización Mundial de la Salud, más de un 90% de la población de entornos urbanos respira un aire que supera los límites recomendados para los contaminantes atmosféricos: $PM_{2.5}$, NO_2 y O_3 . Estos están correlacionados con enfermedades respiratorias, neurodegenerativas y cardiovasculares (World Health Organization, 2024). Presentado en este contexto los sistemas tradicionales basados en estaciones de monitoreo de referencia demostraron ser importantes para generar datos confiables; sin embargo, presentan ciertas limitaciones cuando se habla de cobertura, costo y escalabilidad. La instalación de estos requiere infraestructura especializada, personal técnico y costos elevados de mantenimiento, lo cual resulta complejo para ciudades de países en desarrollo (Argyropoulos et al., 2024).

El avance que ha tenido el Internet de las Cosas (IoT) y la aparición de sensores de bajo costo, por ende, accesible han logrado una transformación relevante en la forma en la que se monitorea la contaminación atmosférica. Estos dispositivos al ser económicos, compactos y eficientes energéticamente pueden desplegarse en grandes cantidades para construir redes para monitoreo densas capaces de ofrecer información dinámica y distribuida (Guerrón et al., 2023). Esta transición ha impulsado el desarrollar nuevos proyectos en ciudades como Valencia, Santiago, Bogotá y Ciudad de México, en donde se han implementado nodos IoT para poder hacer mediciones de contaminantes, apoyando procesos de gobernanza ambiental y permitiendo que exista un empoderamiento ciudadano a través de la ciencia abierta.

En nuestro país, Ecuador, los responsables de la vigilancia oficial de la calidad del aire son autoridades municipales y el Ministerio del Ambiente, Agua y Transición Ecológica. Aunque se dispone de información sólida y valiosa, las redes de monitoreo oficiales como REMMAQ en Quito, donde se realizará el proyecto, se encuentran conformadas por un número bajo de estaciones fijas, lo que limita la capacidad para poder representar variaciones micro espaciales que se producen entre barrios o zonas urbanas con características topográficas diferentes, niveles de tráfico, densidad de población o presencia de fuentes contaminantes. La falta de granularidad hace que la toma de decisiones preventivas sea difícil, generar alertas tempranas y evaluar detalladamente el impacto de ciertas políticas públicas como restricciones vehiculares o el ordenamiento de territorios.

La literatura reciente señala que los sensores de costo asequible pueden complementar de forma efectiva a las estaciones de referencia, siempre que se usen técnicas de corrección, calibración y validación, con esto definido la precisión de los sensores ópticos electroquímicos pueden mejorar significativamente, lo que permite su uso en aplicaciones de monitoreo urbano, abriendo la posibilidad de generar sistemas híbridos en que sensores IoT conectados a través de WiFi o LoRaWan puedan capturar datos de forma continuas y garantizar con la calibración que estos datos sean consistentes, confiables y de calidad.

Dado que en Ecuador aún no existe una plataforma pública, abierta y distribuida que integre tanto sensores IoT de bajo costo, y los datos de la calidad de aire por zonas, se tiene una oportunidad para desarrollar sistemas innovadores que aprovechen tecnologías accesibles, interconexión en las nubes y contar con un lugar para presentar estos datos. Un sistema de este calibre puede apoyar la red oficial, generar datos por zonas, fortalecer la participación de ciudadanos y, sobre todo, promover las prácticas de planificación urbana informadas. De esta forma, los sistemas IoT basados en sensores ambientales no solo representan una solución viable a nivel tecnológico, sino también una herramienta estratégica para crear una sociedad sostenible y ser resilientes ambientalmente en el barrio “El Rosario” en Quito y a futuro en otras ciudades del país.

1.3. Planteamiento del Problema

La contaminación atmosférica en la Ciudad de Quito representa una de las problemáticas ambientales más relevantes de los últimos años, afectando de forma directa la salud de la población urbana y deteriorando el ambiente. Según el ministerio del Ambiente, Agua y Transición Ecológica (2023), los niveles de partículas en suspensión (PM2.5 Y PM10) superan en varias zonas los límites establecidos por la Organización Mundial de la Salud (2023), especialmente en sectores de alta densidad vehicular. Actualmente existen estaciones de monitoreo que son gestionadas por el Municipio, la cobertura geográfica es limitada y no nos permite disponer de información en tiempo real ni a nivel de micro zonas urbanas.

Frente a esta necesidad, se identifica la necesidad de desplegar una arquitectura IoT distribuida que permita medir, transmitir y analizar parámetros ambientales en tiempo real, procesando la información en un servidor central para su visualización. A partir de este contexto, se plantea la siguiente pregunta de investigación: ¿Cómo puede un sistema IoT de bajo costo contribuir al monitoreo, análisis y gestión de la calidad del aire en Quito de forma continua y accesible para autoridades y ciudadanía?

1.4. Justificación

La contaminación del aire en Quito constituye un problema prioritario de salud pública y gestión ambiental. A pesar de los esfuerzos realizados por la REMMAQ, la infraestructura actual cuenta con retos de escalabilidad debido a altos costos operativos y mantenimiento especializado, lo cual impide un monitoreo granular en barrios o zonas críticas.

El desarrollo de este sistema IoT se justifica como una solución tecnológica sostenible que complementa la red oficial, ampliando la cobertura mediante sensores distribuidos y de fácil mantenimiento. Al capturar mediciones de PM2.5, PM10, Gases como CO2, junto con variables meteorológicas en el barrio "El Rosario", se mejora notablemente la resolución espacial de los datos. Además, la centralización de esta información en una aplicación web accesible democratiza los datos ambientales, otorgando a los usuarios

alertas automáticas y tendencias históricas.

El proyecto presenta su viabilidad en la integración de tecnologías emergentes, con un enfoque técnico al 60%, innovador del 25% e investigativo del 15%, con esto no solo se busca fortalecer la gestión urbana inteligente, sino también fomentar la conciencia ciudadana en concordancia con los Objetivos de Desarrollo Sostenible 11 y 13 de la Agenda 2030 (Alsamrai et al., 2024).

1.4.1. Componente de Investigación

Este componente es parte del 15% del proyecto, se va a centrar en obtener el conocimiento necesario para poder garantizar la precisión, fiabilidad y escalabilidad del sistema IoT propuesto. Para esto se realizará una revisión y selección de sensores de bajo costo, que sean los más apropiados y se encuentren disponibles en el mercado para poder medir con precisión las partículas finas PM2.5 Y PM10, gases contaminantes CO2 y CO a la mano de variables meteorológicas como humedad y temperatura. Dentro de este componente se deberá evaluar el rendimiento, calibración y vida útil de los sensores y cuál es su uso en sistemas IoT similares. También. Se investigarán los protocolos a usar para la comunicación inalámbrica que sean más eficientes como LoRa o Wi-Fi para garantizar una transmisión de datos en tiempo real y con un consumo bajo de energía desde el barrio "El Rosario" al servidor central. Finalmente, la investigación abarcará el marco normativo de calidad del aire, analizando los límites establecidos por la OMS y la normativa local del ministerio del Ambiente, Agua y Transición Ecológica para poder definir los umbrales precisos para el módulo de alertas.

1.4.2. Componente de Innovación

La innovación del proyecto será del 25%, radicará en ofrecer una solución complementaria y poder granular al sistema de monitoreo actual de Quito, la REMMAQ, que depende de estaciones costosas y son fijas, con una cobertura limitada. La propuesta utiliza una red de sensores IoT distribuidos y de bajo costo para poder obtener mediciones en tiempo real a nivel de micro zonas urbanas, inicialmente en el barrio "El Rosario", lo cual permite detectar variaciones locales y puntos críticos de contaminación

que el sistema actual no logra captar.

Adicionalmente, la plataforma desarrollada no solo visualizará datos, sino que integrará un módulo de análisis y alertas automáticas basado en umbrales, fomentando la conciencia ciudadana y la participación de todos en la gestión ambiental, en alineación con los Objetivos de Desarrollo Sostenible 11 y 13.

1.4.3. Componente Técnico

El componente técnico es el 60% del proyecto, este se centra en el diseñar e implementar la arquitectura IoT completa. Esta arquitectura comprende tres subsistemas principales: La capa de hardware, que incluye los nodos sensores de bajo costo para la captura de PM2.5, PM10, CO2, CO, temperatura y humedad, usando comunicación inalámbrica para poder transmitir las mediciones en tiempo real; el servidor central o cloud, responsable del almacenamiento, procesamiento y validación de los datos recibidos de la red; y la plataforma web, que actúa como interfaz de usuario para la visualización en tiempo real de la calidad del aire por sectores, e integra el módulo de alertas y reportes que procesa las tendencias históricas y genera notificaciones automáticas ante niveles riesgosos.

1.5. Objetivos

Objetivo General

Diseñar e implementar un sistema IoT para el monitoreo inteligente de la calidad del aire en zonas urbanas de Quito que permita recopilar y visualizar datos ambientales en tiempo real para apoyar la gestión ambiental y la concienciación ciudadana.

Objetivo Específicos

- Analizar los niveles actuales de contaminación del aire y los puntos críticos de la ciudad de Quito.
- Diseñar una red IoT de bajo costo para medir parámetros ambientales relevantes.
- Desarrollar una plataforma web para la visualización en tiempo real de los datos recolectados.
- Integrar un módulo de alertas y recomendaciones automáticas basadas en los niveles detectados.

Capítulo dos

2. Marco Teórico

2.1. Contaminación Atmosférica y Parámetros de Calidad del Aire

La contaminación ambiental se define como la introducción de agentes físicos, químicos o biológicos en el entorno que alteran desfavorablemente las condiciones naturales del ecosistema, de acuerdo con Alsamrai et al. (2024), este fenómeno no solo se limita a la atmósfera, sino que genera una degradación sistémica que afecta la salud humana y la biodiversidad. En las últimas décadas, el crecimiento demográfico y la urbanización han acelerado la liberación de subproductos industriales y vehiculares, convirtiendo la gestión ambiental en una prioridad tecnológica global (Hassan et al., 2024)

La contaminación del aire también está compuesta por gases que se presentarán a continuación y los mismos serán objeto de monitoreo en el proyecto de titulación:

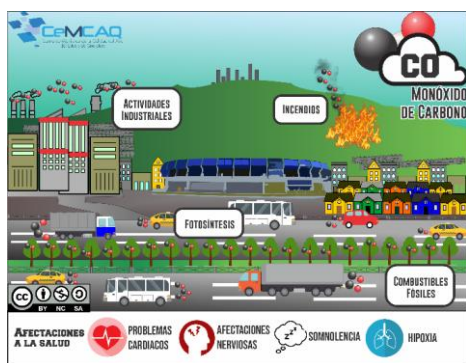
2.1.1. Monóxido de Carbono (CO)

El monóxido de Carbono se define como un compuesto gaseoso organolépticamente indetectable, debido a que carece de color, olor o sabor, y no presenta una propiedad irritante. Según la Agencia de Protección Ambiental (EPA, 2025), este compuesto se encuentra presente tanto en ambientes interiores como en la atmósfera exterior, y su peligrosidad está radicada en que es capaz de alcanzar niveles letales en el hogar antes de que los ocupantes perciban su presencia. Al respecto, la Organización Mundial de la Salud (OMS, 2021) establece que, aunque el CO es un componente de traza natural, su concentración por encima de los límites normales en zonas urbanas es un indicador crítico de combustión incompleta y un riesgo severo para la salud cardiovascular.

Este gas posee diversas fuentes de emisión, en la atmósfera el CO proviene de procesos naturales como incendios forestales, la degradación de la clorofila y la descomposición de materia orgánica. No obstante, la mayor carga contaminante deriva de fuentes antropogénicas. Como señala Argyropoulos et al., (2024), estas incluyen la oxidación de hidrocarburos, procesos industriales térmicos, el uso de sistemas de calefacción deficientes y, fundamentalmente la combustión incompleta en motores de vehículos que utilizan combustibles fósiles. En la Figura 1 se detallan estas fuentes y su impacto en la calidad del aire urbano.

Figura 1

Origen del Monóxido de Carbono



Nota. Adaptado de CeMCAQ y su definición de CO. (<https://aire.cemcaq.mx/monoxido-de-carbono/>).

CC BY 2.0.

2.1.2. Dióxido de Carbono (CO_2)

El Dióxido de Carbono (CO_2) es un gas incoloro e inodoro que, aunque es un componente que está presente en la atmósfera de la tierra y es necesario para nuestro ciclo de vida, se ha convertido en un parámetro crítico de monitoreo debido a su papel como gas de efecto invernadero y su impacto en la calidad del aire interior. Según Alsamrai et al. (2024), el CO_2 es el indicador principal de la tasa de ventilación en edificios; concentraciones elevadas sugieren una acumulación de bio-efluentes humanos y una renovación de aire insuficiente.

A nivel técnico, Dubey et al. (2024) explican que la medición del CO_2 mediante la tecnología de bajo costo ha tenido una evolución en el uso de sensores Infrarrojos No Dispersivos (NDIR). Este método se basa en la propiedad de las moléculas de CO_2 de absorber luz roja infrarroja en una longitud de onda específica (4.26 μm), permitiendo una cuantificación precisa que supera en estabilidad a los sensores electroquímicos tradicionales.

Tabla 1

Efectos de la concentración de CO_2 en la salud y rendimiento

Concentración	Clasificación del aire	Efectos Observados	Referencias
400-450 ppm	Aire exterior Limpio	Nivel Basal normal	(Alsamrai et al., 2024)
600-1000 ppm	Aire interior Bueno	Confort técnico y respiratorio	(Organización Mundial de la Salud (OMS), 2021)
1000-2000 ppm	Aire viciado	Somnolencia, Falta de atención	(Dubey et al., 2024)
2000-5000 ppm	Aire muy contaminado	Cefaleas, náuseas, aumento de frecuencia cardíaca	(Argyropoulos et al., 2024)
> 5000 ppm	Limite ocupacional	Riesgo de asfixia en exposición prolongada	(EPA, 2025)

Nota. En la tabla se observan los niveles de concentración en el aire basado en la revisión de literatura técnica y normas de salud pública.

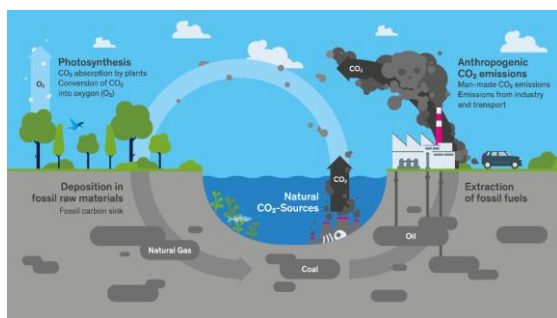
Las fuentes y niveles de concentración se clasifican según el entorno de la siguiente manera:

Exteriores: los niveles basales globales actualmente están dentro de 410-420 ppm, incrementándose en zonas urbanas densas debido a la quema de combustibles fósiles (Argyropoulos et al., 2024).

Interiores: La principal fuente es la respiración humana por su propio metabolismo. De acuerdo con las actualizaciones de la Organización Mundial de la Salud (OMS), (2021) y estándares de ventilación modernos, niveles superiores a las 1000 ppm se asocian con el síndrome del “edificio enfermo”, dentro de la Figura 2 Se puede observar como el ciclo de contaminación por Dióxido de Carbono se presenta.

Figura 2

Ciclo de contaminación del Dióxido de Carbono



Nota. Adaptado de Myclimate y su definición de CO₂.

(<https://www.myclimate.org/en/information/faq/faq-detail/what-is-co2-and-where-does-it-come-from/0>). CC BY 2.0.

2.1.3. Material Particulado (PM2.5 y PM10)

Comentado [2]: Todas las figuras/tablas deben ser nombradas en el texto.

El material particulado o PM consiste en una mezcla compleja de partículas sólidas y líquidas suspendidas en el aire, cuya peligrosidad está directamente relacionada con su diámetro aerodinámico. Según Balagopal et al. (2025), estas se clasifican principalmente en PM10 que son partículas con un diámetro menor a $10\text{ }\mu\text{m}$ y PM2.5 que son menores a $2.5\text{ }\mu\text{m}$. Mientras que las PM10 son retenidas en las vías respiratorias superiores, las PM2.5 cuentan con la capacidad de penetrar profundamente en los alvéolos pulmonares y translocarse al sistema circulatorio, provocando efectos sistémicos adversos (Argyropoulos et al., 2024).

Técnicamente, el monitoreo de PM en estaciones de bajo costo se realiza a través de sensores ópticos basados en la dispersión de luz láser o *Light Scattering*. De acuerdo con Balagopal et al. (2025), el sensor cuantifica el número de partículas y en base a esto hace un estimado de su masa, aunque este proceso es altamente sensible a la higroscopicidad (humedad), lo que requiere algoritmos de calibración avanzada para evitar lecturas sobrestimadas. Dentro de la implementación en el prototipo, el PM2.5 constituye el parámetro de mayor relevancia técnica. Ya que el origen de este en la ciudad de Quito está ligado en su mayoría a la combustión del diésel y la abrasión de frenos o neumáticos, el uso de los sensores como el SDS011 O PMS5003 permitirá generar mapas de riesgo en tiempo real, proporcionando una herramienta de prevención para ciudadanos con enfermedades respiratorias crónicas.

Figura 3

Escala comparativa de partículas PM2.5 y PM10 y su alcance en el sistema respiratorio



Nota. Adaptado de comparación de los tamaños de partículas PM, por United States Environmental Protection Agency, 2008., Science Learning Hub. (<https://www.sciencelearn.org.nz/images/1869-size-comparisons-for-pm-particles>). CC BY 2.0.

2.1.4. Dióxido de Nitrógeno (NO_2)

El dióxido de nitrógeno (NO_2) es un gas de color pardo-amarillento, altamente reactivo y corrosivo, que pertenece a la familia de los óxidos de nitrógeno (NO_x). Según Hassan et al. (2024), la concentración de este gas en entornos metropolitanos refleja de forma fidedigna la densidad del flujo vehicular, al originarse principalmente por procesos térmicos a elevadas temperaturas en motores de combustión interna.

Argyropoulos et al. (2024) destacan que el (NO_2) llega a constituir como un agente severo e irritante para el tracto respiratorio, desempeñando además un papel catalizador en la génesis de contaminantes secundarios como el Ozono troposférico y el smog fotoquímico. Su monitoreo es fundamental dado que exposiciones breves pueden exacerbar cuadros de asma y disminuir la función pulmonar.

El valor actual para que los seres humanos estén seguros de este contaminante según la OMS se representa en la tabla 2:

Tabla 2

Guía de la OMS para el Dióxido de Nitrógeno

Guía de calidad del Aire para el Dióxido de Nitrógeno			
Media Anual	Media de una hora	Nombre de Unidad	Símbolo de Unidad
40	200	Microgramos / Metro Cúbico	$\mu\text{g} / \text{m}^3$

Nota. En la tabla se observan los niveles de medias para el Dióxido de Nitrógeno dados por Organización Mundial de la Salud (OMS), (2021).

2.1.5. Ozono Troposférico (O_3)

A diferencia del ozono estratosférico que se encarga de la protección de la Tierra, el Ozono Troposférico (O_3) es un contaminante secundario a nivel de suelo que no se emite directamente. Según Alsamrai et al. (2024), este se produce mediante reacciones fotoquímicas complejas entre los (NO_x) y los Compuestos orgánicos Volátiles (COV) en presencia de radiación solar intensa. Este contaminante presenta una dinámica diurna muy marcada; sus niveles máximos se registran durante las horas de mayor insolación. Argyropoulos et al. (2024) subrayan que el ozono es un oxidante fuerte que daña no solamente el tejido pulmonar, sino también la vegetación y los materiales poliméricos en las ciudades.

La relación inversa entre el (NO_2) y (O_3) plantea un desafío de diseño para el presente sistema, el ozono requiere de la luz solar para formarse, el nodo sensor debe integrar una correlación de datos con la intensidad lumínica capturada. Esto permitirá verificar si los picos detectados corresponden a una dinámica atmosférica real o a interferencias cruzadas en los sensores de bajo costo, garantizando la veracidad de las alertas generadas.

En la tabla 3 se podrá observar una comparación de comportamiento químico entre el Dióxido de Nitrógeno y el Ozono Troposférico.

Tabla 3

Comparativa de comportamiento químico: (NO_2) y (O_3)

Característica	Dióxido de Nitrógeno	Ozono Troposférico
<i>Tipo de contaminante</i>	Primario / Secundario	Estrictamente Secundario
<i>Fuente predominante</i>	Escape de vehículos (combustible fósil)	Reacción fotoquímica (Sol + NO_2)
<i>Pico de concentración</i>	Horas pico de tráfico (Mañana y tarde)	Mediodía y tarde (debido a alta radiación)
<i>Efecto principal</i>	Inflamación bronquial	Estrés oxidativo y daño celular

Nota. Comparación de contaminantes (NO_2) y (O_3) con elaboración propia basado en Alsamrai et al. (2024) y Argyropoulos et al. (2024).

2.2. Fundamentos del Internet de las Cosas (IoT) y Redes de Sensores

El internet de las cosas (IoT) se puede definir hoy en día como un paradigma tecnológico disruptivo que constituye los cimientos de la arquitectura para poder automatizar y monitorear de forma avanzada los entornos físicos. Según Guerrón et al. (2023), el Internet de las Cosas representa un ecosistema de dispositivos interconectados con capacidades intrínsecas de procesamiento, captación, y transmisión de información mediante redes inalámbricas.

Esta infraestructura no solo es capaz de automatizar tareas operativas, sino que permite actuar como un puente de digitalización entre los fenómenos físicos complejos en datos analizables en tiempo real. En el ámbito del monitoreo ambiental, esta capacidad resulta imprescindible, porque nos permite democratizar la observación de contaminantes que, históricamente, dependían de estaciones oficiales de alto costo, facilitando así una vigilancia continua y ubicua del entorno.

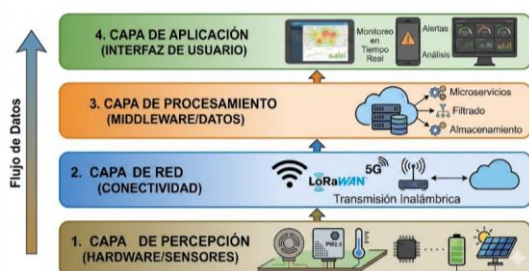
La operatividad de estos sistemas en entornos externos exige cumplir con ciertas características técnicas críticas en el hardware desplegado. Al respecto, Felici-Castell et al. (2023) subrayan que los nodos de monitoreo ambiental deben contar con una autonomía robusta, eficiencia energética alta y la capacidad de poder operara bajo condiciones climáticas variables. Estos nodos integran sensores especializados, microcontroladores y módulos de comunicación de bajo consumo que aseguran un flujo constante de datos hacia la nube. Esta arquitectura distribuida permite capturar micro variaciones locales que las estaciones fijas tradicionales suelen omitir; por ejemplo, la detección de picos de polución en zonas de tráfico denso como el barrio “El Rosario”, o en sectores industriales específicos, donde la resolución espacial del dato es determinante para la toma de decisiones ambientales.

Por último, la eficacia del Internet de las Cosas dentro del contexto ecológico reside en su naturaleza colaborativa y descentralizada. Argyropoulos et al. (2024) explican que las redes de sensores inalámbricos (WSN) permiten una captura de información distribuida que fortalece la resiliencia del sistema global. Dentro de este enfoque, la falla individual de un nodo no compromete la integridad de la red, permitiendo que el sistema mantenga su operatividad ante contingencias técnicas o ambientales. Esta topología descentralizada es ideal para proyectos urbanos de gestión ambiental en donde la heterogeneidad de la infraestructura y las variables del entorno requieren un sistema flexible, escalable y, sobre todo, capaz de generar un inventario digital preciso del estado del medio ambiente en tiempo real.

En la figura 4 se puede observar las capas dentro del Internet de las Cosas y el flujo de datos de este, desde la detección hasta que el usuario final puede ver los mismos.

Figura 4

Capas del IoT



Nota. Hecho con Nano Banana Pro, adaptado de la explicación de las capas del IoT, 2026. CC BY 2.0.

2.2.1. Diseño y componentes del sistema de monitoreo ambiental

La implementación del sistema requiere una arquitectura de hardware robusta y modular. Según la metodología de Guerrón et al. (2023), el nodo se divide en cuatro unidades funcionales: Procesamiento, captación, comunicación y alimentación.

2.2.2. Microcontrolador

El núcleo del nodo está constituido por un microcontrolador que actúa como el gestor de tareas. En el presente proyecto se opta en utilizar el dispositivo Arduino ESP32S como se puede ver en la figura 5. Al contar con la capacidad de manejar múltiples entradas analógicas y digitales de forma simultánea. Guerrón et al. (2023) destacan que estas unidades deben poseer una velocidad de reloj suficiente para poder procesar los algoritmos de calibración local antes de transmitir el dato. Esta unidad es la encargada de transformar las señales de voltaje de los sensores en valores digitales de concentración ya sea en ppm o $\mu g / m^3$.

Figura 5

Arduino ESP32S NodeMCU



Nota. Adaptado de Mercado Libre como Publicación de venta del ESP23S. Mercado Libre, 2026. (<https://www.mercadolibre.com.ec/esp32-nodemcu-modulo-wifi-bluetooth-42-38-pines-con-usb-c-wroom-esp32s-placa-de-desarrollo-de-iot-controlador-wifi-compatible-con-arduino-y-micro-python/p/MEC42485623>). CC BY 2.0.

2.2.3. Sensores IoT

La instrumentación sensorial constituye el núcleo crítico de cualquier arquitectura basada en el Internet de las Cosas (IoT), actuando como la interfaz primaria entre el entorno físico y el ecosistema digital. Estos componentes son indispensables para la generación de flujos de datos fidedignos, ya que la validez de los análisis posteriores es dependiente de la precisión con la que se capturen las magnitudes ambientales de forma directa. El prototipo permite transformar fenómenos analógicos en información digital procesable en base a la función de los objetivos de este, los sensores permiten transformar fenómenos analógicos en información digital procesable. Para efectos del presente trabajo, se presentarán los sensores destinados al monitoreo de la calidad del aire, los cuales fueron seleccionados por la capacidad de respuesta ante contaminantes específicos en zonas urbanas.

Sensor MH-Z19C: para la evaluación de la ventilación ambiental y la densidad de emisiones vehiculares, se considera el uso del sensor MH-Z19C. Este componente descarta metodologías químicas imprecisas a favor de la tecnología NDIR Infrarrojo No Dispersivo. Su funcionamiento se basa en la ley de Beer-Lambert: el módulo emite un haz de luz infrarroja a través de una cámara de aire; dado que las moléculas de CO_2 absorben una longitud de onda altamente específica de aproximadamente $4.26 \mu m$, el sensor mide matemáticamente la atenuación de la luz para determinar la concentración del gas. Su alta selectividad hace que sea la mejor elección para el presente trabajo de titulación, ya que no se ve afectado por la presencia de otros gases o vapores en una vía de alto tráfico y su algoritmo de calibración automática de la línea base, lo que garantiza mediciones exactas en partes por millón ppm a lo largo del tiempo sin requerir mantenimiento manual del nodo. En la figura 6 se puede observar al sensor MH-Z19C.

Figura 6

Sensor MH-Z19C



Nota. Adaptado de Rev Space como Publicación del Sensor de calidad de aire MH-Z19C.Rev Space, s.f. (<https://revspace.nl/MHZ19>). CC BY 2.0.

Las características del sensor se incluyen en la tabla 4:

Tabla 4

Características del sensor MH-Z19C

Característica	MH-Z19C
<i>Voltaje de Alimentación</i>	$5V \pm 0.1DC$
<i>Señal de Salida</i>	Digital, PWM y Analógica
<i>Tipo de Sensor</i>	(NDIR) óptico Infrarrojo No Dispersivo
<i>Tiempo de respuesta</i>	< 120 segundos
<i>Unidad</i>	ppm (partes por millón)
<i>Temperatura de operación</i>	-10 °C~50 °C.
<i>Tamaño</i>	30 mm x 20 mm x 9 mm

Nota. Tabla de características recopiladas de Tinitronics, s.f. Tinytronics.
(<https://www.tinytronics.nl/en/sensors/air/gas/winsen-mh-z19c-co2-sensor-with-cable>)

Sensor PMS5003: El monitoreo de partículas suspendidas es el eje central para determinar el riesgo respiratorio, por ello el sensor óptico Plantower PMS5003 es el encargado de recibir los datos de material particulado en el ambiente. Como contraste con los dispositivos que calculan la polución mediante la atenuación u opacidad lumínica básica, esta unidad instrumental se fundamenta en el principio físico de dispersión por haz láser (Laser Scattering). El mecanismo integra un micro ventilador de flujo continuo que conduce el aire circundante hacia una cavidad óptica interna, dentro de la cual un diodo emite luz coherente sobre las partículas suspendidas. Posteriormente, un elemento fotodetector registra los pulsos de radiación refractada, permitiendo que un procesador embebido aplique los postulados matemáticos de la teoría de dispersión de Mie para determinar con precisión tanto al diámetro aerodinámico como la concentración en masa del material particulado.

Desde el punto de vista epidemiológico, este sensor es crítico ya que permite separar las métricas, cuantificando con precisión el material PM10 el cual es el que se deposita dentro del tracto bronquial y el material PM2.5 el cual son partículas finas capaces de atravesar la barrera alveolar e ingresar al torrente sanguíneo. En la figura 7 se puede visualizar físicamente el sensor.

Figura 7

Sensor PMS5003



Nota. Adaptado de Solectro. Solectro, s.f. (<https://solectroshop.com/es/sensores-calidad-del-aire/1195-sensor-pms5003-de-concentracion-de-particulas-alta-precision-5905323237360.html?srsId=AfmBOor2vNgcEjXzl7MrVJkSEaQuVorH-EFWYmiEYAD8kB9Uj-5AwicI>). CC BY 2.0.

El sensor cuenta con las siguientes características presentadas en la tabla 5:

Tabla 5

Características técnicas del sensor PMS5003

Característica	PMS5003
<i>Voltaje de Alimentación</i>	$5V \pm 0.1DC$
<i>Señal de Salida</i>	Digital
<i>Tipo de Sensor</i>	Óptico por dispersión laser
<i>Unidad</i>	$\mu g / m^3$ microgramo por metro cúbico
<i>Tiempo de respuesta</i>	≤ 10 segundos
<i>Temperatura de operación</i>	-10 °C~60 °C.
<i>Tamaño</i>	50 mm x 38 cm x 21 cm

Nota. Tabla de características recopiladas de Solectro, s.f. Solectro.

(<https://solectroshop.com/es/sensores-calidad-del-aire/1195-sensor-pms5003-de-concentracion-de-particulas-alta-precision-5905323237360.html?srsId=AfmBOor2vNgcEjXzI7MrVJkSEaQuVorH-EFWYmiEYAD8kB9Uj-5AwicI>)

Módulo DHT22: Este dispositivo constituye un transductor digital de alta precisión diseñado para la monitorear variables termodinámicas ambientales. A diferencia de sensores básicos, el DHT22 integra un sensor capacitivo de humedad y un termistor NTC, proporcionando una señal digital pre-calibrada que asegura una alta fiabilidad y estabilidad a largo plazo. Según Felici-Castell et al. (2023), la integración de este módulo es mandatorio en nodos de monitoreo de aire, ya que los datos de temperatura y humedad son fundamentales para aplicar algoritmos de compensación sobre los sensores de gases (CO y NO_2) y material particulado, los cuales presentan derivas significativas ante cambios en las condiciones higrotérmicas del entorno (ver figura 10).

Figura 10

Módulo DHT22



Nota. Adaptado de Mercado Libre como Publicación de venta del s. Mercado Libre, 2026.

(<https://novatronicec.com/index.php/product/dht22-am2302-sensor-capacitivo-de-temperatura-y-humedad-digital-copia/>). CC BY 2.0.

La tabla 8 cuenta con las características del sensor DHT22.

Tabla 8*Características del sensor DHT22*

Característica	DHT22
<i>Voltaje de Alimentación</i>	3.3. V A 6V
<i>Señal de Salida</i>	Digital
<i>Corriente de consumo</i>	Máximo 2.5 mA
<i>Unidad</i>	°C y %
<i>Tamaño</i>	20 x 15 x 8 mm

Nota. Tabla de características recopiladas de Aosong Electronics, Aosong Electronics 2016.

(<https://cdn-shop.adafruit.com/datasheets/DHT22.pdf>)

1.3. Conectividad y Protocolos de Comunicación en el Ecosistema IoT

La conectividad representa el eje que articula al Internet de las Cosas, se encarga del transporte de las tramas de datos desde la capa de percepción hacia los servicios de almacenamiento en la nube, según Guerrón et al. (2023), la selección del protocolo de comunicación no solo depende de la disponibilidad de la red en la zona de despliegue, sino también de las restricciones de consumo energético y el volumen de datos a transmitir. En sistemas de monitoreo ambiental, donde la latencia no es tan crítica como la integridad del dato, se busca priorizar tecnologías que garanticen una conexión robusta en entornos urbanos densos.

1.3.1. Tecnologías de red inalámbrica

- **Wi-Fi (IEEE 802.11):**

Constituye el medio de conectividad predilecto en escenarios metropolitanos que cuentan con instalaciones de red preexistentes. Ofrece una elevada capacidad en la tasa de transferencia de datos, garantizando una entrega ágil de la información, si bien demanda un gasto de potencia eléctrica mayor en comparación con otras alternativas de comunicación inalámbrica (Alsamrai et al., 2024).

- **LoRaWAN (Long Range Wide Area Network):**

Se posiciona como una de las alternativas tecnológicas óptimas para el despliegue de cobertura extensa. Según los hallazgos de Balagopal et al. (2025), esta topología de red facilita la transmisión a distancias de múltiples kilómetros manteniendo un consumo de energía sumamente reducido, convirtiéndose en la opción adecuada para nodos autónomos alimentados por baterías en zonas con señal Wi-Fi deficiente o nula.

- **Redes celulares (4G / 5G):**

Utilizadas principalmente para estaciones que requieren movilidad o transmisión en tiempo real de alta densidad (Felici-Castell et al., 2023).

1.3.2. Protocolo de mensajería MQTT

Más allá del medio físico de transmisión, se requiere un protocolo a nivel de aplicación que sea ligero. El estándar dentro de la industria de Internet de las Cosas es MQTT (Message Queuing Telemetry Transport), el cual se define como un sistema de mensajería asíncrona diseñado para dispositivos con recursos limitados y redes de baja capacidad. Según Hassan et al. (2024), MQTT minimiza el encabezado de los paquetes de datos, reduciendo drásticamente el consumo de ancho de banda y asegurando que la información de los sensores ambientales llegue al servidor incluso bajo condiciones de redes inestables. Este protocolo tiene un origen industrial profundo; el protocolo MQTT fue diseñado originalmente en 1999 por Andy Stanford-Clarck de IBM y Arlen Nipper de Eurotech, con el fin de optimizar las comunicaciones satelitales en la industria petrolera (**MQTT Version 3.1.1, n.d.**). Esta herencia de eficiencia es la que permite que hoy sea aplicado con éxito en el monitoreo ambiental, donde la estabilidad de la red pueda que sea variable por el entorno urbano.

A diferencia del modelo tradicional Cliente-Servidor (donde el cliente debe solicitar activamente la información), MQTT opera bajo un paradigma de publicación/suscripción (Pub/Sub). Este paradigma desacopla completamente al emisor del receptor con un agente central denominado Broker.

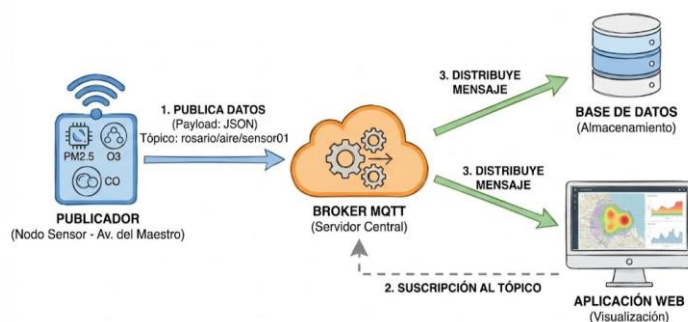
Dicho Broker actúa como el centro neurálgico que recibe todos los mensajes, los filtra y los distribuye a los nodos interesados. Su función es crítica para la escalabilidad de la red, ya que permite que múltiples dispositivos reciban la información sin que el nodo sensor tenga que gestionar múltiples conexiones simultáneas (Guerrón et al., 2023). Si hablamos de la calidad del servicio dentro del protocolo, MQTT nos permite definir niveles de fiabilidad en la entrega de mensajes, según Alsamrai et al. (2024), el uso de niveles de QoS (calidad del mensaje) garantiza que en caso de pérdida temporal de señal del nodo sensor, el sistema permita retransmitir los datos de contaminantes críticos (Como niveles de ozono o monóxido de carbono) una vez que la conexión se restablezca.

Para la implementación técnica en este proyecto, el Broker que actúa como servidor de registro es Eclipse Mosquitto. Según Light. (2017) "Mosquitto es un bróker de mensajes que ayuda a implementar el protocolo MQTT proporcionando un servidor que puede recibir y filtrar mensajes para luego distribuirlos a los clientes interesados". Dicha solución open source destaca por ser sólido y ligero, logrando procesar flujos masivos de telemetría instantánea con un rendimiento óptimo. De forma complementaria, el acoplamiento con la placa de desarrollo se estructura dentro del ecosistema de Arduino mediante la biblioteca PubSubClient. Este recurso cumple una función intermediaria indispensable al permitir que el microprocesador serialice las variables ambientales en objetos JSON y gestione su despacho autónomo y continuo hacia el nodo central o broker. El uso de esta librería especializada es vital ya que, como señalan Hassan et al. (2024), permite que dispositivos con recursos de procesamiento limitados mantengan una conexión constante con la nube, transformando voltajes analógicos en mensajes digitales que viajan a través de la red wifi en milisegundos.

Finalmente, la fiabilidad de la comunicación se garantiza a través del mecanismo de calidad del servicio QoS, el cual permite definir niveles de garantía en la entrega de la información en relación con la importancia que tienen. Según Alsamrai et al. (2024) el protocolo permite la configuración tres niveles de confirmación: QoS 0, en donde el mensaje se envía una vez sin confirmar, lo cual es ideal para datos frecuentes del clima; QoS 1, que asegura que el mensaje llegue al menos una vez al destino; y QoS 2, que garantiza una entrega única y exacta para datos críticos. Esta flexibilidad asegura que, ante una pérdida temporal de señal en el nodo sensor, el sistema permita retransmitir los datos de contaminantes críticos como niveles de monóxido de carbono u ozono, una vez que la conexión se reestablezca. De esta manera, MQTT no solo trabaja como un medio de transporte, sino como una capa de inteligencia que asegura la integridad de los datos para el análisis histórico y la generación de alertas de salud pública en beneficio de la comunidad.

Figura 11

Modelo de pub/sub dentro de MQTT para uso de sensores IoT



Nota. Hecho con Nano Banana Pro, adaptado de la explicación del funcionamiento pub/sub dentro de MQTT, 2026. CC BY 2.0.

1.3.3. Estructura y dinámica operativa del protocolo MQTT

El protocolo MQTT basa su eficiencia en una arquitectura desacoplada y un formato de mensaje minimalista, lo que reduce la sobrecarga de la red. Según Hassan et al. (2024), esta estructura permite que dispositivos con baja capacidad de procesamiento, como los nodos puedan mantener flujos de datos constantes con un consumo energético al mínimo.

Estructura de la arquitectura

Se organiza mediante los tres actores principales:

- **Cliente:**

en este proyecto, el microcontrolador actúa como el cliente publicador. Su función es capturar los datos de los sensores ($PM_{2.5}$, CO , O_3) y enviarlos al servidor.

- **Broker:**

Es el servidor central, por ejemplo, AWS IoT, es el encargado de gestionar el tráfico. Su responsabilidad es recibir los mensajes y distribuirlos únicamente a los clientes suscritos como la aplicación web o la base de datos.

- **Tópico:**

Es la estructura de direccionamiento lógica. (Guerrón et al., 2023) señalan que una jerarquía de tópicos bien definida, por ejemplo, v1/rosario/monitoreo/aire, es vital al momento de querer ver el tema de la escalabilidad del sistema, lo que permite que organicemos la información por ubicación o tipo de contaminante.

Estructura del Paquete de datos

Un mensaje MQTT se compone de tres partes esenciales que garantizan que la carga útil sea ligera.

- **Cabecera fija:**

Ocupa solamente 2 Bytes y contiene el tipo de paquete y la longitud del mensaje.

- **Cabecera variable:**

Contiene información adicional como el identificador del paquete o el nombre del tópico.

- **Carga útil:**

Es el contenido real del mensaje. En el presente trabajo de titulación la carga útil va a consistir en una cadena en formato JSON que agrupa las lecturas de los sensores.

Acciones y paquetes de control

La interacción entre el nodo y el bróker se rige por una serie de acciones o métodos ya definidos dentro de la especificación original de Stanford-Clark y Nipper. (1999). Estas acciones aseguran que la conexión sea fiable:

Tabla 9

Acciones de MQTT

Acción	Función Técnica	Flujo
<i>CONNECT</i>	El nodo solicita la apertura de una sesión y el bróker confirma la conexión	Cliente -> Broker
<i>PUBLISH</i>	Acción mediante la cual el nodo envía los datos de calidad del aire al bróker	Cliente <-> Broker
<i>SUBSCRIBE</i>	La aplicación web solicita escuchar un tópico específico para actualizar los gráficos en tiempo real	Aplicación -> Broker
<i>PINGREQ</i>	Paquetes enviados periódicamente para confirmar que el nodo en el barrio "El Rosario" sigue activo, incluso si no está transmitiendo datos en ese instante	Broker -> Cliente
<i>DISCONNECT</i>	Cierre formal de la sesión para poder liberar los recursos en el bróker	Cliente -> Broker

Nota. Elaboración propia basada en Hassan et al. (2024).

1.3.4. Infraestructura de Transporte y Estructuración de Datos

Una vez definidos los protocolos de aplicación como lo es MQTT, es necesario analizar las tecnologías de transporte y los formatos de intercambio que garantizan que la información llegue de forma íntegra y legible al usuario final.

1.3.5. Protocolo de Control de Transmisión (TCP)

El protocolo TCP (Transmission Control Protocol) actúa como la base de transporte para MQTT y otras aplicaciones críticas del sistema. En contraste con los esquemas de comunicación no orientados a conexión, el protocolo TCP asegura la entrega íntegra, secuencial y libre de anomalías en los paquetes de información mediante el uso sistemático de confirmaciones de recepción (acknowledgment).

Al respecto, Guerrón et al., (2023) destacan que la adopción de TCP resulta determinante en plataformas de vigilancia ecológica para salvaguardar la fidelidad de las tramas transmitidas. En entornos urbanos donde el ruido electromagnético suele degradar los enlaces inalámbricos, este protocolo administra la reemisión de los segmentos extraviados, garantizando que los registros de gases y material particulado alcancen el servidor central sin sufrir alteraciones ni pérdidas durante el tránsito de red.

TCP cuenta con cinco niveles dentro de su estructura, cada uno de ellos cumple una función específica, encapsulando la información de los sensores en diferentes estructuras de datos a medida que desciende en la jerarquía.

- **Capa de Aplicación**

Es el nivel superior donde el software y los protocolos de usuario interactúan directamente con los datos. En el presente proyecto es aquí donde residen los protocolos MQTT y WebSockets. Como señalan Hassan et al. (2024), esta capa es la encargada de traducir en bits la información y que puedan tener un significado; por ejemplo, permite que un usuario pueda visualizar los niveles de ozono de forma legible en una gráfica, abstrayendo toda la complejidad técnica de la red subyacente. Los datos del sensor SEN0460 se formatean aquí, generalmente en JSON antes de empezar el viaje por la red.

- **Capa de transporte**

Es la encargada de la comunicación de extremo a extremo y del control del flujo. Aquí opera TCP para garantizar que no se pierdan datos importantes de los sensores y UDP para priorizar la velocidad sobre la fiabilidad.

A diferencia de otros protocolos, TCP está orientado a la conexión. Antes de enviar datos, establece un diálogo inicial llamado 3-way handshake, esto permite asegurar que el servidor esté listo. Además, numera cada paquete; si un dato de contaminación se pierde en el camino, TCP lo detecta y solicita que se envíe de nuevo.

Esta capa utiliza puertos para poder dirigir el tráfico hacia la aplicación correcta; por ejemplo, el puerto 1883 es el estándar para la comunicación MQTT. Mientras TCP prioriza la integridad de los datos de los sensores, se puede emplear UDP en los casos donde se priorice la velocidad extrema sobre una fiabilidad total.

- **Capa de Red o Internet**

Esta capa tiene como misión el direccionamiento y el enrutamiento de los paquetes a través de diferentes redes mediante el protocolo IP, el cual actúa como identificador lógico global para cada dispositivo en la red. Según **Wetherall D. & Tanenbaum A. (2012)**, “La función del protocolo IP es proporcionar un mecanismo de entrega de paquetes lo mejor posible (best-effort) desde una fuente hasta un destino, sin importar si estas máquinas están en la misma red o en redes diferentes” (p.443). dentro del contexto del sistema de monitoreo, el protocolo IP ya sea en su versión IPVv4 o IPV6 permite que cada nodo sensor sea identificado de manera única. Asegurando que los paquetes de datos encuentren la ruta más eficiente desde el router hacia el Broker MQTT remoto. Como Señalan Guerrón et al. (2023), este direccionamiento es lo que garantiza que la información recolectada no se extravíe y pueda llegar con precisión al servidor en la nube para su posterior procesamiento y visualización.

- **Capa de Enlace de Datos**

Esta capa es la responsable de la transferencia confiable de información a través de un enlace físico de transmisión. Su función principal es el Enramado o Framing, el cual consiste en tomar los paquetes IP y dividirlos en unidades de datos denominadas tramas. Según **(Stallings & Stallings, n.d.)**, “la capa de enlace de datos transforma una instalación de transmisión pura, es decir, una capacidad de transmisión de bits en bruto, en un enlace que parece estar libre de errores de transmisión para la capa de red” (p. 68). Dentro del presente proyecto, esta capa asegura que, si un dato de contaminación enviado sufre una interferencia eléctrica en el aire, el error sea detectado y corregido antes de llegar al router local.

Para poder con sus objetivos, esta capa se apoya con tres componentes tecnológicos fundamentales:

Direccionamiento MAC (Media Access Control): a diferencia de la dirección IP, que es algo lógico y puede cambiar, la dirección MAC es un identificador único grabado de fábrica en el hardware. En esta capa, cada trama lleva la dirección MAC del nodo sensor y la del router. Esto garantiza que la información de los sensores SEN0460 pueda ser entregada exactamente al punto de acceso correcto y no se confunda con otros dispositivos de la red doméstica de los vecinos.

Protocolo Wi-Fi (IEEE 802.11): es la tecnología inalámbrica que se utilizará para conectar los nodos sensores con el punto de acceso a internet. El Wi-Fi se encarga de empaquetar los datos IP en tramas de radiofrecuencia. Usa mecanismos como CSMA/CA que es un Acceso Múltiple por Detección de Portadora y Evitación de Colisiones para poder asegurar que, si varios sensores de la zona intentan transmitir al mismo tiempo, no se produzcan “choques” de señales que corrompan los datos.

Protocolo Ethernet (IEEE 802.3): Pese a que la arquitectura da una preferencia al enlace inalámbrico en las unidades periféricas, la norma Ethernet es indispensable dentro de la topología de soporte que realiza la interconexión entre enrutadores y servidores locales. Este estándar permite administrar el flujo de tramas de cableado por trenzado (UTP), proveyendo una robustez y tasa de transferencia superiores durante la conducción de los paquetes hacia el segmento troncal de la red metropolitana.

La interacción de estas tecnologías en la capa de enlace permite que el flujo de datos sea ordenado y eficiente. Al poder detectar errores mediante sumas de verificación, la capa de enlace garantizará que la información que llega a la capa de red sea una réplica exacta de lo que el sensor midió originalmente, manteniendo la integridad científica necesaria para el presente trabajo.

- **Capa Física**

Constituye el nivel más bajo del modelo de referencia y es la base sobre la cual se sustenta toda la comunicación del sistema. Esta capa se encarga de la transmisión de bits puros a través de un canal de comunicación físico, definiendo las especificaciones mecánicas, eléctricas y funcionales de la interfaz necesaria para el transporte de señales. Según Forouzan (2012), “la capa física coordina las funciones necesarias para transmitir un flujo de bits sobre un medio físico” (p.33), lo que abarca desde la definición de los niveles de voltaje y la temporización de los bits hasta las características físicas del medio de transmisión.

Dentro del presente proyecto, esta capa corresponde estrictamente al hardware y a la infraestructura de señales. Es aquí donde los módulos de radio integrados en los nodos sensores transforman la información digital en ondas electromagnéticas para que puedan viajar por el espacio libre, esta capa utiliza los siguientes elementos tecnológicos:

Las señales de radiofrecuencia (RF) se definen como las frecuencias de operación, tanto el espectro de 2.4 GHz en enlaces Wi-Fi como las frecuencias Sub-GHz orientadas a LoRaWAN, los cuales definen el esquema de modulación necesario para la transmisión. El principal beneficio de este enfoque se presenta al suprimir el tendido de cables físicos, lo que logra disminuir de forma notable los costos asociados al despliegue en exteriores. Asimismo, estas ondas presentan una propagación idónea frente a barreras arquitectónicas en el entorno metropolitano, lo que asegura que las tramas logren alcanzar el punto de acceso más próximo. De acuerdo con Forouzan, (2012), modular dichas señales salvaguarda la precisión y consistencia del bit aun ante presencia de interferencias electromagnéticas del medio.

Por su parte, las unidades microcontroladoras logran constituir circuitos integrados que agrupan los bloques de un sistema de cómputo en una sola pastilla de silicio, desempeñándose como el núcleo de computación para cada estación que se encargara de medir los datos. Estas unidades son particularmente interesantes debido a el gasto energético contenido y la flexibilidad que tienen para administrar los canales de entrada y salida de forma simultánea. De este modo, procesan la información de los transductores en lapsos de milisegundos a la par que gestionan la comunicación de red. De igual forma, integran estados de Deep sleep, una capacidad esencial para maximizar la autonomía operativa y prologa la durabilidad del dispositivo en el campo.

De la mano del microcontrolador, están los sensores, en este caso son los de calidad del aire para el presente proyecto, estos son dispositivos transductores que detectan cambios en las propiedades físicas o químicas del entorno ya sea por presencia de gases o partículas y los convierten en una señal eléctrica medible. La ventaja de estos sensores a nivel competitivo es la capacidad que tienen de ofrecer una alta resolución temporal, entregando datos cada pocos segundos. A diferencia de las estaciones gubernamentales masivas, estos sensores permiten una granularidad micro espacial, detectando picos de contaminación específicos en esquinas o calles con alto tráfico. Según Argyropoulos et al. (2024), cuando se calibran correctamente, estos dispositivos ofrecen una precisión suficiente para aplicaciones de alerta temprana en salud pública.

La variable energética se presenta como el elemento transversal de la arquitectura de hardware, haciendo que demande la observancia de normas de alimentación eléctrica estandarizadas, habitualmente con fijación en 3.3V o 5V según los requerimientos operativos de los sensores y la placa ESP32. La regulación estricta y el acondicionamiento del voltaje son indispensables para mantener la estabilidad en la conversión analógica-digital. Operar bajo estos parámetros normalizados previene la sobrecarga en el circuito, mitiga la disipación térmica y evita que existan inconvenientes por sobrecalentamiento, garantizando la precisión de las mediciones y la fidelidad técnica del registro cronológico de contaminantes.

Para poder garantizar que los datos de calidad del aire lleguen desde los sensores hasta el servidor, se ha seleccionado una topología de red inalámbrica basada en Wi-Fi, complementada con el protocolo de transporte MQTT. La elección de MQTT se respalda en que cuenta con características positivas como su ligereza y eficiencia energética, las cuales son características vitales para el sistema IoT donde el ancho de banda puede ser limitado. Según Felici-Castell et al. (2023), el esquema MQTT aventaja significativamente al protocolo HTTP estándar en despliegues sensoriales gracias a su modelo de publicación / suscripción, el cual optimiza el tráfico de red y reduce el gasto de potencia del microprocesador.

1.4. Mecanismos de seguridad y persistencia en Tiempo real

La arquitectura de software de este proyecto no solo tiene el enfoque en capturar los datos, sino en garantizar que la información sea accesible de forma segura y visualizable sin ninguna latencia perceptible para el usuario final. En sistemas de monitoreo crítico, la seguridad asegura que solo personal autorizado modifique los sensores, mientras que la persistencia en tiempo real permite que los ciudadanos vean cambios en la contaminación al instante. Para ello, se emplean dos tecnologías fundamentales: JWT y Websockets.

o JSON Web Token (JWT) y su paradigma de autenticación sin estado

Un JSON Web Token (JWT) es un estándar abierto (RFC 7519) que funciona como un pasaporte digital, compacto y autónomo para poder transmitir información de forma segura. La principal bondad de JWT es que permite la autenticación sin estado o stateless. Dentro de la Web tradicional, el servidor debe recordar quien está conectado, por ende, debe guardar una lista en su memoria o sesiones; con JWT, el servidor no guarda nada, ya que el token que lleva el usuario contiene toda la información necesaria para identificarse.

Como explican Hassan et al. (2024), la estructura de un JSON Web Token (JWT) se compone de 3 bloques codificados en Base64URL:

1. **El Encabezado o header:** Especifica la tipología del objeto criptográfico y el algoritmo de cifrado o firma empleado para su validación junto a HS256.

2. **La carga útil o payload:** Agrupa las declaraciones de identidad (Claims) e información contextual del usuario, tales como el identificador personal y el nivel de privilegios que se asignan, por ejemplo, el perfil de consulta pública para ciudadanos o si cuenta con facultades operativas de calibración para administradores.
3. **La firma o signature:** Constituye el segmento de control de integridad más crítico, obtenido a partir del cómputo criptográfico entre el encabezando, la carga útil y una clave secreta residente exclusivamente en el backend. Este mecanismo permite que se generen adulteraciones en el tránsito, cualquier intento de escalar permisos de forma ilícita invalida la firma generada, lo que prova el rechazo automático de la petición por parte del servidor y garantiza la seguridad perimetral de la plataforma.

- **WebSockets**

Para el monitoreo de contaminantes en puntos críticos, el tiempo de respuesta es fundamental. Tradicionalmente, la web opera bajo el modelo HTTP de petición-respuesta, comparable a una conversación donde el cliente debe preguntar constantemente: “¿tengo datos nuevos?”. Este proceso genera latencia y una sobrecarga en el sistema innecesariamente.

Los WebSockets resuelven este problema al proporcionar un canal de comunicación full-dúplex y persistente, en términos sencillos, el “full-dúplex” es si tomáramos una llamada telefónica donde ambas partes pueden hablar y escuchar al mismo tiempo, a diferencia del HTTP donde funciona como un walkie-talkie donde solamente uno habla a la vez. El proceso inicia con un handshake que eleva la conexión normal a una conexión de alta velocidad.

Según (Guerrón et al., 2023) esta tecnología permite la implementación de la técnica Server-Push. Esto significa que, en cuanto el Broker MQTT recibe una lectura de material particulado del nodo sensor, el servidor “empuja” proactivamente esa información a todos los usuarios conectados. Esta bondad garantiza que el dashboard de calidad del aire se actualice en milisegundos sin que el usuario tenga que refrescar la página, permitiendo observar las curvas de contaminación en vivo, tal como ocurren en el entorno físico.

1.5. Servicios en la Nube (Cloud Computing)

El uso de plataformas en la nube constituye el pilar fundamental para la escalabilidad y disponibilidad del sistema de monitoreo ambiental. Para un entendimiento integral, la computación en la nube se define como la entrega bajo demanda de potencia de cómputo, base de datos, almacenamiento y otras aplicaciones a través de internet. Según **Mell & Grance (2011)** en el estándar del NIST, “la computación en la nube es un modelo para habilitar el acceso red ubicuo, conveniente y bajo demanda a un conjunto compartido de recursos de computación configurables que pueden ser rápidamente aprovisionados” (p.2). Más allá de ser un simple repositorio, la nube actúa como un Middleware o capa de orquestación que permite desacoplar la generación de datos en los nodos sensores de su consumo en la interfaz de usuario. Según Hassan et al. (2024), la integración con plataformas Cloud es mandatorio para sistemas de bajo costo, ya que permite delegar las tareas pesadas de procesamiento, filtrado y análisis avanzado desde el hardware limitado del microcontrolador hacia servidores con alta capacidad de cómputo.

Bajo este contexto, la implementación del sistema se sustenta en los servicios de Amazon Web Services (AWS) debido a su amplia penetración e infraestructura distribuida a nivel global. Esta plataforma ofrece un índice de disponibilidad del 99.9%, asegurando que la telemetría ambiental se encuentre accesible de manera ininterrumpida las 24h del día para la consulta ciudadana y el análisis de los organismos pertinentes. La integración con AWS responde a su catálogo de servicios administrados, los cuales logran simplificar la orquestación técnica y optimizan el esfuerzo de despliegue en la arquitectura del proyecto. De acuerdo con Guerrón et al. (2023), la computación en la nube actúa como el integrador final que permite analizar de forma masiva los datos y asegurar que al API esté alojada de forma protegida. En este proyecto, el servidor en la nube no solo almacena la información, sino que gestiona la seguridad mediante tokens JWT, asegurando que solo el personal autorizado pueda modificar los parámetros de configuración de red.

La infraestructura de AWS proporciona tres ventajas técnicas críticas a identificar dentro del trabajo de titulación:

1. **Elasticidad y escalabilidad:** AWS permite que la red crezca de forma orgánica. Si se deciden instalar más nodos a lo largo del barrio “El Rosario”, la nube puede absorber el incremento en el flujo de datos automáticamente, aumentando los recursos de procesamiento sin necesidad de reconfigurar la arquitectura física.
2. **Disponibilidad Continua:** al alojar el Broker MQTT (Mosquitto) y el backend en instancias de AWS, se garantizará que el sistema sea resiliente ante fallos eléctricos o de red locales, permitiendo el acceso a los reportes históricos en cualquier momento.
3. **Seguridad centralizada:** AWS facilita la implementación de protocolos avanzados para el cifrado de datos en tránsito. De acuerdo con Guerrón et al. (2023), centralizar estos servicios facilita el control de accesos, protegiendo la integridad de la información ambiental frente a posibles manipulaciones.

Finalmente, en este entorno, la nube no solamente almacenará el dato, sino que lo transforma. Mediante el uso de una arquitectura de microservicios, que según (Newman, 2021), es un modelo de desarrollo donde una aplicación se construye como un conjunto de servicios pequeños y autónomos que se ejecutan de forma independiente y se comunican a través de protocolos ligeros; las tramas crudas enviadas por los sensores de Ozono y Material Particulado son gestionadas de forma eficiente. El desacoplamiento funcional asegura que se aíslen los procesos, se previene fallos en cascada y mantiene la continuidad operativa de la plataforma ante eventuales errores en módulos individuales. Mediante este esquema distribuido, los paquetes de telemetría se rigen a un estándar, están depurados e indexados con marcas temporales exactas previas al almacenamiento definitivo. Dicho tratamiento en la nube transforma las variaciones de potencial eléctrico de los transductores en variables ambientales procesadas, coherentes y de alto valor preventivo para la vigilancia de la salud comunitaria.

a. Sistemas de gestión de Bases de Datos

En una red de vigilancia ambiental, el repositorio de datos actúa como el núcleo computacional y analítico para generar diagnósticos. Con el fin de soportar el despliegue de el nodo, se adoptó un esquema de persistencia híbrida que articula motores de base de datos diferenciados en base al tipo de carga necesaria. Con este diseño se administra de forma paralela los metadatos de configuración de la red física y el ingreso intensivo de la red física y el ingreso intensivo de lecturas cronológicas, preservando la estabilidad del servidor, los tiempos de respuesta y la consistencia de la información.

o Bases de Datos de series temporales

Dado que el núcleo de la presente investigación es el comportamiento de los contaminantes a lo largo del tiempo, es indispensable el considerar a InfluxDB, es una base de datos de series temporales que se encuentra optimizada para manejar datos cuya clave principal sea una marca temporal de tiempo. Mientras que una base de datos tradicional se saturaría al calcular promedios de millones de registros, InfluxDB utiliza índices temporales específicos que permiten la generación de reportes de variación horaria o diaria en milisegundos.

De acuerdo con Guerrón et al. (2023) y Argyropoulos et al. (2024), esta tecnología es la preferida en proyectos de Smart Cities debido a tres bondades fundamentales: la compresión masiva de datos para optimizar costos en AWS, las políticas de retención que eliminan o resumen datos crudos automáticamente tras un periodo definido, y sus funciones analíticas nativas que facilitan la creación de curvas de contaminación. InfluxDB garantiza que el historial ambiental pueda ser analizado durante meses o años sin que se degrade el rendimiento del sistema, proporcionando una base científica sólida para la vigilancia de la salud pública.

b. Lenguajes de programación y ecosistema de desarrollo

La implementación de una red de monitoreo IoT requiere que tenga una arquitectura de software multicapa y políglota. Para garantizar que el desarrollo sea eficiente y escalable, se hace un uso diverso de Frameworks, definidos como estructuras de software reutilizables que proporcionan una funcionalidad genérica y un conjunto de herramientas estandarizadas para facilitar el desarrollo de aplicaciones complejas. Según Sommerville (2011), un framework actúa como un “esqueleto” que permite a los desarrolladores enfocarse en la lógica específica del problema, en el caso presente sería el monitoreo ambiental sin tener que programar desde cero las funciones básicas de red o interfaz.

El sistema se estructura bajo un paradigma de Microservicios de igual forma, el cual se define como un enfoque arquitectónico donde una aplicación se descompone en un conjunto de servicios pequeños, independientes y débilmente acoplados. Según Lewis (2014), en una arquitectura de microservicios, cada componente se ejecuta de forma autónoma y se comunica mediante mecanismos ligeros como protocolos HTTP o MQTT.

La arquitectura proporciona tres bondades críticas para el proyecto, permite que el servicio que recibe los datos de los sensores sea independiente del servicio que gestiona los usuarios o el dashboard web, permite tener escalabilidad independiente lo cual ayuda si el tráfico de sensores aumenta, se puede escalar solo el módulo de ingesta de datos en AWS sin afectar el resto del sistema y hace que sea resiliente, si un microservicio falla, por ejemplo el generar reportes en PDF, los demás servicios como la captura de datos en tiempo real siguen funcionando con normalidad.

- **C++ y el entorno de desarrollo embebido**

Para la programación del nodo sensor o capa de percepción física, se utiliza C++ dentro del ecosistema de Arduino y el framework ESP-IDF. C++ es un lenguaje complicado de propósito general que ofrece un control directo sobre los recursos de hardware y la gestión de memoria. Se define como el estándar para sistemas embebidos debido a la alta eficiencia que tiene y la velocidad de ejecución. Según Felici-Castell et al. (2023), el uso de lenguajes de bajo nivel permite implementar ciclos de trabajo optimizados para la gestión energética, facilitando que el microcontrolador entre en estados de bajo consumo entre mediciones.

En el contexto del presente proyecto, C++ actúa como el motor central del microcontrolador. Es el encargado de gestionar la lectura digital de los sensores de gases y material particulado a través de protocolos de comunicación serial (UART/I2C), procesando las tramas de bytes para transformarlas en unidades de medición exactas, tales como partes por millón (ppm) O microgramos por metro cúbico ($\mu g / m^3$). Finalmente coordina la serialización de esta información y el envío concurrente de paquetes de datos hacia la nube, utilizando la librería PubSubClient para la transmisión bajo el protocolo ligero MQTT.

Como forma de sustentación adicional del uso de este lenguaje en la arquitectura física del sistema, la tabla 10 detalla las bondades, beneficios y virtudes técnicas que C++ aporta al desarrollo del nodo sensor.

Tabla 10

Bondades y virtudes técnicas de C++ en la capa de percepción IoT

Atributo técnico	Descripción Arquitectónica	Beneficio directo en el proyecto
<i>Ejecución determinista y baja latencia</i>	Es la capacidad de ejecutar rutina de código con tiempos de respuesta predecibles y exactos, sin las pausas introducidas por lenguajes interpretados.	Garantiza la lectura precisa de los pulsos láser del sensor de material particulado en tiempo real, evitando la pérdida de datos clínicos por retrasos del procesador.
<i>Gestión explícita de memoria</i>	Permite el control manual sobre la asignación y liberación de memoria sin depender de procesos automáticos.	Previene desbordamientos de memoria al serializar objetos JSON y gestionar la conexión de la red en un microcontrolador con RAM limitada.
<i>Eficiencia y perfilamiento energético</i>	Otorga acceso profundo a los registros físicos del procesador para la manipulación de estados de energía (Felici-Castell et al., 2023).	Facilita la implementación de modos Deep Sleep, reduciendo drásticamente El consumo eléctrico de la estación sensor entre envíos de telemetría.
<i>Interoperabilidad Nativa del hardware</i>	Soporte estandarizado e integración directa con protocolos de comunicación electrónica (I2C, SPI, UART).	Permite la extracción limpia y directa de datos desde los sensores sin necesidad de capas de traducción intermedias.

Nota. Elaboración propia

o **Python**

En la capa de servidor y middleware, el cual es el núcleo lógico de la arquitectura, el lenguaje predominante es Python. Se define como un lenguaje interpretado, multiparadigma y de alto nivel, el cual se seleccionó por la capacidad robusta que tiene para orquestar protocolos de red y el procesamiento matemático de datos. En el entorno de soluciones IoT, Python se desempeña como el encargado de articular entre los canales de mensajería liviana como el bróker MQTT y el modelo de persistencia dual del sistema. En este sentido Hassan et al., (2024), señalan que este lenguaje agiliza la vinculación directa de redes sensoriales distribuidas con avanzadas en la nube, optimizando la ejecución de rutinas para la verificación, filtrado y depuración de la telemetría ambiental.

El adoptar a Python dentro del proyecto en la capa de procesamiento del servidor metodológicamente tiene su sustento debido al ser compatible directamente con el marco FastAPI. La ingesta continua de mediciones ambientales constituye una tarea con limitaciones predominantes de entrada y salida de red. A diferencia de los modelos web tradicionales basados en bloqueos por hilo de ejecución ante cada solicitud, el esquema asíncrono implementado en FastAPI mediante bucles de eventos permite capturar simultáneamente las concentraciones de partículas suspendidas. Contrastarlas contra los límites sanitarios y registrarlas en los repositorios de datos sin generar latencias ni degradar la disponibilidad del servicio.

Como sustento de este lenguaje a utilizar en el presente proyecto la tabla 11 desglosa las bondades y beneficios técnicos que Python y su ecosistema aportan al sistema central.

Tabla 11

Bondades y virtudes técnicas de Python en la capa de orquestación y middleware

Atributo técnico	Descripción Arquitectónica	Beneficio directo en el proyecto
Concurrencia asíncrona ASGI	Implementación de un bucle de eventos no bloqueante	Permite procesar peticiones simultáneas de múltiples nodos sensores y clientes web sin sufrir cuellos de botella en la red.

	mediante el marco de trabajo FastAPI	
<i>Validación estricta de esquemas</i>	Capacidad nativa a través de librerías como Pydantic para forzar y validar tipos de datos antes de su procesamiento.	Garantiza la integridad del historial; si un sensor envía un dato corrupto o nulo el sistema lo rechaza antes de contaminar la base de datos.
<i>Orquestación multiprotocolo</i>	Ecosistema maduro que unifica protocolos dispares bajo la misma lógica de programación.	Permite al servidor suscribirse ininterrumpidamente a los tópicos de telemetría paho-mqtt y, de forma simultánea despachar alertas en tiempo real en el panel ciudadano.
<i>Madurez científica y analítica</i>	Disponibilidad del ecosistema de librerías más avanzado para el procesamiento de series temporales y ciencias de datos	Facilita el cálculo inmediato de medias móviles, agrupaciones temporales como ventanas de 5 minutos.

Nota. Elaboración propia

- **JavaScript**

En la capa de presentación ciudadana y visualización epidemiológica, el desarrollo del cliente web se fundamenta usando JavaScript, implementando a través de la biblioteca declarativa React.js. JavaScript es un lenguaje interpretado, asíncrono y orientado a eventos, que constituye el estándar absoluto para la creación de Aplicaciones de Página única. De acuerdo con los postulados de Guerrón et al., (2023), la visualización en tiempo real representa el componente crítico para la democratización de la información, traduciendo telemetría técnica cruda en indicadores epidemiológicos y ambientales comprensibles para la población.

La adopción de React.js en el marco del presente proyecto de titulación resuelve una limitante arquitectónica fundamental de la web tradicional: el alto costo computacional de la manipulación imperativa del Document Object Model (DOM). Al implementar un algoritmo de reconciliación basado en un DOM virtual, el sistema procesa de forma eficiente los flujos de datos continuos de calidad del aire. Cuando ingresa una nueva lectura telemétrica, el algoritmo calcula matemáticamente la diferencia de estados en la memoria y redibuja de manera exclusiva los componentes que registran cambios ya sean los gráficos vectoriales o los colores de riesgo a exposición. Dicho mecanismo hace que la actualización de la interfaz sea dinámica e inmediata al suprimir la consulta periódica a través de HTTP, lo que aminora la carga computacional para el navegador del usuario y optimiza el uso del ancho de banda de red. De igual forma, JavaScript refuerza la seguridad en el extremo del cliente al contar con esquemas de autenticación sin estado fundamentados en la validación de JSON Web Tokens, garantizando un acceso restringido y confiable a las series históricas de telemetría ambiental.

Como fundamento a la elección de este ecosistema de desarrollo del panel médico, la tabla 12 detalla las bondades arquitectónicas que JavaScript y React.js aportan al proyecto.

Tabla 12

Bondades y virtudes técnicas de JavaScript / React.js en la capa de presentación

Atributo técnico	Descripción Arquitectónica	Beneficio directo en el proyecto
<i>Reconciliación mediante DOM virtual</i>	Mantiene una representación ligera de la interfaz en memoria para calcular cambios de estado antes de alterar el DOM nativo.	Permite actualizar gráficos de comportamiento respiratorio sin causar parpadeos visuales ni fugas de memoria en el dispositivo del usuario.
<i>Reactividad y estado global</i>	Gestión centralizada de datos (State Management) donde las modificaciones lógicas propagan actualizaciones automáticas a la vista.	Asegura que si un contaminante, por ejemplo, dióxido de carbono, supera el límite de la OMS, el panel, gráficos y el área de exposición cambien simultáneamente al estado de alerta clínica.
<i>Asincronía orientada a eventos</i>	Capacidad no bloqueante para mantener conexiones bidireccionales persistentes (WebSockets) y realizar llamadas asíncronas a la API (Fetch).	Recibe la telemetría del Broker MQTT en tiempo real optimizando el ancho de banda, al eliminar la necesidad de realizar peticiones HTTP (encuestas o polling) redundantes.
<i>Interoperabilidad nativa JSON</i>	Análisis, estructuración y manipulación directa del formato estándar de intercambio de datos (JavaScript Object Notation).	Procesa con máxima eficiencia las matrices de datos médicos exportadas por FastAPI, permitiendo filtrar y descargar historiales epidemiológicos en milisegundos.

Nota. Elaboración propia

c. Metodologías de Gestión y Desarrollo de proyectos tecnológicos

La complejidad que abarca un sistema de monitoreo ambiental desde la electrónica analógica de los sensores a ubicar, hasta la visualización en la nube, exige un marco de gestión que reduzca el riesgo de fallos técnicos. Las metodologías ágiles han emergido como la solución preferida en la ingeniería moderna, desplazando a los modelos tradicionales en cascada debido a su capacidad de respuesta ante imprevistos.

o Kanban y el control del flujo visual

La metodología Kanban, originada en el sistema de producción de Toyota, se basa en la entrega Justo a Tiempo o Just In Time. Su núcleo es el tablero Kanban, una herramienta de gestión visual que permita al investigador observar el estado de cada tarea de un vistazo.

Según Anderson (2010), Kanban se rige por la limitación del trabajo en progreso (WIP), lo que evita que exista una saturación por parte del desarrollador. En el presente proyecto, esto se traduce en no iniciar la programación del servidor en Python hasta que la lectura del sensor C++ sea estable y precisa. La presente metodología es útil sobre todo en la fase de despliegue en el barrio "El Rosario". Permite gestionar tareas lógicas como la compra de componentes, el ensamble de circuitos, las pruebas de estanqueidad de las carcasas y la instalación en campo, asegurando que cada etapa se complete con calidad antes de pasar a la siguiente.

Figura 12

Modelo Kanban



Nota. Adaptado de GanttPro y su explicación en cómo mejorar la gestión de trabajo con Kanban, 2026. (<https://blog.ganttpro.com/es/metodo-kanban-para-mejorar-el-flujo-de-trabajo-1/>). CC BY 2.0.

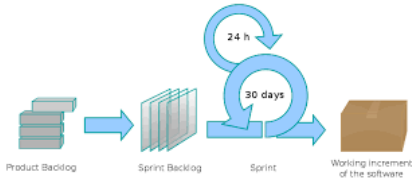
- Scrum: El Marco de trabajo de Alta Adaptabilidad

Scrum no es solamente una metodología, sino un framework que fomenta la mejora continua. Se basa en el empirismo, donde el conocimiento proviene de la experiencia y la toma de decisiones se fundamenta en lo observado. De acuerdo con Schwaber et al. (2020), Scrum se sostiene sobre lo transparente que es, sus avances son visibles, la evaluación frecuente del proyecto y la adaptación en caso de desviación dentro del proyecto. Este proceso se organiza en torno al Product Backlog, para este presente trabajo de titulación se encuentran incluidos el calibrar el sensor MQ-09 hasta el diseño de la interfaz dentro de React.js. estos requisitos son divididos en Sprints, lo que permite que el sistema de monitoreo crezca de forma modular. Al final de cada ciclo, se obtiene un incremento funcional, que reduce drásticamente la probabilidad de errores catastróficos al final del proyecto.

Dentro del desarrollo de sistemas IoT se presenta un desafío único: la convergencia del mundo físico y el digital. Mientras que el software es flexible, el hardware es rígido y costoso de modificar. Estudios recientes como el de Guerrón et al. (2023), sugieren que el uso de Scrum dentro de redes de sensores urbanos permite una validación temprana de la conectividad. En el contexto del barrio “El Rosario” significa que se puede probar si la señal Wi-Fi es suficiente para transmitir datos mediante MQTT en las primeras semanas, permitiendo ajustar la antena o la ubicación del nodo antes de que el resto del sistema esté construido.

Figura 13

Metodología Scrum



Nota. Adaptado de JavierGarzas y su explicación en la evolución del diagrama Scrum, 2012.

(<https://javiergarzas.com/2012/08/diagrama-scrum.html>). CC BY 2.0.

Para el presente trabajo de integración curricular, se determina que Scrum es la metodología más robusta por las siguientes consideraciones, al realizar Sprints permite enfocarse en la precisión de los distintos temas a diseñar, por ejemplo, el sensor MQ-09 y la recolección de datos del material particulado, asegurándonos que los datos que se recolectan sean científicamente válidos antes de proceder a la analítica de datos.

Por otro lado, la modularidad Full-Stack facilita la construcción del proyecto por capas, facilita que el sistema tenga un backend funcional en fastAPI recibiendo datos simulados mientras el hardware aún está en el proceso de ensamblaje final. Dentro del tema de la base de datos, Scrum permite realizar pruebas de estrés en la base de datos de forma temprana, asegurando que el sistema soporte de InfluxDB soporte el almacenamiento de datos históricos por periodos prolongados sin pérdida de información. Por último, a través de revisiones al final de cada Sprint, se pueden ajustar las visualizaciones en JavaScript para poder asegurar que los ciudadanos puedan visualizar e interpretar fácilmente los índices de calidad del aire.

Capítulo tres

Análisis

El presente capítulo detallará el proceso de ingeniería aplicado para la construcción del sistema de monitoreo ambiental, la fase de este análisis constituye parte del proceso fundamental para poder traducir las necesidades de monitoreo ambiental del barrio “El Rosario” en especificaciones técnicas operativas y funcionales. Bajo el marco de trabajo de SCRUM, este proceso inicia con la identificación de los requerimientos críticos que permitan una captura precisa de contaminantes para el presente proyecto asegurando que la arquitectura de microservicios sea capaz de gestionar el flujo de datos desde los nodos sensores hasta la infraestructura en la nube de AWS.

El análisis no solo contempla la viabilidad técnica de la transmisión mediante el protocolo MQTT, sino que prioriza la experiencia del usuario final, garantizando que la información procesada y almacenada de forma híbrida en MongoDB e InfluxDB sea presentada de manera íntegra, segura y oportuna. Este diagnóstico inicial permite definir el Product Backlog, asegurando que cada funcionalidad desarrollada en los Sprints posteriores responda directamente a los objetivos de salud pública y transparencia informativa planteados en el proyecto.

Comentado [3]: Revisar faltas en todo el documento

3.1. Análisis de requisitos

El presente análisis define el “Qué” del sistema. Se identifican las necesidades de los usuarios finales y las restricciones técnicas impuestas por el entorno urbano y la infraestructura de red. Los requisitos funcionales describen las acciones específicas que el sistema debe realizar.

Tabla 10

Requisitos Funcionales (RF)

ID	Requisito Funcional	Descripción
RF-01	Adquisición de datos	El sistema debe leer niveles de material particulado y gases de forma continua.

<i>RF-02</i>	Transmisión inalámbrica	El nodo sensor debe enviar los datos al bróker MQTT mediante la red Wi-Fi local.
<i>RF-03</i>	Persistencia híbrida	El backend debe almacenar metadatos en MongoDB y series temporales en InfluxDB.
<i>RF-04</i>	Visualización en tiempo real	El Dashboard web debe mostrar gráficas dinámicas que se actualicen vía WebSockets.
<i>RF-05</i>	Gestión de alertas	El sistema debe notificar visualmente cuando los niveles de contaminación superen los límites de la OMS.
<i>RF-06</i>	Autenticación de acceso	El acceso a la configuración del sistema debe estar protegido mediante tokens JWT.

Nota. Elaboración propia basado en los requerimientos funcionales del TIC

Los requisitos no funcionales representan las cualidades de calidad, rendimiento y seguridad que el sistema debe poseer. A continuación, en la tabla 11 se encuentran los requisitos no funcionales a considerar para el TIC:

Tabla 11

Requisitos No Funcionales (RF)

ID	Requisito No Funcional	Descripción
<i>RNF-01</i>	Disponibilidad	El sistema alojado en AWS debe garantizar una disponibilidad del 99.9%, permitiendo acceso 24/7.
<i>RNF-02</i>	Escalabilidad	La arquitectura debe permitir la adición de nuevos nodos sensores sin degradar el rendimiento.
<i>RNF-03</i>	Seguridad	Toda comunicación entre el cliente y el servidor debe estar cifrada mediante SSL/TLS
<i>RNF-04</i>	Latencia	El tiempo transcurrido desde la captura del sensor hasta la visualización en la web debe ser inferior a 1 segundo.
<i>RNF-05</i>	Bajo consumo	El firmware en C++ debe optimizar el uso de energía del microcontrolador para prolongar la vida útil del hardware.

Nota. Elaboración propia basado en los requerimientos no funcionales del TIC

3.1.2. Clasificación y Roles de los actores del sistema

A continuación, se presenta la matriz de actores identificados para el sistema de monitoreo ambiental del barrio “El Rosario”. Esta clasificación es la base para la configuración de la seguridad mediante JWT y la definición de las interfaces en React.js.

Tabla 12

Actores del sistema

ID	Descripción del Rol	Nivel de Acceso	Interés Principal
AC-01	Ciudadano / vecino: Habitante del sector que consume la información ambiental.	Público de solo lectura.	Salud personal y niveles de contaminación en tiempo real.
AC-02	Administrador: responsable técnico de la infraestructura informática.	Total, de lectura y escritura.	Estabilidad del sistema, gestión de la nube AWS y seguridad.

Nota. Elaboración propia basado en los actores identificados

3.1.3. Historias de Usuario

Las historias de usuario representan unidades de trabajo que describen una funcionalidad desde la perspectiva del actor final, bajo el marco de trabajo de SCRUM, estas historias nos permiten definir los criterios de aceptación técnicos para cada incremento del sistema. A continuación, en la tabla se detallan las historias que se consideraron prioridad para el sistema de monitoreo ambiental:

Tabla 12

Actores del sistema

ID	Actor	Historia de Usuario	Criterios de Aceptación
HU-01	Administrador	Como responsable técnico, quiero autenticarme mediante un sistema seguro para gestionar los nodos sensores sin riesgo de accesos no autorizados.	1. El inicio de sesión debe generar un token JWT. 1. El token debe expirar tras un periodo de inactividad para garantizar seguridad.
HU-02	Administrador	Como técnico, quiero monitorear el estado de conexión de los	1. Se debe mostrar un indicador de “En línea /

Comentado [4]: La redacción del documento, siempre en tercera persona.

		dispositivos para identificar fallos en la red Wi-Fi de forma remota.	Fuera de Línea" basado en la telemetría de MongoDB. 2. El sistema debe registrar el último mensaje recibido del nodo.
HU-03	Administrador	Como responsable del sistema, quiero dar de alta o baja nuevos nodos sensores en el mapa del barrio de forma dinámica.	1. El sistema debe permitir registrar el ID del microcontrolador en MongoDB. 2. El nuevo nodo debe aparecer automáticamente en la interfaz web.
HU-04	Ciudadano	Como habitante del barrio, quiero visualizar los niveles de Dióxido de Carbono en tiempo real para conocer la calidad del aire antes de salir de casa.	1. La interfaz debe actualizarse mediante WebSockets de forma Automática. 2. Se debe mostrar una escala de colores según la normativa OMS.
HU-05	Ciudadano	Como usuario, quiero acceder a gráficas de la última semana para identificar las horas de mayor contaminación en la Avenida del Maestro.	1. El sistema debe consultar los datos en InfluxDB en menos de dos segundos. 2. Las gráficas deben ser interactivas y claras mostrando horarios de forma clara.
HU-06	Ciudadano	Como usuario, quiero recibir una notificación visual inmediata cuando la calidad del aire sea "Dañina" Según la OMS.	1. El dashboard debe cambiar de color y mostrar un mensaje de advertencia. 2. La alerta debe activarse sin que el usuario recargue la página.

Nota. Elaboración propia basado en las historias de usuario

3.1.4. Casos de Uso del Sistema

Los diagramas de casos de uso (UML) permiten modelar la funcionalidad del sistema centrándose en los servicios que este ofrece a sus diferentes actores. Dentro de la presente sección se describen los comportamientos del sistema de monitoreo para el barrio “El Rosario”, vinculando los requerimientos capturados en el análisis con la arquitectura técnica propuesta.

Dentro de la tabla 13 se consolidan todas las funcionalidades identificadas, categorizadas por el actor principal y el objetivo del negocio. Esta tabla nos permite observar la trazabilidad entre la necesidad del usuario y la respuesta técnica del sistema.

Tabla 13

Resumen general de Casos de Uso

ID	Módulo	Caso de Uso	Actor Principal	Funcionalidad
CU-01	Monitoreo	Visualizar monitoreo en Vivo.	Ciudadano	Observar niveles de CO_2 y O_3 .
CU-02	Monitoreo	Consultar Históricos	Ciudadano	Generar gráficas de tendencias temporales con consulta a través de InfluxDB
CU-03	Monitoreo	Interactuar con mapa de nodos	Ciudadano	Localización geográfica de los sensores dentro del área de recolección.
CU-04	Seguridad	Gestionar Sesión (Login).	Administrador	Validar identidad y obtener privilegios mediante tokens JWT.
CU-05	Seguridad	Recuperar contraseña.	Administrador	Flujo de seguridad para el restablecimiento de credenciales de acceso.

CU-06	Gestión	Registrar nuevo nodo sensor.	Administrador	Dar de alta nuevos microcontroladores insertando metadatos en MongoDB.
CU-07	Gestión	Configurar umbrales de alerta.	Administrador	Ajustar los rangos de toxicidad visual de acuerdo con normativa OMS.
CU-08	Mantenimiento	Calibrar sensor remotamente.	Administrador	Ajustar factores de escala en el hardware mediante mensajes MQTT.
CU-09	Mantenimiento	Monitorear salud del nodo.	Administrador	Revisar estado de conexión, batería y señal Wi-Fi en MongoDB.
CU-10	Mantenimiento	Reiniciar nodo remotamente.	Administrador	Envío de comando de reinicio al microcontrolador vía MQTT.
CU-11	Reportes	Auditar Logs de Actividad	Autoridad Ambiental	Revisar el Historial de cambios y accesos realizados en la plataforma.

Nota. Elaboración propia basado en los casos de uso

3.1.4.1. Casos de uso Específicos

A. CU-01:

Tabla 14

Caso de Uso CU-01

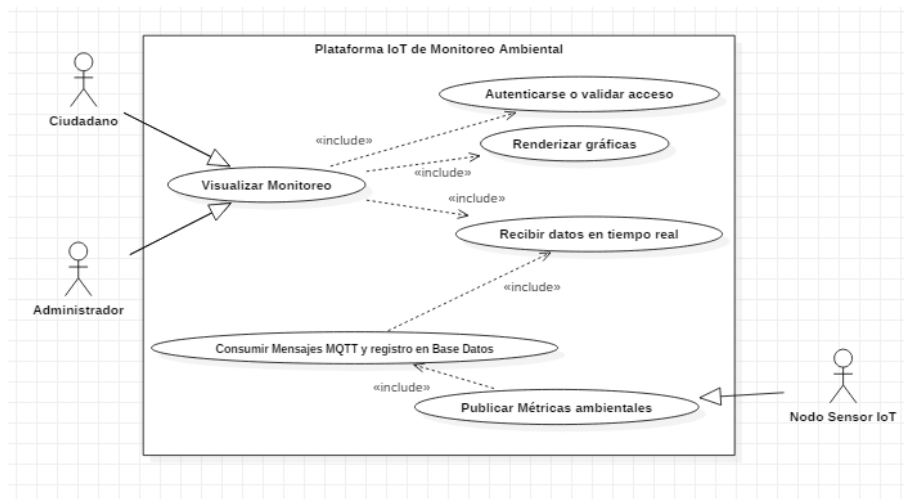
Detalle	
<i>Identificador</i>	CU-01
<i>Nombre</i>	Visualizar Monitoreo
<i>Descripción</i>	El sistema permite al ciudadano observar de forma reactiva los niveles de contaminación capturados a través de una conexión de datos persistentes.

<i>Precondición</i>	El nodo sensor está transmitiendo datos y el servicio de WebSockets en el servidor de AWS se encuentra activo.
<i>Postcondición</i>	El usuario visualiza cambios en las gráficas en milisegundos sin necesidad de interactuar con el navegador.
<i>Actores</i>	Ciudadano, Administrador, Nodo sensor
<i>Flujo de interacción</i>	• El ciudadano accede a la plataforma web cargada mediante el framework React.js. • El frontend solicita al backend (FastAPI) la apertura de un túnel de comunicación bidireccional mediante WebSockets. • El nodo sensor físico publica una trama JSON con las métricas ambientales hacia el Broker MQTT de AWS. • El backend consume el mensaje del Broker, valida su origen y lo registra de forma paralela en la base de datos. • El servidor empuja la información procesada a través del WebSocket activo hacia el cliente. • El frontend recibe el paquete de datos y actualiza instantáneamente los componentes visuales del dashboard.
<i>Comentario</i>	El flujo permite garantizar que exista una actualización en tiempo real, minimizando el consumo de recursos de red en el dispositivo del usuario.

Nota. Elaboración propia basado en el análisis del caso

Figura 14

Caso de Uso graficado CU-01



Nota. Adaptado de la tabla 14 y realizado con StarUML .CC BY 2.0.

B. CU-02:

Tabla 15

Caso de Uso CU-02

	Detalle
Identificador	CU-02
Nombre	Consultar históricos
Descripción	El sistema permite al usuario consultar y analizar el comportamiento pasado de las métricas ambientales mediante el uso de filtros temporales específicos.
Precondición	La base de datos de series temporales debe contar con registros previos almacenados para el periodo y los parámetros solicitados.
Postcondición	El usuario visualiza gráficas de tendencias históricas detalladas en la interfaz web para su análisis.
Actores	Ciudadano
Flujo de interacción	- El usuario selecciona el periodo de interés ya sea día, mes o semana y los parámetros específicos en el panel de filtros del dashboard. - El frontend envía una solicitud de consulta al backend con los parámetros temporales definidos por el usuario.

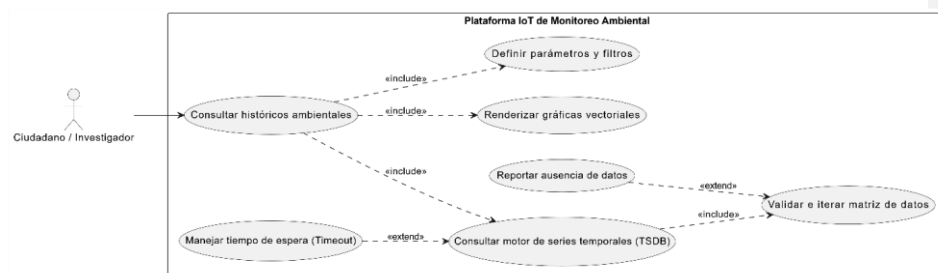
- El backend procesa la petición y realiza una búsqueda optimizada en el motor de base de datos externo de series temporales.
- La base de datos recupera los registros cronológicos correspondientes y los devuelve al servidor backend de forma estructurada.
- El backend valida la integridad de la información recibida y envía el paquete de datos procesados hacia el navegador del cliente.
- El frontend recibe la respuesta y renderiza las gráficas de tendencias históricas de forma interactiva en la pantalla del usuario.

Comentario El sistema debe estar capacitado para manejar volúmenes considerables de datos históricos sin comprometer la velocidad de respuesta de la interfaz.

Nota. Elaboración propia basado en el comportamiento a esperar de la base de datos dentro del caso

Figura 15

Caso de Uso graficado CU-02



Nota. Adaptado de la tabla 15 y hecho con StarUML. CC BY 2.0.

C. CU-03:

Tabla 16

Caso de Uso CU-03

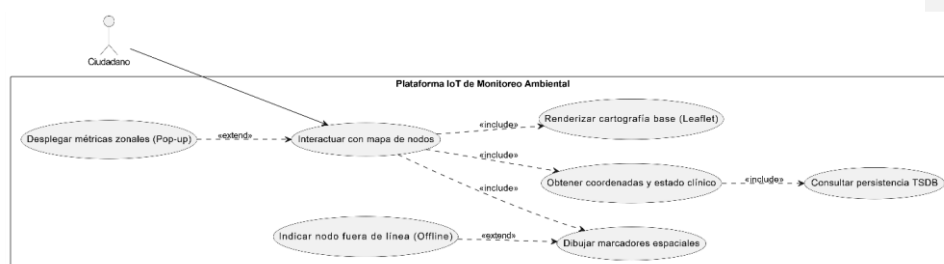
	Detalle
Identificador	CU-03
Nombre	Interactuar con mapa de nodos
Descripción	Permite al ciudadano visualizar la ubicación geográfica exacta de los puntos de monitoreo en el barrio “El Rosario” para identificar los focos de contaminación en su entorno cercano.
Precondición	Los metadatos de ubicación latitud y longitud del dispositivo debe estar correctamente registrado en la base de datos.
Postcondición	El usuario identifica geográficamente los sensores y accede a un resumen rápido de datos por zona.

Actores	Ciudadano
Flujo de interacción	- El ciudadano selecciona la sección de “Mapa de Red” dentro de la interfaz web desarrollada en React.js. - El frontend realiza una petición al backend para obtener la lista de coordenadas y el estado de salud de todos los nodos desplegados. - El backend consulta en la base de datos externa InfluxDB los registros de ubicación y los identificadores únicos de cada microcontrolador. - El sistema de base de datos devuelve la información geográfica y el servidor la envía a la cliente debidamente estructurada. - El frontend utiliza la librería Leaflet para renderizar los marcadores interactivos sobre la capa de mapas. - El usuario interactúa con un marcador específico para desplegar una ventana emergente que muestra el nombre del nodo y el último índice de calidad del aire registrado.
Comentario	El mapa utilizará una codificación de colores en los marcadores para indicar si un nodo está activo o fuera de línea.

Nota. Elaboración propia basado en la posible interacción de los nodos.

Figura 16

Caso de Uso graficado CU-03.



Nota. Adaptado de la tabla 16 y hecho con StarUML. CC BY 2.0.

D. CU-04:

Tabla 17

Caso de Uso CU-04

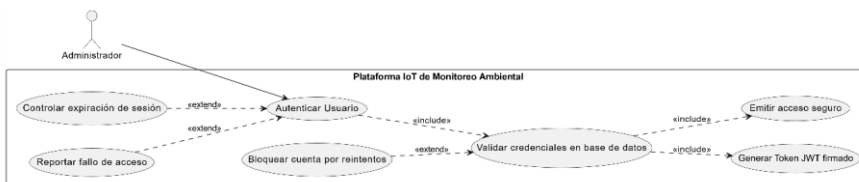
	Detalle
Identificador	CU-04
Nombre	Gestionar Sesión

<i>Descripción</i>	Permite la validación de identidad de los usuarios administrativos para habilitar el acceso a las funciones de gestión y mantenimiento de la red de sensores.
<i>Precondición</i>	Las credenciales del usuario deben estar registradas previamente en el motor de base de datos NoSQL del sistema.
<i>Postcondición</i>	El sistema otorga un token de seguridad JWT que permite el acceso a funcionalidades y rutas protegidas.
<i>Actores</i>	Administrador
<i>Flujo de interacción</i>	- El usuario ingresa sus credenciales de acceso como correo y contraseña en el formulario de inicio de sesión de la plataforma. - El frontend captura la información y la envía de forma segura mediante una petición cifrada hacia el servidor backend. - El backend consulta en la base de datos externa InfluxDB para poder verificar la existencia del usuario y la validez de su contraseña. - Tras validar exitosamente lo anterior, el servidor genera un token JWT firmado digitalmente que contiene el rol y el tiempo de expiración. - El sistema envía el token de seguridad de vuelta al navegador del usuario para su almacenamiento en estado local. - El frontend reconoce el token valido y habilita automáticamente las herramientas administrativas correspondientes al perfil del usuario.
<i>Comentario</i>	El sistema denegará el acceso y notificará al usuario en caso de que las credenciales no coincidan con los registros en la base de datos.

Nota. Elaboración propia basado en la administración del login

Figura 17

Caso de Uso graficado CU-04



Nota. Adaptado de la tabla 17 y hecho con StarUML. CC BY 2.0.

E . CU-05:

Tabla 18

Caso de Uso CU-05

Detalle	
<i>Identificador</i>	CU-05

Figura 18

```

    usecaseDiagram
        actor Admin as Administrador
        actor Email as Servicio de Correo Externo

        usecase U1 as Notificar cuenta inexistente
        usecase U2 as Verificar identidad de cuenta
        usecase U3 as Generar Token de seguridad temporal
        usecase U4 as Recuperar Contraseña
        usecase U5 as Cifrar nueva credencial (Hashing)
        usecase U6 as Actualizar persistencia interna
        usecase U7 as Manejar Token expirado (15 min)

        Admin --> U4
        U7 -.->|«extend»| U4
        U1 -.->|«extend»| U2
        U2 -.->|«include»| U3
        U4 -.->|«include»| U5
        U5 -.->|«include»| U6
        U4 -.->|«invocation»| Email
    
```

Nota. Adaptado de la tabla 18 y hecho con StarUML. CC BY 2.0.

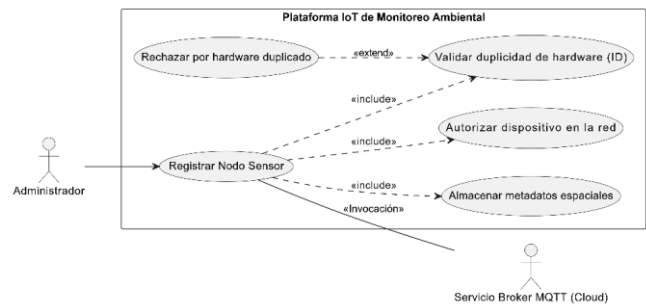
F. CU-06:**Tabla 19***Caso de Uso CU-06*

Detalle	
<i>Identificador</i>	CU-06
<i>Nombre</i>	Registrar Nodo Sensor
<i>Descripción</i>	Permite al administrador dar de alta un nuevo dispositivo físico en la plataforma, vinculando su identidad técnica con su ubicación geográfica en el barrio.
<i>Precondición</i>	El administrador debe contar con una sesión activa (JWT) y poseer el identificador único ID o dirección MAC del microcontrolador.
<i>Postcondición</i>	El nuevo nodo queda habilitado en la base de datos para comenzar a recibir y procesar sus tramas de datos ambientales.
<i>Actores</i>	Administrador, AWS Cloud.
<i>Flujo de interacción</i>	• El administrador ingresa al módulo de gestión de dispositivos y completa el formulario con el ID del sensor, descripción y coordenadas de instalación. • El frontend en React.js valida el formato de los datos y envía una petición de registro hacia el backend del sistema. • El backend (FastAPI) verifica que el identificador del hardware no exista previamente para evitar duplicidad en la red. • El sistema interactúa con la base de datos externa InfluxDB para insertar el nuevo documento con los metadatos y el estado inicial del dispositivo. • El backend comunica al servicio de mensajería en la nube la autorización del nuevo ID para permitir futuras publicaciones vía protocolo MQTT. • El sistema confirma la creación exitosa en la interfaz y actualiza automáticamente el listado global de nodos activos en el dashboard.
<i>Comentario</i>	Una vez registrado, el nodo pasará al estado "pendiente" hasta que el hardware realice su primera transmisión hacia la nube de forma exitosa.

Nota. Elaboración propia basado en el flujo para añadir un nuevo nodo sensor

Figura 19

Caso de Uso graficado CU-06



Nota. Adaptado de la tabla 19 y diseñado con StarUML. CC BY 2.0.

G. CU-07:

Tabla 20

Caso de Uso CU-07

Detalle	
Identificador	CU-07
Nombre	Configurar Umbrales de alerta
Descripción	Permite definir y actualizar los rangos de concentración de contaminantes ($PM_{2.5}$ y O_3) que disparan cambios visuales y alertas en la plataforma, basándose en normativas de salud.
Precondición	El administrador debe contar con una sesión activa y privilegios de escritura en la base de datos.
Postcondición	Los nuevos parámetros se aplican globalmente a la lógica de visualización y al motor de notificaciones del sistema.
Actores	Administrador
Flujo de interacción	- El administrador accede al módulo de "Configuración de normativa" en el panel administrativo de la web. - El sistema recupera los valores de umbrales actuales desde la colección de configuración en la base de datos InfluxDB. - El administrador modifica los límites de toxicidad, por ejemplo, niveles de "Bueno", "moderado", "Dañino" siguiendo las guías actualizadas de la OMS. - El frontend envía la nueva matriz de parámetros al backend, el cual valida que los rangos sean lógicos y numéricamente consistentes.

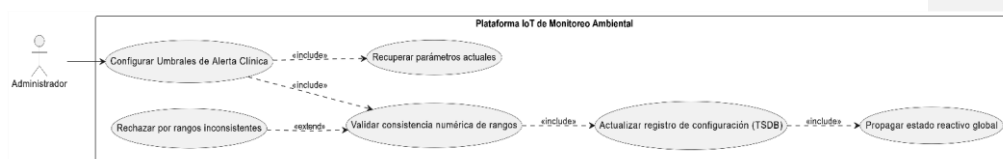
- El backend actualiza el documento de configuración en MongoDB y emite una señal de refresco de caché para los servicios de monitoreo.
- El sistema confirma la actualización exitosa y ajusta automáticamente el color de los indicadores en el dashboard de todos los usuarios activos.

Comentario El sistema debe mantener un registro de la versión de la normativa aplicada para permitir la trazabilidad histórica de los cambios de criterio en las alertas.

Nota. Elaboración propia basado en el flujo de configuración para los umbrales de alerta

Figura 20

Caso de Uso graficado CU-07



Nota. Adaptado de la tabla 20 y diseñado con StarUML. CC BY 2.0.

H. CU-08:

Tabla 21

Caso de Uso CU-08

	Detalle
Identificador	CU-08
Nombre	Calibrar Sensor Remotamente
Descripción	Permite ajustar los parámetros de conversión analógico-digital del hardware físico para corregir desviaciones en las lecturas de gas y partículas.
Precondición	El técnico debe tener una sesión activa y el Nodo Sensor debe estar suscrito al tópico de control en el Broker MQTT de AWS.
Postcondición	El microcontrolador actualiza su lógica de cálculo en tiempo real y el sistema registra la nueva configuración en la base de datos.
Actores	Administrador, Nodo Sensor, AWS Broker MQTT
Flujo de interacción	• El Administrador ingresa los nuevos coeficientes de calibración para el sensor específico a través del panel técnico en la interfaz de React.js. • El backend valida la consistencia de los parámetros y genera un comando estructurado bajo el protocolo de mensajería del sistema. • El sistema publica el comando de actualización en el tópico de suscripción del hardware mediante el Broker MQTT alojado en la infraestructura de AWS.

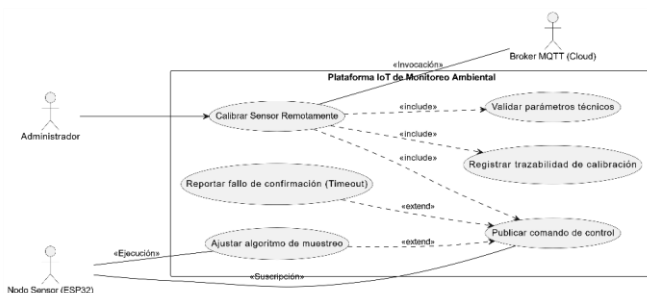
- El nodo Sensor recibe la instrucción de forma inalámbrica, actualiza los valores en su memoria volátil y ajusta inmediatamente su algoritmo de muestreo.
- El hardware envía un mensaje de confirmación ACK hacia el servidor para validar que la nueva configuración ha sido aplicada correctamente.
- El backend registra la fecha y los valores de la nueva calibración en InfluxDB para poder mantener la trazabilidad y notifica el éxito de la operación en el frontend.

Comentario Si el nodo no confirma la recepción en un tiempo determinado, el sistema alertará al técnico sobre un posible fallo de comunicación o latencia exclusiva.

Nota. Elaboración propia basado en el flujo de calibración remota

Figura 21

Caso de Uso graficado CU-08



Nota. Adaptado de la tabla 21 y diseñado con StarUML. CC BY 2.0.

3. CU-09:

Tabla 22

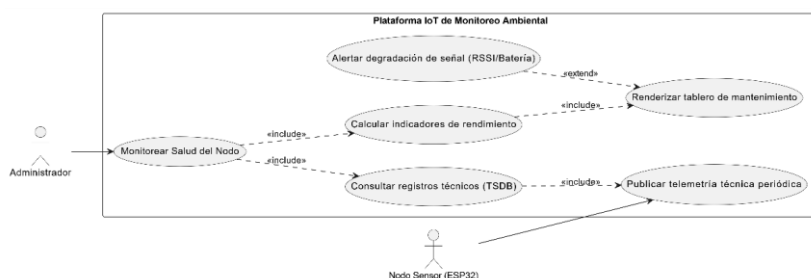
Caso de Uso CU-09

Detalle	
Identificador	CU-09
Nombre	Monitorear Salud del Nodo
Descripción	Permite al técnico supervisar el estado operativo del hardware, analizando variables críticas como la intensidad de señal Wi-Fi, el nivel de batería y el tiempo de actividad.
Precondición	Los nodos sensores deben estar enviando paquetes de telemetría técnica de forma periódica a la infraestructura de red.
Postcondición	El técnico obtiene un diagnóstico preciso del estado físico y lógico de cada punto de monitoreo en el barrio.

Actores	Administrador, Nodo Sensor
Flujo de interacción	- El Administrador accede al panel de "Mantenimiento técnico" dentro de la plataforma web en React.js. - El frontend solicita al backend la última ráfaga de datos técnicos y telemetría de todos los dispositivos registrados. - El backend realiza una consulta a la base de datos externa InfluxDB para extraer los valores de señal, voltaje y última conexión. - El sistema de base de datos devuelve los registros de salud técnica ordenados por marca de tiempo hacia el servidor. - El backend procesa los datos y calcula los indicadores de rendimiento, por ejemplo, el porcentaje de paquetes perdidos o degradación de señal. - El sistema renderiza en la interfaz una serie de indicadores visuales
Comentario	El sistema resaltará en color ámbar o rojo aquellos nodos cuyo nivel de batería o señal Wi-Fi se encuentre por debajo de los umbrales operativos mínimos.
Nota. Elaboración propia basado en el flujo de calibración remota	

Figura 22

Caso de Uso graficado CU-09



Nota. Adaptado de la tabla 22 y diseñado con StarUML. CC BY 2.0.

J. CU-10:

Tabla 23

Caso de Uso CU-10

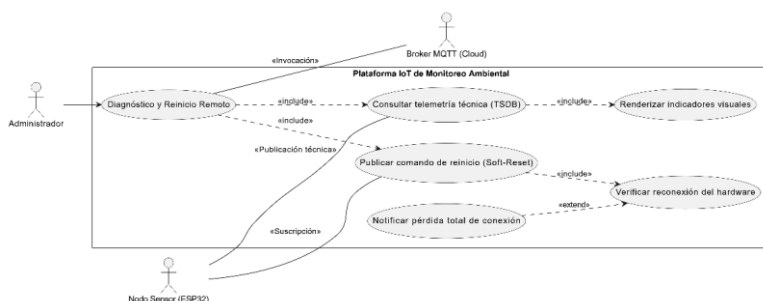
	Detalle
Identificador	CU-10
Nombre	Reiniciar Nodo Remotamente
Descripción	Permite enviar una instrucción de reinicio lógico al microcontrolador para restaurar su estado operativo en caso de latencia alta o errores de ejecución.

Precondición	Los nodos sensores deben estar enviando paquetes de telemetría técnica de forma periódica a la infraestructura de red.
Postcondición	El técnico obtiene un diagnóstico preciso del estado físico y lógico de cada punto de monitoreo en el barrio.
Actores	Administrador, Nodo Sensor, Broker MQTT
Flujo de interacción	- El administrador accede al panel de "Mantenimiento técnico" dentro de la plataforma web en React.js. - El frontend solicita al backend la última ráfaga de datos técnicos y telemetría de todos los dispositivos registrados. - El backend realiza una consulta a la base de datos externa InfluxDB para extraer los valores de señal, voltaje y última conexión. - El sistema de base de datos devuelve los registros de salud técnica ordenados por marca de tiempo hacia el servidor. - El backend procesa los datos y calcula los indicadores de rendimiento, por ejemplo, el porcentaje de paquetes perdidos o degradación de señal. - El sistema renderiza en la interfaz una serie de indicadores visuales
Comentario	El sistema resaltará en color ámbar o rojo aquellos nodos cuyo nivel de batería o señal Wi-Fi se encuentre por debajo de los umbrales operativos mínimos.

Nota. Elaboración propia basado en el reinicio remoto planeado para el nodo

Figura 23

Caso de Uso graficado CU-10



Nota. Adaptado de la tabla 23 y diseñado con StarUML. CC BY 2.0.

K. CU-11:

Tabla 25

Caso de Uso CU-11

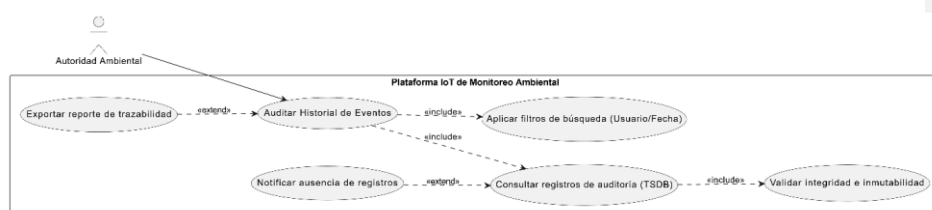
Detalle

<i>Identificador</i>	CU-11
<i>Nombre</i>	Auditar Logs
<i>Descripción</i>	Permite la supervisión y revisión de la bitácora de eventos para identificar qué usuarios realizaron cambios críticos en el sistema (Registros, calibraciones o reinicios).
<i>Precondición</i>	El Usuario debe tener el rol de "Autoridad Ambiental" y el sistema debe haber registrado previamente las acciones en la colección de auditoría.
<i>Postcondición</i>	El analista obtiene una visión transparente y cronológica de la operatividad administrativa de la plataforma.
<i>Actores</i>	Administrador, Autoridad Ambiental
<i>Flujo de interacción</i>	- El administrador ingresa al módulo de "Auditoría de Sistema" en la interfaz de usuario para revisar la trazabilidad de las acciones. - El frontend solicita al servidor la lista de eventos, permitiendo filtrar por usuario, tipo de acción o rango de fechas. - El backend realiza una consulta a la colección de logs en la base de datos externa InfluxDB para extraer los registros de actividad. - El motor de la base de datos devuelve los documentos que incluyen el ID del usuario, la marca de tiempo, la dirección IP y la descripción de la acción realizada. - El backend procesa la información y la envía al cliente asegurando que la data sea inmutable y esté debidamente formateada para su lectura. - El frontend renderiza una tabla interactiva que permite a la autoridad inspeccionar cada evento ocurrido en la infraestructura.
<i>Comentario</i>	El sistema garantizará que los registros de auditoría no puedan ser modificados ni eliminados por ningún usuario, cumpliendo con estándares de seguridad de la información.

Nota. Elaboración propia basado en el análisis del flujo en la exportación de datos auditables

Figura 24

Caso de Uso graficado CU-11



Nota. Adaptado de la figura 24 y diseñado con StarUML. CC BY 2.0.

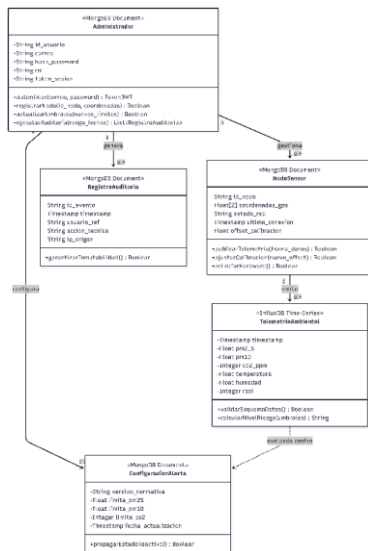
3.4.1.2. Diagrama de Clases

Para abstraer la estructura estática del sistema, se realiza el siguiente diseño del diagrama de clases enfocado en el modelo de dominio. En el contexto de la arquitectura backend planteada a desarrollar en Python, estas clases representan los esquemas de validación de datos (Data Transfer Objects) y las entidades lógicas que interactúan con el motor de InfluxDB y Broker MQTT.

En la figura 26 se ilustra las propiedades, métodos y multiplicidades de las entidades centrales del presente proyecto.

Figura 25

Diagrama de Clases



Comentado [5]: Aquí se habla de InfluxDB, y sobre las entidades de MongoDB?

Nota. figura 25 en base al diseño propuesto y diseñado con StarUML. CC BY 2.0.

3.1.5. Planificación de Sprints

La implementación del sistema se divide en tres iteraciones (Sprints), priorizando la infraestructura técnica antes que la estética visual.

Sprint 1: Infraestructura de Red, Seguridad y Gestión de Nodos

Objetivo: Establecer la base de comunicación segura entre el **hardware** y la nube de AWS, habilitando el control administrativo. Dentro del presente **sprint** estarán proyectadas las tareas relacionadas con las historias de usuario: HU-01, HU-02, y HU-03. En la tabla 26 se pueden ver el orden y clasificación de tareas estimando su prioridad y tiempo de ejecución.

Comentado [6]: Usa: Hardware

Comentado [7]: Usa: Sprint

Tabla 26

Sprint 1

Código	Actividad / Tarea	Prioridad	Estimación
T1-1	Configuración del entorno de backend (FastAPI) y conexión con InfluxDB.	Alta	5 días
T1-2	Desarrollo del servicio de autenticación y generación de tokens JWT (HU-01).	Alta	4 días
T1-3	Configuración del Broker AWS IoT Core y creación de políticas de seguridad.	Alta	3 días
T1-4	Implementación del firmware base en el ESP32 para el envío de telemetría (HU-02).	Media	6 días
T1-5	Diseño de la interfaz de registro de nodos y almacenamiento de metadatos (HU-03).	Media	4 días

Nota. Elaboración propia basado en el análisis de las tareas técnicas necesarias para iniciar el primer sprint

Sprint 2: Monitoreo en Tiempo Real y Lógica de Alertas

Objetivo: Desarrollar el flujo de datos dinámico y la interpretación de calidad de aire según la OMS. En este segundo Sprint se encuentran presentes las tareas relacionadas con

las historias de usuario HU-05. HU-07. Dentro de la tabla 27 se pueden encontrar la clasificación de dichas tareas.

Tabla 27

Sprint 2

Código	Actividad / Tarea	Prioridad	Estimación
T2-1	Implementación de WebSockets en el servidor para transmitir datos en tiempo real (HU-05).	Alta	5 días
T2-2	Creación del Dashboard principal en React.js con indicadores reactivos.	Alta	7 días
T2-3	Desarrollo de la lógica de negocio para escala de colores y alertas de la OMS (HU-07).	Alta	3 días
T2-4	Configuración de las reglas de AWS IoT para la persistencia de datos en InfluxDB.	Media	4 días
T2-5	Pruebas de Integración Hardware-Nube-Frontend.	Media	3 días

Nota. Elaboración propia basado en el análisis de las tareas lógicas de desarrollo para continuar la implementación en el TIC

Sprint 3: Monitoreo en Tiempo Real y Lógica de Alertas

Objetivo: Finalizar las herramientas de análisis a largo plazo y la capacidad de exportación legal de datos. En el tercer y último sprint encontramos todas las tareas relacionadas con las historias de usuario: HU-04 y HU-06, dentro de la tabla 28 se pueden contemplar las actividades a realizar para cumplir dichas historias.

Tabla 28

Sprint 3

Código	Actividad / Tarea	Prioridad	Estimación
T3-1	Desarrollo de consultas analíticas sobre InfluxDB para gráficas temporales (HU-06).	Alta	5 días

T3-2	Integración del mapa georreferenciado con Leaflet para ubicación de nodos.	Media	4 días
T3-3	Optimización de tiempos de respuesta en la API para consultas masivas.	Baja	3 días
T3-4	Despliegue final de la solución en infraestructura Cloud y documentación técnica.	Alta	4 días

Nota. Elaboración propia basado en el análisis de las actividades finales a realizar en el tercer sprint

3.1.6. Planificación en base al análisis

Tras el levantamiento de requerimientos y la definición de las Historias de Usuario y Casos de Uso, se determinó que la ejecución del presente proyecto de titulación no puede ser lineal, sino incremental. La planificación se fundamenta en la ruta crítica de datos: desde la percepción físicas (Sensores) hasta el punto de persistencia y visualización.

3.1.6.1. Priorización de requerimientos

Para asegurar un Producto Mínimo Viable funcional desde las primeras etapas, se aplicó un criterio de priorización basado en la dependencia técnica, siguiendo la lógica descrita a continuación:

- **Capa de conectividad:** Es crítico ya que sin una comunicación entre el ESP32 y el Broker AWS IoT Core, no existe flujo de información, por ello, el registro de nodos y la telemetría técnica tienen una prioridad máxima.
- **Capa de seguridad y persistencia:** La gestión de sesiones mediante JWT y la estructuración de base de datos (InfluxDB) deben estar listas antes de cualquier intento de visualización para garantizar la integridad de los datos.
- **Capa de presentación:** La interfaz de usuario en React.js se desarrolla sobre servicios ya probados, asegurando que el ciudadano reciba información veraz y en tiempo real.

3.1.6.2. Estrategia de Mitigación de Riesgos Técnicos

Considerando el análisis previo, se identificaron los siguientes riesgos que dictan el orden de la planificación, estos riesgos se encuentran listados en la tabla 29:

Tabla 29

Riesgos de planificación

Riesgo Identificado	Impacto	Estrategia de Mitigación
<i>Inestabilidad de Red Wi-Fi</i>	Alto	Implementación de Reinicio Remoto (CU-10) y monitoreo de señal en el Sprint 1.
<i>Saturación de InfluxDB</i>	Medio	Diseño de consultas agregadas y límites de exportación (CU-11) desde la fase de desarrollo.
<i>Latencia en datos en Vivo</i>	Medio	Uso de WebSockets para evitar el refresco constante de la página y optimizar el consumo de recursos.

Nota. Elaboración propia basado en el análisis de las actividades finales a realizar en el tercer sprint

3.1.6.3. Estimación de Recursos y Esfuerzo

Para garantizar la viabilidad del proyecto de titulación relacionado con el monitoreo ambiental en el Barrio “El Rosario”, se ha realizado una cuantificación detallada de los elementos necesarios. Esta estimación no solamente contempla los componentes físicos, sino el esfuerzo intelectual y tecnológico requerido para la integración de la infraestructura IoT.

I. **Talento Humano y Esfuerzo de Desarrollo**, el proyecto es ejecutado por un único investigador, quien asume roles multidisciplinarios. El esfuerzo se calcula en base a una jornada estimada de 20 horas semanales durante un periodo de 12 semanas (un total de 240 horas-hombre), distribuidas de la siguiente manera:

- **Analista de sistemas (15%):** levantamiento de requerimientos, definición de historias de usuario y diagramación UML.
- **Desarrollador de firmware (30%):** programación de microcontroladores ESP32, gestión de protocolos MQTT y calibración de sensores de CO_2 y $PM_{2.5}$.
- **Ingeniero Cloud/Backend (30%):** configuración de la infraestructura en AWS, gestión de base de datos NoSQL y seguridad JWT.

- **Desarrollador Frontend (25%):** Diseño de la interfaz en React.js, integración de mapas y visualización de datos en tiempo real.

II. **Recursos de Hardware para la capa de Percepción y Desarrollo**, se detallarán únicamente los insumos físicos críticos para la captura de datos y el entorno de construcción:

- **Dispositivos finales:** Microcontrolador ESP32 con Módulo Wi-Fi integrado.
- **Instrumentación:** sensores de partículas finas $PM_{2.5}$, sensores Infrarrojos de CO_2 y módulos de gestión de energía.
- **Hardware de desarrollo:** Laptop con capacidad de generar contenedores en Docker y compilación de *firmware*.

Comentado [8]: Se usó virtualización?

III. **Recursos de Software y Servicios Cloud**, la arquitectura se apoya en herramientas de código abierto y servicios bajo demanda para minimizar costos iniciales y maximizar la escalabilidad:

- **Entornos de desarrollo:** Visual Studio Code (IDE), Arduino IDE, Postman para pruebas de API.
- **Gestión de Datos:** MongoDB Atlas para la persistencia documental e InfluxDB Cloud para el manejo de series temporales de alta frecuencia.
- **Plataformas IoT:** AWS IoT Core para la gestión del bróker de mensajería MQTT, autenticación segura de los dispositivos mediante certificados mTLS y el enrutamiento bidireccional de datos hacia la nube.

IV. **Resumen de Carga de Trabajo por Fase**

Comentado [9]:

En la tabla 30 se puede reflejar de manera resumida y concreta la carga de trabajo por cada fase del proyecto.

Tabla 30

Carga de trabajo por fases

Fase del Proyecto	Esfuerzo (HH)	Nivel de Complejidad	Entregable Principal
Análisis y Requisitos	36h	Media	Documentos de especificación
Desarrollo de Firmware	72h	Alta	Código fuente del ESP32
Infraestructura y API	72h	Alta	Servicios Cloud y Endpoints
Interfaz y Reportes	60h	Media	Aplicación Web Funcional

Nota. Elaboración propia basado en el desglose de las fases del proyecto

Comentado [10]:

Comentado [11]: Innecesario

Capítulo Cuatro

Desarrollo de la Solución

El presente capítulo detalla la fase de construcción e implementación de la red de monitoreo epidemiológico. Para poder garantizar la trazabilidad y calidad del **software** a construir, la construcción se rige bajo una metodología ágil iterativa, dividida en ciclos de desarrollo (Sprints). El proceso inicia con un Sprint 0 fundamental destinado a la arquitectura estructural, seguido de Sprints incrementales donde se codifican, prueban y validan las Historias de Usuario definidas en el marco de los requisitos funcionales.

Comentado [12]: Usa: Software

4.1. Sprint 0: Diseño de la solución (Modelo C4)

El Sprint 0 constituye la fase de cimentación técnica del proyecto. Para modelar la arquitectura de la solución, se adopta el estándar C4 (Contexto, Contenedores, Componentes y Código). Este enfoque jerárquico permite abstraer la complejidad del sistema IoT, ofreciendo diferentes niveles de profundidad visual, desde la interacción que tendrá el ciudadano con el ecosistema global hasta las rutinas internas del servidor Backend.

4.1.1. Diseño lógico de la Base de Datos (Persistencia Híbrida)

Comentado [13]: No se nombre o habla de MongoDB

En concordancia con el modelo arquitectónico y el diagrama de clases establecido en capítulos anteriores, el diseño lógico de la persistencia no solamente se limita a un único motor, sino que implementa una arquitectura políglota para poder satisfacer dos naturalezas transaccionales distintas dentro del sistema:

Tenemos a MongoDB como la base de datos Documental NoSQL, esta se encarga de gestionar la información de estado, configuración de la plataforma y seguridad del sistema. Entidades como los perfiles de administradores, el catálogo de nodos físicos desplegados y los umbrales dinámicos de alerta se estructuran bajo un esquema de colecciones y documentos BSON, el cual es un JSON Binario. Este paradigma es óptimo para los datos de lectura frecuente y actualización moderada. Además, ofrece una alta flexibilidad; si en etapas futuras del proyecto se requiere añadir nuevos metadatos a la configuración de un dispositivo, el esquema documental lo admite nativamente sin requerir migraciones estructurales complejas en la base de datos.

InfluxDB es nuestra base de datos de series temporales. A diferencia de las entidades administrativas, el monitoreo ambiental continuo genera un flujo masivo, ininterrumpido y de alta frecuencia de telemetría. Almacenar esta información histórica en una base documental tradicional degradaría rápidamente el rendimiento del servidor y generaría cuellos de botella debido a la sobrecarga de los índices. Por esto, el esquema lógico para los sensores deja a un lado el modelo de JSON Binario y se estructura de forma nativa con el paradigma de Series Temporales.

Este motor optimiza la ingesta masiva organizando la información mediante un modelo de protocolo de línea, la tabla 31 detalla el diseño lógico y el diccionario de datos implementado para la colección principal de telemetría.

Tabla 31

Diseño lógico y diccionario de datos en InfluxDB (Measurement: telemetria_ambiental)

Clasificación Lógica	Llave / identificador	Tipo de Dato	Descripción técnica / Epidemiológica
<i>Tag (Indexado)</i>	nodo_id	String	Identificador único del microcontrolador, por ejemplo, ESP32-MAESTRO-01.
<i>Tag (Indexado)</i>	geocodigo	String	Cuadrante espacial o identificador de la zona, por ejemplo, UIO-NORTE.
<i>Field (valor)</i>	pm2_5	Float	Concentración de partículas finas ($\mu\text{g} / \text{m}^3$) y penetración alveolar.

<i>Field (valor)</i>	pm10	Float	Concentración de partículas gruesas ($\mu g / m^3$) y deposito bronquial.
<i>Field (valor)</i>	co2_ppm	Integer	Concentración de Dióxido de Carbono. Indicador de ventilación.
<i>Field (valor)</i>	temperatura	Float	Grados Celsius (°C). Variable termodinámica de control.
<i>Field (valor)</i>	humedad	Float	Porcentaje de humedad relativa. Factor de coalescencia.
<i>Field (valor)</i>	rssi	Integer	Nivel de intensidad de señal Wi-Fi (dBm) para auditoria de red.
<i>Timestamp</i>	time	UNIX Epoch	Marca de tiempo absoluta en resolución de nanosegundos.

Nota. Elaboración propia basado. La separación estricta entre Tags y Fields optimiza el uso de la memoria RAM del servidor durante la ejecución de consultas analíticas masivas.

4.1.2. Diagrama de Componentes

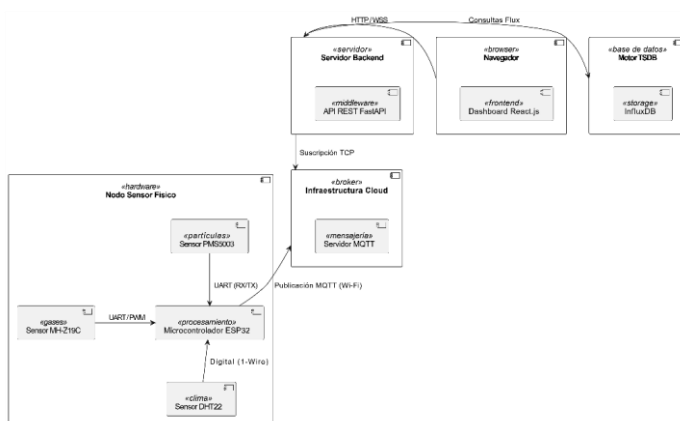
El diagrama de componentes ilustra la arquitectura de software y hardware del sistema, detalla el flujo de información de extremo a extremo a través de sus módulos principales, los cuales se comunican mediante protocolos estandarizados. La captura de datos inicia en el sensor físico, donde el microcontrolador ESP32 actúa como el núcleo de procesamiento a nivel local. Este componente consolida la información interactuando con la instrumentación a través de buses específicos, se encarga de leer las concentraciones de partículas del sensor PMS5003 mediante un puerto serial UART (RX/TX), logra obtener los niveles de gases del MH-Z19C a través de una interfaz UART y captura las variables climáticas desde el sensor DHT22 usando un bus digital.

Cuando ya se tienen las lecturas estructuradas, el microcontrolador utiliza su conectividad Wi-Fi para enlazarse con la nube. La transferencia se ejecuta a través de una publicación MQTT hacia el servidor de mensajería el cual funciona como un bróker intermediario. De forma simultánea, el servidor *backend*, se encarga de encapsular el API REST que se desarrolló en Fast API como la capa del medio, tiene una suscripción TCP asíncrona con el bróker. Esta arquitectura por eventos permite que el servidor pueda consumir los mensajes en el instante exacto en que se generan.

Para poder lograr la persistencia de este volumen masivo de datos el servidor *backend* interactúa con el motor de InfluxDB ejecutando consultas, permitiendo optimizar la inserción de forma cronológica en el componente de almacenamiento. Por último, la información procesada se expone en el navegador del usuario; el *frontend* interactivo ejecutado en React.js solicita los datos históricos y en tiempo real al servidor mediante peticiones HTTP. Este flujo de extremo a extremo culmina con una visualización reactiva que opera de forma totalmente desacoplada de la complejidad de la infraestructura física subyacente.

Figura 26

Diagrama de componentes



Comentado [14]: Se debe agregar una explicación de la interacción del diagrama de componentes.

Nota. figura 26 en base al diseño del diagrama de componentes y diseñado con StarUML. CC BY 2.0

4.1.3. Arquitectura lógica y física

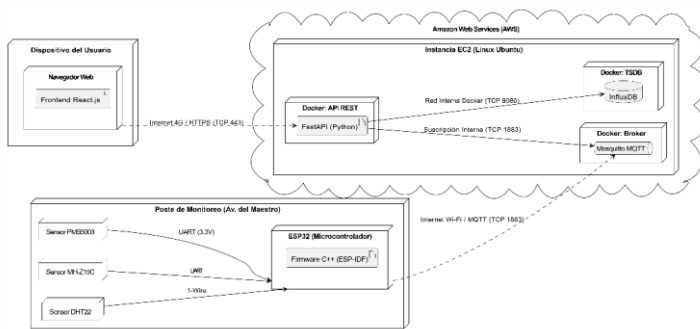
Para poder realizar un mapeo de como los artefactos de software se ejecutan sobre la infraestructura de *hardware* e *internet*, se implementa un Diagrama de despliegue UML, dentro de este esquema topológico se ilustra los nodos de procesamiento físico, los protocolos de comunicación de red y la distribución de los contenedores lógicos en la nube de AWS.

La arquitectura física se divide en tres entornos geográficamente separados: el entorno de captación en la vía pública, el clúster de procesamiento en la nube, y el dispositivo cliente del ciudadano, todo esto se refleja en la siguiente figura.

Comentado [15]: Usar: Internet

Figura 27

Diagrama de Topología de Red.



Nota. figura 27 en base al diseño y análisis de la topología de red, diseñado con StarUML. CC BY 2.0

Comentado [16]: Revisar y agregar/modificar, el uso de Render

El análisis de los componentes estructurales de la arquitectura es el siguiente, en la capa de presentación el navegador contiene el módulo desarrollado en React.js, encargado de renderizar la interfaz y procesar el estado reactivo. Este componente interactúa con el servidor mediante peticiones HTTP para poder acceder a la telemetría histórica en tiempo real. Como núcleo lógico, el servidor backend aloja el marco de trabajo de FastAPI, actúa como el orquestador que se encarga de validar los esquemas de datos, genera la autenticación JWT y coordina la comunicación bidireccional. Hay que destacar que tanto el frontend como el API REST del backend se encuentran desplegados de forma nativa en render, esta es una plataforma como servicio en la nube que nos garantiza la disponibilidad necesaria, integración continua y conectividad segura mediante certificados HTTPS para ambas capas.

Por otro lado, el motor de la base de datos temporales es InfluxDB, siendo el componente de almacenamiento que está disponible para soportar una alta frecuencia de escritura y consultas analíticas temporales masivas. Al hablar de conectividad de dispositivos, la infraestructura en la nube en AWS provee la mensajería asíncrona gestionando las colas de tópicos MQTT. Permite desacoplar la emisión del hardware de la recepción del servidor. Finalmente, el microcontrolador actúa como el nodo sensor en

campo, cuya función principal es compilar las lecturas eléctricas de los sensores en tramas digitales estructuradas para publicarlas después hacia el bróker mediante la lógica de su firmware integrado.

4.1.3. Arquitectura lógica y física

Para poder mitigar riesgos técnicos y validar la experiencia de usuario (UX) previo a la etapa de codificación en React.js, se desarrolló un conjunto de prototipos no funcionales de fidelidad media. El diseño de estas interfaces se llevó a cabo utilizando la herramienta colaborativa Figma (<https://www.figma.com/>), aplicando una arquitectura de información orientado a un diseño web adaptable. Este enfoque garantiza que esta plataforma sea totalmente accesible y funcional tanto en monitores de escritorio como centros de control como en navegadores de celulares, permitiendo a la ciudadanía consultar la calidad del aire desde cualquier dispositivo sin requerir la instalación de una aplicación nativa.

Comentado [17]: Seguro son de "alta fidelidad"

4.1.4.1. Sistema de Diseño Y Paleta Semántica

Dentro del diseño visual se debe pensar el mismo siguiendo las normativas de salud pública de parte de la Organización Mundial de la Salud (WHO, 2021). Para ello se implementó una paleta de colores semántica basada en los índices estandarizados, lo cual facilita a los usuarios una rápida interpretación del nivel de riesgo asociado a los datos ambientales.

Comentado [18]: Agregar referencia de esto

- Tonos fríos (verdes/azules): indican niveles óptimos de la calidad del aire, por ejemplo, PM2.5 bajo, CO₂ normal.
- Tonos cálidos (naranjas/rojos): alertan sobre umbrales de toxicidad que requieren precaución o representan un peligro respiratorio.

A continuación, se detallan las vistas principales prototipadas que dan cobertura a los Casos de Uso del sistema:

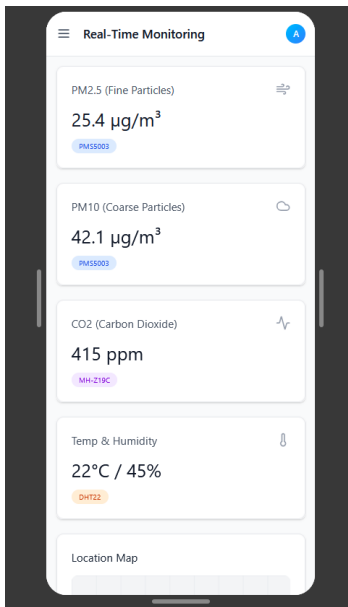
Dashboard Principal (Vista Ciudadana)

Representa la interfaz principal de telemetría. Su diseño incluye tarjetas de lectura rápida para los contaminantes principales (PM2.5, PM10, CO₂ Temperatura y Humedad) y

un área destinada a la renderización de gráficas vectoriales de series temporales para el análisis de tendencias históricas.

Figura 28

Prototipo de Figma: Dashboard de Monitoreo Ambiental



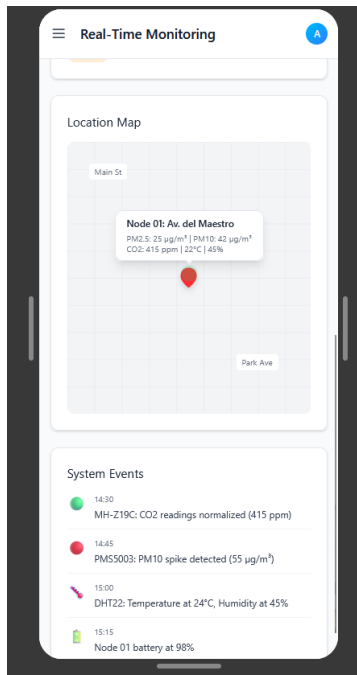
Nota. figura 28 en base al diseño y Caso de Uso, diseñado con Figma. CC BY 2.0

Mapa Georreferenciado de Nodo

Interfaz diseñada para la ubicación espacial del sensor en el sector de estudio. El prototipo ilustra la intergración de cartografía digital de la mano de la herramienta Leaflet, donde los marcadores interactivos por colores indican el estado de salud de la red y el nivel de contaminación focalizada.

Figura 29

Prototipo de Figma: Mapa Interactivo de nodo sensor



Nota. figura 29 en base al diseño y Caso de Uso, diseñado con Figma. CC BY 2.0

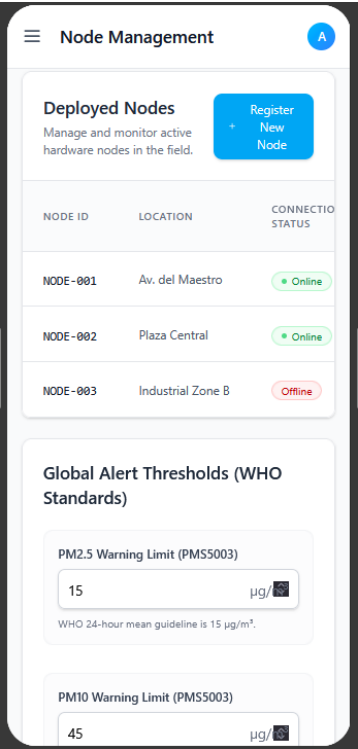
Panel de Administración y Mantenimiento Técnico

Es la vista protegida destinada al perfil que cuente con un rol de Administrador. El diseño contempla formularios limpios para la gestión de dispositivos, ajuste de calibración remota y configuración de los umbrales de alerta clínica. En la figura 30 y 31 se pueden observar los diseños planteados.

Dentro del siguiente enlace se puede acceder a dicho prototipo y las funcionalidades pensadas: <https://post-finish-01041761.figma.site/>

Figura 30

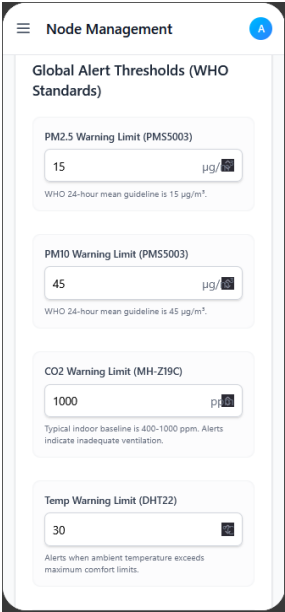
Prototipo de Figma: Administración de nodos



Nota. figura 30 en base al diseño y Caso de Uso, diseñado con Figma. CC BY 2.0

Figura 31

Prototipo de Figma: Configuración de Umbrales



Nota. figura 31 en base al diseño y Caso de Uso, diseñado con Figma. CC BY 2.0

Comentado [19]: Agregar enlace al proyecto de figma; para acceder a todo el prototipo de pantallas

4.2. Sprint 1: Arquitectura Base, Seguridad y Conectividad Edge-to-Cloud

El sprint 1 constituye la fase fundamental del ciclo iterativo de desarrollo, se enfocada en establecer la infraestructura tecnológica central del sistema IoT. Bajo la metodología ágil Scrum adaptada para proyectos que combinan hardware y software, este primer ciclo le dio prioridad al despliegue de los servicios en nube los cuales son el Backend y la base de datos, la configuración de la seguridad perimetral y la programación inicial de los microcontroladores en campo.

El objetivo principal de esta fase fue habilitar un canal de comunicación seguro mediante el protocolo MQTT y establecer el modelo de persistencia híbrida. Esto garantizó que la plataforma central estuviera completamente operativa y fortificada antes de iniciar el

ensamblaje físico y la calibración de los sensores de calidad del aire SDS011 y MH-19B. Al desacoplar el desarrollo del hardware y del software desde un inicio, se lograron mitigar los riesgos tempranos de latencia y cuellos de botella en la red.

4.2.1. Backlog y Planificación del Sprint 1

A partir del análisis de requisitos del sistema, se extrajeron las Historias de Usuario y se desglosaron en tareas técnicas específicas para este primer ciclo. La priorización del backlog se enfocó en actividades de criticidad “Alta”, indispensables para la orquestación de la API RESTful, el almacenamiento en InfluxDB y la generación de políticas de acceso criptográfico en AWS IoT Core.

A continuación, en la tabla 32, se detalla la planificación estructurada de las actividades a realizar, estableciendo el código de trazabilidad, el nivel de prioridad arquitectónica y la estimación del tiempo en días requerida para la ejecución de cada tarea.

Tabla 32

Sprint 1

Código	Actividad / Tarea	Prioridad	Estimación
T1-1	Configuración del entorno de backend (FastAPI) y conexión con InfluxDB.	Alta	5 días
T1-2	Desarrollo del servicio de autenticación y generación de tokens JWT (HU-01).	Alta	4 días
T1-3	Configuración del Broker AWS IoT Core y creación de políticas de seguridad.	Alta	3 días
T1-4	Implementación del firmware base en el ESP32 para el envío de telemetría (HU-02).	Media	6 días
T1-5	Diseño de la interfaz de registro de nodos y almacenamiento de metadatos (HU-03).	Media	4 días

Nota. Elaboración propia.

4.3. Desarrollo de las Tareas dentro del Sprint 1

T1-1: Configuración del entorno de backend (FastAPI) y conexión con InfluxDB.

El desarrollo de la capa de procesamiento inicio con la estructuración del orquestador central utilizando el framework FastAPI basado en Python. Se seleccionó dicha tecnología por su soporte nativo para procesos asíncronos ASGI, una característica indispensable para manejar eficientemente la alta concurrencia de peticiones en arquitecturas IoT.

Para establecer el modelo de persistencia híbrida, previo a integrar esto en código, se ejecutó el aprovisionamiento de los motores de bases de datos. Por un lado, se configuró el entorno de InfluxDB definiendo la organización, el bucket de almacenamiento y los tokens de seguridad necesarios para la escritura de las métricas. Paralelamente, se desplegó un clúster en MongoDB Atlas, el cual cuenta con una estructura de colecciones documentales iniciales para el ecosistema.

Tras llevar a cabo la configuración de los entornos en la nube, se procedió a configurar los módulos de comunicación dentro del servidor central. Inicialmente, se articuló el conector hacia InfluxDB, aprovechando su arquitectura orientada a columnas y especializada en series cronológicas para absorber y resguardar flujos continuos de mediciones ambientales a alta velocidad. En paralelo, se integró el controlador oficial para MongoDB Atlas, asignado a esta base de datos orientada a documentos el control del estado general de la página y la gestión de la lógica operativa.

Su implementación permite almacenar configuraciones dinámicas como los parámetros y umbrales de alerta global, el catálogo de los nodos físicos y las credenciales de los perfiles de usuario, garantizando un control administrativo eficiente y totalmente desacoplado de la telemetría masiva, dentro de la figura 32 se puede evidenciar un fragmento del código correspondiente al Backend con la configuración de dichas bases de datos, de igual forma en el siguiente enlace al repositorio de GitHub se lo puede evidenciar:

<https://github.com/fmejia666/tesis-iot-ambiental/blob/main/backend/main.py>

Comentado [20]: En cada Sprint de debe dejar un enlace al repositorio de GitHub, donde se inicia el proceso.

Figura 32

Fragmento de código ilustrando la inicialización y conexión a las bases de datos InfluxDB y MongoDB

```

1 from fastapi import FastAPI, HTTPException
2 from fastapi.middleware.cors import CORSMiddleware
3 from pydantic import BaseModel
4 from typing import List, Optional
5 from pymongo import MongoClient
6 from bson import ObjectId
7 from datetime import datetime
8 from influxdb_client import InfluxDBClient, Point
9 from influxdb_client.client.write_api import SYNCHRONOUS
10 from pymongo import MongoClient
11 from pymongo.errors import PyMongoError
12 from fastapi.middleware.cors import CORSMiddleware
13
14 INFLUX_URL = "https://us-east-2-1.aws.cloud.influxdata.com"
15 INFLUX_TOKEN = "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9"
16 INFLUX_ORG = "nodata"
17 INFLUX_BUCKET = "weather_data"
18
19 influx_client = InfluxDBClient(url=INFLUX_URL, token=INFLUX_TOKEN, org=INFLUX_ORG)
20 write_api = influx_client.write_api(write_options=SYNCHRONOUS)
21 query_api = influx_client.query_api
22
23 MONGO_URI = "mongodb://[username]:[password]@[hostname]:27017/?authSource=admin"
24 mongo_client = MongoClient(MONGO_URI)
25 db_mongo = mongo_client["weather_data"]
26 collection_mongo = db_mongo["weather_data"]
27
28 read_client = MongoClient("mongodb://localhost:27017/?authSource=admin")
29 read_db = read_client["weather_data"]
30 read_collection = read_db["weather_data"]
31
32 app = FastAPI(title="Backend HealthZuf - API")
33
34 app.add_middleware(
35     CORSMiddleware,
36     allow_origins=["*"],
37     allow_credentials=True,
38     allow_methods=["*"],
39     allow_headers=["*"],
40 )
  
```

Nota. Captura tomada desde el entorno local de desarrollo

T1-2: Desarrollo del servicio de autenticación y generación de tokens JWT (HU-01).

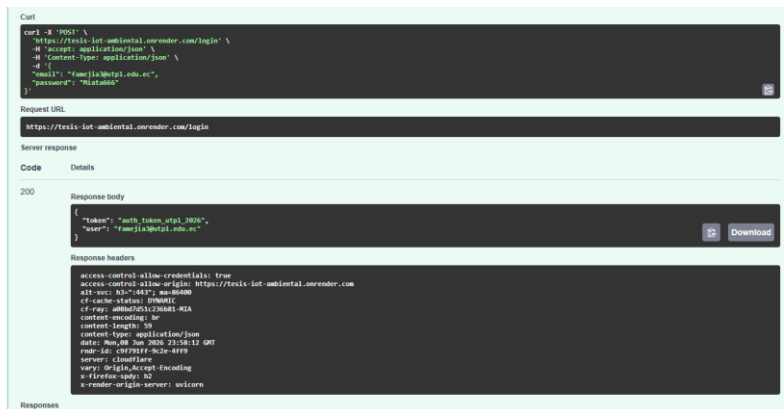
Para garantizar la seguridad perimetral a nivel de software y proteger el panel de control, se desarrolló un módulo de autenticación y control de accesos. Esta tarea dio cumplimiento a la Historia de Usuario 01, estableciendo las bases para la comunicación segura entre el cliente web y el servidor.

Para poder materializar el requerimiento, se construyó el endpoint POST /login el cual está expuesto en el entorno de producción en el siguiente enlace: <https://tesis-iot-ambiental.onrender.com/docs#/>. A nivel técnico, este servicio recibe las credenciales del cliente y las procesa con modelos de validación de pydantic, lo que garantiza que los datos sean válidos en su entrada antes de contrastarlos con la colección de administradores en MongoDB. Para asegurar la confidencialidad de la información, las contraseñas almacenadas se gestionan utilizando el algoritmo de hashing criptográfico de una vía Bcrypt.

Tras una validación exitosa de las credenciales, la API RESTful procede a la firma digital utilizando el algoritmo simétrico HS256 para poder generar y retornar un JSON Web Token. Dicho token encapsula una carga que contiene el identificador del usuario y una marca de tiempo de expiración. Este componente permite implementar un mecanismo de autorización sin estado, mediante el cual la plataforma web desarrollada en React.js adjunta automáticamente el JWT en las cabeceras de autorización HTTP de sus futuras peticiones, logrando así consumir de forma segura las rutas protegidas del sistema. Dentro de la figura 33 podemos observar dicho proceso de validación a través de Swagger UI. De igual forma en el siguiente enlace al repositorio de GitHub se lo puede evidenciar:

Figura 33

Validación del servicio de autenticación y respuesta del token JWT mediante Swagger UI.



Nota. Captura tomada desde Swagger

T1-3: Configuración del Broker AWS IoT Core y creación de políticas de seguridad.

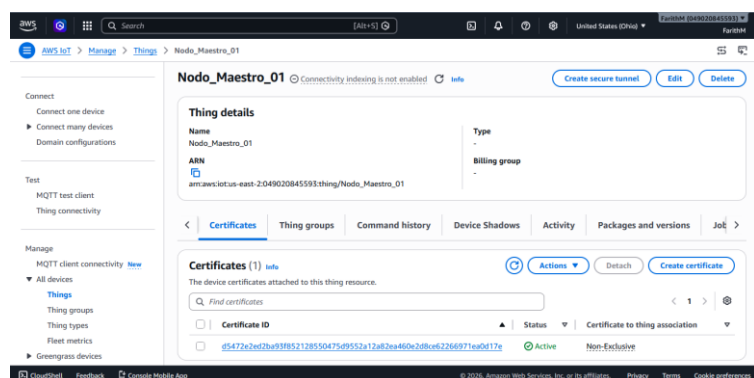
Para la capa de comunicación y red, se descartaron los brokers públicos y se aprovisionó la infraestructura utilizando AWS IoT Core. Esta elección al diseñar permitió tener una infraestructura caracterizada por una elevada tolerancia a fallos y un control de accesos perimetral riguroso. El procedimiento se centró en la creación de una entidad lógica denominada "Thing" o dispositivo virtual en la consola de AWS. Dicho componente sirve como el reflejo abstracto y punto de enlace del dispositivo físico en la nube, facilitando el monitoreo continuo de su operatividad, la administración integral de su ciclo de vida y la asociación unívoca con los certificados de seguridad asignados.

El aspecto más crítico dentro de la presente tarea fue la habilitación de la autenticación mutua mTLS, la cual se hizo desde la plataforma, se generaron y descargaron dichos certificados criptográficos X.509: el certificado del dispositivo, la llave privada y el certificado raíz Root CA de Amazon. Para que la comunicación segura sea efectiva, dichas credenciales fueron embebidas directamente en el ESP32. De esta manera, el hardware utiliza dichas llaves para ejecutar el handshake y cifrar el canal de comunicación antes de enviar cualquier trama de datos, garantizando que tanto el cliente como el servidor validen mutuamente sus identidades.

De forma adicional, se estructuraron las políticas de seguridad de AWS IoT en formato JSON para el dispositivo. Se aplica el principio de mínimos privilegios, el documento JSON se configuró para autorizar estrictamente la acción `iot:Connect` vinculada al identificador único del cliente del microcontrolador. De igual forma, se restringieron las acciones de mensajería, permitiendo la ejecución de `iot:Publish` e `iot:Subscribe` única y exclusivamente sobre los tópicos MQTT predefinidos para la telemetría y recepción de comandos del nodo. Esta configuración es vital porque permite mitigar el riesgo de que un dispositivo que esté comprometido pueda ser capaz de realizar inyecciones de datos de terceros o suscribirse a canales de otros nodos dentro de la red.

Figura 34

Consola de administración de AWS IoT Core detallando las políticas de seguridad y certificados mTLS



Nota. Captura tomada desde AWS del Nodo configurado

T1-4: Desarrollo de firmware para el microcontrolador ESP32 y captura de telemetría (HU-02)

La ejecución de esta tarea tuvo en su desarrollo la programación e implementación del firmware base en los microcontroladores ESP32, satisfaciendo los requerimientos funcionales establecidos en la Historia de Usuario HU-02. Dentro del contexto de la arquitectura IoT propuesta, el dispositivo de borde adquiere un rol crítico ya que es el encargado directo de la adquisición sincrónica de las variables físicas y su posterior digitalización. Para garantizar una lectura eficiente, el código embebido se estructuró utilizando temporizadores no bloqueantes y el manejo adecuado de los buses de comunicación hacia los sensores periféricos, esta técnica de programación evita que el procesador se congele durante las lecturas y asegura que la conexión en red sea estable en tiempo real.

Comentado [21]: Esto es comprobable?

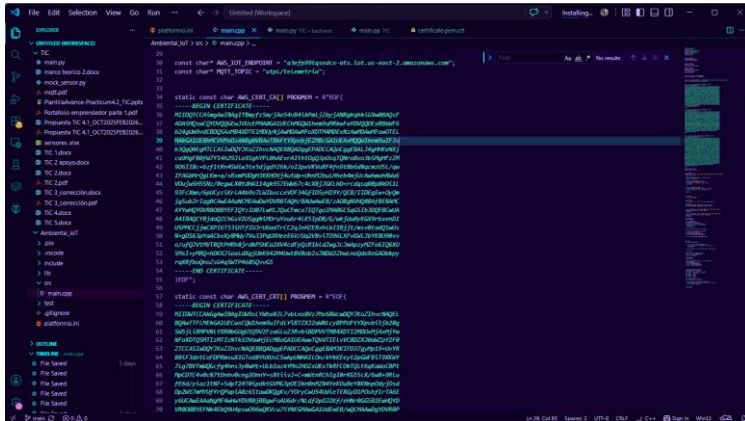
Posterior a la fase de adquisición, el firmware ejecuta rutinas de acondicionamiento y encapsulamiento de la información, estructurando las lecturas en tramas estandarizadas dentro del formato JSON para asegurarnos de que sea interoperable con los servicios en la nube. La transmisión de esta telemetría se realiza de forma continua y bidireccional a través del protocolo ligero MQTT. Con el objetivo de dotar al hardware de tolerancia a fallos, se programaron algoritmos lógicos de reconexión automática que permite salvaguardar los datos frente a interrupciones transitorias de la red inalámbrica, esto nos garantiza la resiliencia del nodo remoto.

Por último, para comprobar el desempeño de la arquitectura en condiciones reales, se completó el armado y la instalación física del sensor en el barrio "El Rosario". En el ámbito de la protección de datos, el firmware incorporó las claves y certificados criptográficos emitidos previamente en AWS. La figura 35 presenta el entorno de desarrollo y la transferencia de estas credenciales hacia la memoria del microcontrolador, en tanto que la figura 36 presenta la instalación final en

Figura 35

Codificación de Nodo ESP32 Junto a los Certificados de AWS

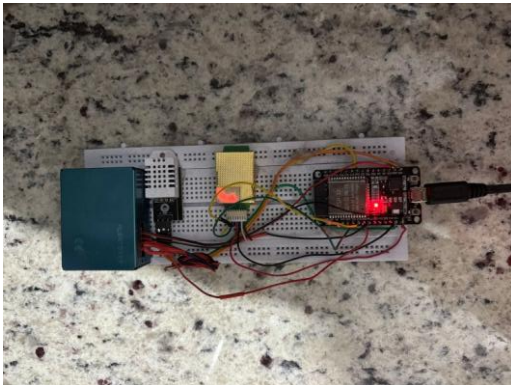
Comentado [22]: Mejorar la explicación; faltan IMGs que apoyen lo realizado; resaltar físicamente la ubicación del sensor



Nota. Captura tomada desde VS Code

Figura 36

Despliegue físico del nodo de monitoreo en el barrio El Rosario



Nota. Imagen tomada desde el punto de recolección de datos

T1-5: Diseño de la interfaz de registro de nodos y almacenamiento de metadatos

Esta última tarea del primer sprint aborda el desarrollo de la interfaz orientada al registro de nodos y la administración de sus metadatos, constituyendo un componente esencial para la escalabilidad y trazabilidad de la red de sensores. Desde la perspectiva de la ingeniería de software, esta etapa se materializó mediante la construcción de una vista administrativa en React.js, la cual emplea hooks de estado para el manejo dinámico de los formularios de ingreso. Este módulo interactivo actúa como el cliente web que consume de forma segura los endpoints del backend, lo que permite no solo dar de alta nuevas instancias de microcontroladores, sino también poder vincular cada hardware físico con identificadores lógicos únicos y parámetros operativos específicos.

A nivel técnico, el proceso de administración se ejecuta enviando peticiones asíncronas a través de métodos HTTP con el POST para la creación de nodos, PUT para la edición y DELETE para eliminación dirigidas hacia la ruta /nodos de la API construida en FastAPI. El servidor recibe la carga útil en formato JSON, valida estrictamente la estructura de los datos utilizando modelos de tipado estricto en Pydantic y, tras la validación, persiste el registro como un documento dentro de la colección respectiva en MongoDB Atlas. Al estructurar y almacenar de esta manera metadatos claves tales como la ubicación espacial del dispositivo por coordenadas, estado operativo, la plataforma adquiere la capacidad de poder gestionar dinámicamente la infraestructura física distribuida, proveyendo al usuario administrador un control centralizado y auditable sobre todo el ecosistema tecnológico.

Comentado [23]: Falta explicar técnicamente el proceso realizado aquí.

Figura 37

GUI desarrollada para la parametrización, control y registro de nodos en la plataforma.



Nota. Captura tomada desde Firefox

4.4. Desarrollo de las Tareas dentro del Sprint 2

4.4.1. Backlog y Planificación del Sprint 2

Para la ejecución del segundo sprint, se utilizó una metodología de gestión ágil, desglosando el proyecto en historias de usuario priorizadas según su valor técnico y funciona. El objetivo central de este sprint fue consolidar la capa de persistencia de datos en InfluxDB y MongoDB junto con la interfaz de usuario en React.js.

A continuación, en la tabla 33 se detalla el sprint de este periodo:

Tabla 33

Sprint 2

Código	Actividad / Tarea	Prioridad	Estado
T2-1	Implementación de WebSockets en el servidor para transmitir datos en tiempo real (HU-05).	Alta	5 días - Completado
T2-2	Creación del Dashboard principal en React.js con indicadores reactivos.	Alta	7 días - Completado
T2-3	Desarrollo de la lógica de negocio para escala de colores y alertas de la OMS (HU-07).	Alta	3 días – Completado
T2-4	Configuración de las reglas de AWS IoT para la persistencia de datos en InfluxDB.	Media	4 días - Completado
T2-5	Pruebas de Integración Hardware-Nube-Frontend.	Media	3 días - Completado

[Nota. Elaboración propia

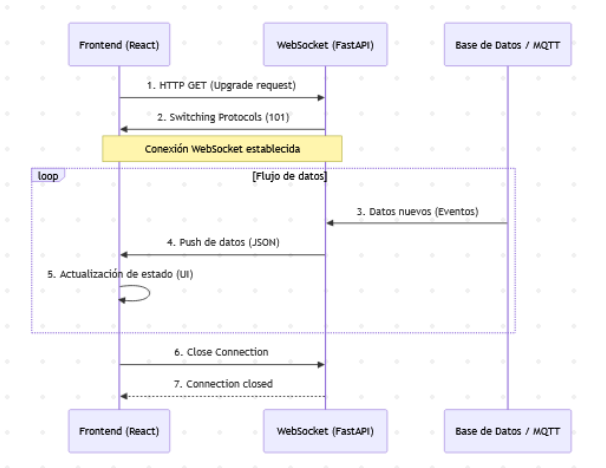
T2-1: Implementación de WebSockets en el servidor para transmitir datos en tiempo real

Esta tarea aborda la necesidad crítica de reducir la latencia en la transmisión de información desde el servidor hacia las interfaces de usuario. En sistemas de monitoreo ambiental, donde las condiciones físicas pueden fluctuar rápidamente, el modelo tradicional de peticiones HTTP resulta ineficiente y genera una sobrecarga del ancho de banda. Como solución a esto, se implementó una arquitectura basada en el protocolo WebSockets sobre el entorno de FastAPI. A nivel técnico, esto se logró estructurando un endpoint asíncrono que utiliza la clave nativa WebSocket del framework. El servicio expone una ruta bajo el esquema seguro wss://, la cual se encarga de aceptar las peticiones entrantes de los clientes mediante el método `accept()` y mantiene la conexión de red abierta, estableciendo un canal bidireccional, persistente y de baja latencia.

Desde el punto de vista de la Ingeniería de TI, esta implementación requiere que el servidor logre gestionar un registro de estado en memoria, es decir una lista de administradores con todas las sesiones activas. Permite emitir de forma directa la nueva telemetría hacia las interfaces en el instante exacto en que el backend recibe y procesa el mensaje desde el bróker MQTT, utilizando métodos asíncronos de envío como `send_json()`. Al adoptar esta topología orientada a eventos, el sistema garantiza que los tomadores de decisiones visualizarán las actualizaciones de los sensores sin tener necesidad de recargar la plataforma, cumpliendo a cabalidad con el criterio de tiempo real exigido en la historia de usuario HU-05. Dentro de la figura 38 podemos encontrar un diagrama de secuencia que expone el proceso de conexión entre estas tres entidades.

Figura 38

Diagrama de secuencia de la conexión WebSocket entre FastAPI y React



Nota. Realizado con Mermaid Live Editor

T2-2: Creación del Dashboard principal en React.js con indicadores reactivos

Comentado [24]: ¿Cómo técnicamente se hizo esto?

La Tarea T2-2 se centró en el desarrollo de la capa de presentación de la plataforma utilizando React.js, con el objetivo de transformar los datos crudos emitidos por los microcontroladores en una interfaz gráfica intuitiva. En el plano de desarrollo en el frontend, la interfaz se estructuró usando hooks nativos como `useState`, orientado a la administración del estado reactivo de las variables y `useEffect`, empleado para gobernar el ciclo de vida del aplicativo. Se adoptó un patrón arquitectónico modular fundamentado en componentes funcionales JSX, integrando tarjetas dinámicas de síntesis informativa. Para enriquecer la experiencia analítica, estos módulos se articularon con herramientas de gráficos para trazar series cronológicas y un componente cartográfico para la georreferenciación interactiva de las mediciones.

El núcleo técnico de esta fase radicó en garantizar una reactividad fluida en tiempo real. Para ello, se descartó el polling HTTP tradicional en favor de una conexión bidireccional de baja latencia. Dentro del cliente web, se instanció un objeto WebSocket cuya escucha de eventos (onmessage) fue enlazada directamente al servidor FastAPI. La implementación permite que, al recibir un nuevo mensaje, el payload de JSON entrante sea parseado e inyectado instantáneamente en los manejadores de estado. Al actualizarse el estado, React activa su algoritmo de reconciliación, calculando las diferencias y ejecutando actualizaciones de precisión sobre el DOM virtual sin necesidad de recargar la página completa.

Finalmente, el diseño de la interfaz fue optimizado para el contexto de este TIC: se aplicó un estricto formateo de un solo decimal a nivel numérico para evitar el desbordamiento visual de variables como la temperatura, y se omitieron métricas de hardware innecesarias para la experiencia ciudadana, dado que el nodo opera de forma estacionaria conectado a la red eléctrica. Estas decisiones técnicas permitieron focalizar el renderizado visual de la aplicación exclusivamente en el monitoreo ambiental.

Figura 38

Captura de pantalla de la interfaz de usuario creada en React.js



Nota. Vista de usuario al Acceder al sitio web

T2-3: Desarrollo de la lógica de negocio para escala de colores y alertas de la OMS

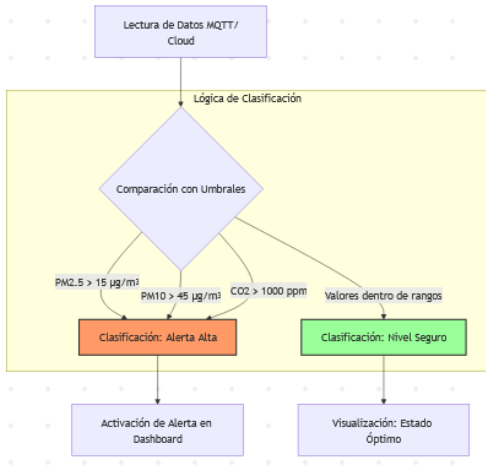
La ejecución de la tarea T2-3 representa el componente analítico y de valor agregado del sistema, transformando las lecturas físicas crudas en información semántica con relevancia para la comunidad. La captura aislada de variables como el material particulado (PM2.5, PM10) y el dióxido de carbono (CO_2) carece de utilidad si no se interpreta bajo un marco normativo estandarizado. Por consiguiente, se procedió al desarrollo e implementación de un módulo de utilidades dentro de la capa lógica, encargado de procesar cada payload de datos entrante. A nivel técnico, esto se implementó mediante estructuras condicionales que cruzan los valores de tipo float emitidos por los sensores contra un diccionario de constantes estáticas en el código; dichas constantes reflejan exactamente los umbrales de exposición definidos en las guías de calidad del aire de la OMS (Organización Mundial De La Salud).

Comentado [25]: ¿Cómo técnicamente se hizo esto?

Desde la perspectiva del desarrollo de software, esta evaluación continua alimenta un manejador de estado que categoriza dinámicamente el nivel de riesgo del entorno. En el cliente web React.js, dependiendo del rango calculado, el componente ejecuta una renderización condicional, modificando dinámicamente las propiedades CSS de la interfaz. Esto se refleja en una escala colorimétrica estandarizada: verde para condiciones óptimas, amarillo para niveles de precaución y naranja o rojo para escenarios de riesgo crítico. Adicionalmente para el disparo de alertas proactivas, se integró la notificación API nativa de los navegadores. Se programó un hook que escucha los cambios de estado y, si detecta que los límites permisibles han sido superados, invoca métodos asíncronos para desplegar una alerta tipo push directamente en la pantalla del usuario. Esta conversión de métricas cuantitativas a indicadores visuales preventivos cumple directamente con los criterios de aceptación de la historia de usuario HU-07, dotando a la plataforma de una funcionalidad crítica para la toma de decisiones. Dentro de la figura 39 podemos ver el algoritmo de clasificación de alertas y su funcionamiento.

Figura 39

Diagrama del algoritmo de la clasificación de alertas y mapeo de umbrales basados en la OMS



Nota. Realizado con Mermaid Live Editor

T2-4: Configuración de las reglas de AWS IoT para la persistencia de datos en InfluxDB

La Tarea T2-4 Aborda la arquitectura de almacenamiento permanente y la preservación histórica de la telemetría, un requisito indispensable para garantizar la trazabilidad de los datos y permitir análisis retrospectivos dentro del trabajo de integración curricular. Debido a que las lecturas provenientes del nodo IoT poseen una naturaleza puramente cronológica y de una frecuencia alta, se descartó usar un sistema relacional convencional en favor a InfluxDB Cloud, un motor que está optimizado para el manejo de series de tiempo. La persistencia de la información se estructuró a través del motor de reglas integrado en AWS IoT Core, actuando como un puente asíncrono y desacoplado entre el flujo de mensajería MQTT y el repositorio en la nube.

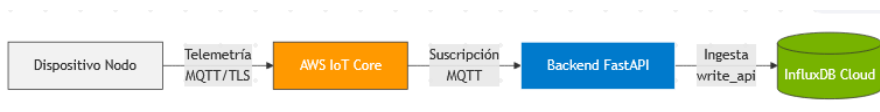
Para consolidar esta integración de forma técnica, se configuró una regla de enrutamiento utilizando la sintaxis SQL-Like nativa de AWS IoT. Específicamente, se implementó una sentencia estructurada, por ejemplo, `SELECT * FROM "telemetry/#"` para poder interceptar automáticamente los mensajes entrantes en el tópico asignado al microcontrolador. Una vez capturado el payload JSON, la regla acciona un destino de tipo HTTP Action, el cual fue configurado apuntando directamente al endpoint de la API REST de escritura de InfluxDB `/api/v2/write`. Para garantizar el acceso seguro, se inyectaron las cabeceras HTTP de autorización correspondientes, pasando el Token criptográfico del bucket de destino.

De esta forma, la regla extrae, valida y mapea los campos de interés, transformándolos al formato Line Protocol requerido por el motor y enviándolos de forma directa. El diseño de este mecanismo de ingesta automatizado asegura que el almacenamiento masivo se ejecute de manera totalmente independiente a la capa de presentación web, optimizando el uso de recursos de red, logrando prevenir la pérdida de paquetes ante ráfagas de tráfico y garantizando la consistencia temporal de cada registro mediante marcas de tiempo de precisión nanométrica. Dentro de la figura 40 podemos observar el flujo del enrutamiento hacia InfluxDB.

Comentado [26]: ¿Cómo técnicamente se hizo esto?

Figura 40

Enrutamiento de AWS IoT Core hacia InfluxDB



Nota. Realizado con Mermaid Live Editor

T2-5: Pruebas de Integración y validación del Sistema End-to-End

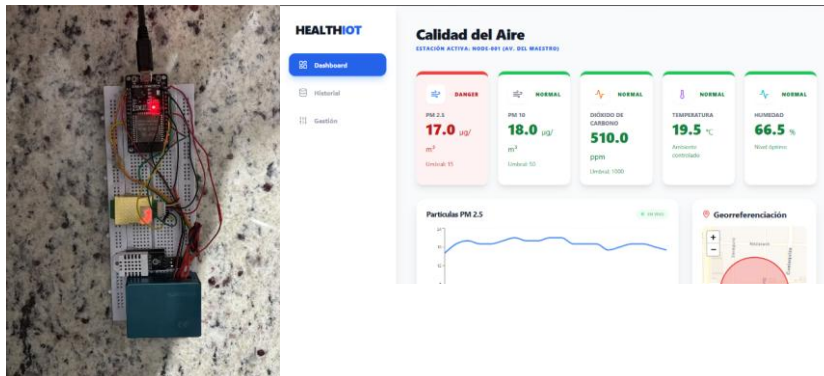
Esta última tarea de este segundo sprint se enfocó en auditar y validar el flujo de datos de forma completa de la plataforma. El objetivo primordial fue certificar la interoperabilidad ininterrumpida de todas las capas arquitectónicas construidas en las fases previas. A nivel técnico, se emplearon clientes de depuración MQTT para interceptar e inspeccionar los payloads en formato JSON. Esto permitió auditar la integridad de los paquetes a lo largo de toda su ruta: desde la captura física de las variables ambientales en el nodo ESP32, verificando el enrutamiento criptográfico seguro en AWS IoT Core y la persistencia en InfluxDB, hasta culminar la medición de la latencia para su renderización reactiva dentro del dashboard de React.js.

Metodológicamente, la validación se ejecutó con escenarios de inyección de fallos a nivel de hardware y de conexión. Se evaluó la resiliencia del sistema desconectando de forma intencional los buses de datos de los periféricos como la línea 1-Wire del sensor DHT22 y el puerto UART del sensor PMS5003. Dichas pruebas nos permitieron corroborar que el firmware del microcontrolador maneja adecuadamente los errores de lectura asignando valores nulos, evitando el bloqueo del procesador por tiempos de espera infinitos.

Simultáneamente, se certificó que el backend en FastAPI inspecciona estos esquemas irregulares a través del modelo de Pydantic, al detectar un dato anómalo, el orquestador captura la excepción, descarta la trama corrupta antes de ejecutar operaciones de escritura y previene la contaminación de la base de datos, garantizando la alta disponibilidad del servidor.

Figura 40

Nodo Físico en operación y Dashboard presentando los datos recolectados



Nota. Elaboración propia y captura de la página web activa

Comentado [27]: Detallar técnicamente lo realizado en esta frase de pruebas.

4.5. Desarrollo de las Tareas dentro del Sprint 3

4.5.1. Backlog y Planificación del Sprint 3

En este tercer periodo de desarrollo, el enfoque se desplazó hacia la robustez del sistema, la optimización de la experiencia de usuario (UI/UX) y la consolidación del despliegue en el entorno de producción. El objetivo fue asegurar que la plataforma fuera funcional, segura y fácil de mantener.

Dentro de la tabla 34 se encuentra el product backlog el cual se orientó en pulir detalles técnicos y asegurar la conectividad bidireccional entre el usuario y nodo.

Tabla 34

Sprint 3

Código	Actividad / Tarea	Prioridad	Estimación
T3-1	Desarrollo de consultas analíticas sobre InfluxDB para gráficas temporales (HU-06).	Alta	5 días – Completado
T3-2	Integración del mapa georreferenciado con Leaflet para ubicación de nodos.	Media	4 días – Completado
T3-3	Optimización de tiempos de respuesta en la API para consultas masivas.	Baja	3 días – Completado

T3-4	Despliegue final de la solución en infraestructura Cloud y documentación técnica.	Alta	4 días - Completado
------	---	------	---------------------

Nota. Elaboración propia

T3-1: Desarrollo de consultas analíticas sobre InfluxDB para gráficas temporales (HU-06)

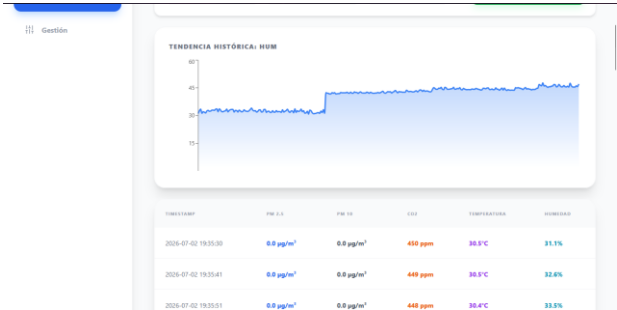
Comentado [28]: ¿Cómo técnicamente se hizo esto?

Esta tarea consistió en el desarrollo de consultas avanzadas utilizando el lenguaje Flux para la extracción eficiente de series temporales desde InfluxDB. El principal reto se presentó al estructurar sentencias eficientes para extraer altos volúmenes de telemetría sin comprometer el consumo de memoria del backend. En términos de implementación, se diseñaron rutinas que delimitan el marco temporal de consulta mediante la función `range()`, adaptándose dinámicamente al intervalo elegido por el usuario. A continuación, se incorporaron cláusulas `filter()` para discriminar las etiquetas del nodo emisor y se ejecutó la instrucción `aggregateWindow()`, calculando promedios continuos para consolidar las mediciones de PM 2.5, PM10, CO2, Temperatura y Humedad en intervalos regulares. Finalmente, con la operación `pivot()` para transformar los datos de un esquema columnar nativo a una estructura tabular en filas.

Como resultado de este procesamiento, se consolidaron los endpoints analíticos en FastAPI. El servidor utiliza el cliente oficial de InfluxDB para ejecutar estas sentencias Flux de manera síncrona; una vez que recibe la respuesta, el backend itera sobre la estructura resultante y la serializa de forma inmediata en un arreglo de objetos JSON. El preprocesamiento de los registros en el backend asegura que la interfaz en `react.js` obtenga las estructuras de datos normalizadas y lista para alimentar las gráficas lineales y de área, disminuyendo la sobrecarga de cómputo y retrasos en el cliente web. Esta solución garantiza que la herramienta ofrezca series cronológicas robustas e integra, posibilitando una interacción ágil e intuitiva al alternar entre distintos periodos de tiempo.

Figura 40

Visualización de la interfaz de Historial dentro de la plataforma web



Nota. Capturado desde la plataforma Web

T3-2: Integración del mapa georreferenciado con Leaflet para la ubicación de nodos

Comentado [29]: ¿Cómo técnicamente se hizo esto?

La Integración de la biblioteca de código abierto Leaflet, mediante su encapsulamiento para el entorno de React, permitió dotar al sistema de una dimensión espacial necesaria para la contextualización de los datos ambientales recolectados. Se convirtió las coordenadas estáticas en un plano visual interactivo dentro de la interfaz, para este fin, tiene una codificación funcional que genera el contenedor cartográfico y superpone las capas de mosaicos geoespaciales provistas por un servicio de mapas abiertos.

En cuanto a la visualización de los dispositivos, el módulo se suscribe al estado central de la plataforma, recorriendo el conjunto de nodos registrados para trazar dinámicamente indicadores geográficos y ventanas informativas vinculadas a las mediciones ambientales en tiempo real. Además, uno de los desafíos técnicos de esta integración radicó en la convivencia de Leaflet con el DOM virtual de React. Para evitar problemas de renderizado incompleto o mapas cortados al cambiar el tamaño de las vistas, se implementó un hook especializado como useMap combinado con useEffect. Con esto la lógica escucha los cambios en redimensionamiento del contenedor padre y ejecuta programáticamente el método interno invalidateSize(), forzando al motor de Leaflet a recalcular y ajustar las dimensiones del mapa de forma responsiva.

Gracias a esta arquitectura de componentes, el sistema ahora ofrece una interfaz altamente interactiva donde el monitoreo ambiental tiene un referente físico concreto. La capacidad de observar la georreferenciación exacta de los nodos, combinada con la actualización reactiva del DOM, mejora significativamente la experiencia del usuario y facilita la toma de decisiones al vincular la calidad del aire con cuadrantes geográficos específicos.

Por esta estructura modular, la plataforma brinda un entorno visual sumamente intuitivo en el que los indicadores ambientales adquieren una correspondencia geográfica tangible. La visualización cartográfica precisa de cada estación, articulada con la reactividad inmediata de los componentes web, eleva la experiencia de navegación y optimiza la capacidad de análisis comunitario al asociar los índices de contaminación con sectores urbanos específicos.

Figura 41

Sección de georreferenciación en el dashboard



Nota. Capturado desde la plataforma Web

T3-3: Optimización de tiempos de respuesta en la API para consultas masivas

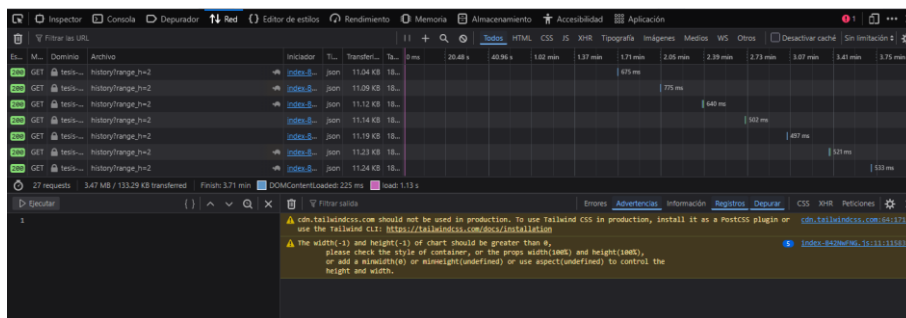
Esta tarea tuvo como enfoque el ajuste de rendimiento del backend para poder gestionar consultas masivas de datos históricos sin comprometer la estabilidad del sistema. Se llevaron a cabo técnicas de optimización server-side, incluyendo la estructuración lógica de los queries de InfluxDB y la reducción del payload de las respuestas JSON. El objetivo fue centrarse en reducir la latencia del paquete de datos procesado, evitando que el tratamiento de grandes volúmenes de telemetría provoque cuellos de botella o interrupciones por timeouts en el backend.

Esta optimización permite que la plataforma tenga una mayor tolerancia frente al incremento en la tasa de muestreo de los nodos periféricos. Al agilizar los tiempos de transferencia, la interfaz mantiene un rendimiento fluido aun durante la consulta de rangos temporales extensos, consolidando la eficiencia operativa global del sistema.

Este ajuste de rendimiento es fundamental para mantener la calidad del servicio y asegurar una experiencia de usuario profesional en la explotación analítica de los datos.

Figura 42

Eficiencia de la API al procesar consultas masivas de datos históricos



Nota. Rendimiento real recolectado desde el Navegador web

T3-4: Despliegue final de la solución en infraestructura Cloud y documentación física**Comentado [30]:** ¿Cómo técnicamente se hizo esto?

La fase final del sprint consistió en la migración y despliegue del sistema completo hacia la plataforma de nube Render. A nivel técnico, este proceso se ejecutó mediante la integración directa del repositorio de control de versiones en GitHub con los servicios cloud. Se configuraron dos instancias separadas: un Web Service para el orquestador de FastAPI, definiendo el comando de arranque de producción mediante el servidor ASGI Uvicorn, y un entorno de alojamiento de sitios estáticos para la página cliente, el cual ejecuta el proceso de compilación de React.js con la ejecución del comando `npm run build`, se estructuró un flujo de integración y despliegue continuo soportado por webhooks. Este mecanismo asegura que cada commit enviado a la rama principal active automáticamente la compilación del contenedor y su puesta en producción, eliminando la necesidad de intervenciones manuales.

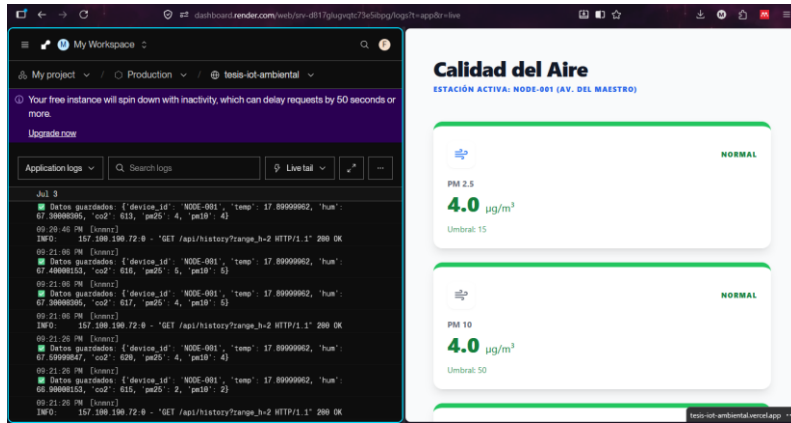
Entrando en el ámbito de la ciberseguridad, se siguieron lineamientos de código limpio para desacoplar los secretos y credenciales del repositorio fuente. Parámetros críticos como la cadena de conexión a MongoDB, Los tokens de autenticación para InfluxDB y la clave simétrica del esquema de JWT se inyectaron directamente en el modulo de variables del entorno del servidor en Render.

Adicionalmente, el entorno gestiona la emisión e instalación automatizada de certificados TLS mediante la entidad Let's Encrypt. Dicha configuración habilita la exposición segura de los endpoints del backend y la interfaz web a través del protocolo HTTPS, neutralizando posibles ataques de interceptación de tráfico o Man In the Middle.

Con este despliegue, el proyecto supera la fase de prototipado y se consolidó como un sistema de alta disponibilidad accesible en la nube, marcando la culminación del ciclo de vida de desarrollo propuesto. La plataforma se encuentra actualmente operativa, cumpliendo los estándares de seguridad web necesarios para su uso y validación en entornos reales. Se puede acceder al dashboard de monitoreo ciudadano a través del siguiente enlace de producción: <https://tesis-iot-ambiental.vercel.app/>.

Figura 43

Interfaz del sistema desplegado y operativo



Nota. Plataforma activa desde el navegador junto al log de Render con los datos recolectados

Capítulo Cinco

Pruebas y Validación

5.1. Pruebas de Usabilidad

Las Pruebas de Usabilidad tienen como propósito fundamental poder medir el grado en el que la interfaz de usuario y la experiencia de usuario (UX/UI) de la plataforma HealthIoT permiten a los ciudadanos y administradores alcanzar sus objetivos con efectividad, eficiencia y satisfacción. Dado que el sistema está orientado a la interpretación de datos ambientales críticos, la claridad visual y la navegación intuitiva son requisitos innegociables.

5.1.1. Escala de Usabilidad del Sistema (SUS) como instrumento de evaluación

Para la cuantificación empírica de la usabilidad, se adoptó el System Usability Scale por sus siglas SUS, este es un instrumento estandarizado y validado dentro de la industria de desarrollo de Software. El SUS proporciona una métrica fiable a través de un cuestionario de 10 ítems los cuales 5 están redactados de forma positiva y 5 de forma negativa, se llevó a cabo una encuesta en donde los usuarios pueden responder mediante una escala de Likert de 5 puntos desde el 1 que es: "Totalmente en desacuerdo" hasta 5: "Totalmente de Acuerdo".

El instrumento evaluó las dimensiones clave de la interacción humano-computadora, tales como la facilidad de aprendizaje, consistencia en el diseño y la necesidad de contar con soporte técnico externo.

5.1.2. Diseño del Escenario de Prueba

Se conformó una muestra no probabilística de 11 participantes, abarcando perfiles heterogéneos que incluyeron estudiantes, profesionales y residentes en general del sector urbano de análisis, dentro del apéndice 1 se puede ver como algunos de estos ciudadanos experimentan y conocen a la plataforma.

Antes de contestar el cuestionario, cada participante tuvo una interacción con el Dashboard en un entorno no controlado, se les solicitó completar tres tareas fundamentales sin una asistencia guiada:

Comentado [31]: Se necesita evidencias de las pruebas a usuarios.

1. Identificar la lectura actual de Material Particulado PM2.5 y Dióxido de Carbono CO2 del nodo activo.

2. Interpretar el nivel de riesgo ambiental basándose exclusivamente en la semaforización visual de la interfaz.

3. Navegar hacia el módulo de datos históricos y visualizar la tendencia temporal de una variable específica.

Tras completar las tareas establecidas, se procedió a la aplicación del instrumento de evaluación. Para medir la usabilidad percibida se realizó el cuestionario estandarizado SUS mencionado anteriormente, este instrumento se seleccionó por su alta fiabilidad y validez comprobada en la industria del software.

Las 10 preguntas evaluadas en el formulario fueron las siguientes:

- Creo que me gustará utilizar con frecuencia este sistema.
- Encontré el sistema innecesariamente complejo.
- Me pareció que fue fácil usar el sistema.
- Creo que necesitaría el apoyo de una persona técnica para ser capaz de usar este sistema.
- Encontré que las diversas funciones de este sistema estaban bien integradas.
- Pensé que había demasiada inconsistencia en este sistema.
- Imagino que la mayoría de las personas aprenderían a usar este sistema de forma muy rápida.
- Encontré que el sistema era muy engorroso (difícil) de usar.
- Me sentí muy confiado/a en el manejo del sistema.
- Siento que necesito aprender muchas cosas antes de poder manejar el sistema.

5.1.3. Procesamiento y Resultados

Una vez que se concluyeron con las tareas de interacción en el Dashboard, se procedió a la tabulación de las respuestas del instrumento SUS, exportadas del archivo fuente "Evaluación de Usabilidad – Plataforma HealthIoT Quito".

Para el procesamiento cuantitativo, se aplicó el algoritmo estándar de calificación del cuestionario SUS. Para los ítems Impares, se le restó 1 valor al seleccionado por el usuario. Para los ítems pares, se restó el valor seleccionado a 5. Posteriormente, la sumatoria de estas puntuaciones ajustadas se multiplicó por un factor constante de 2.5, escalando el resultado final a un rango porcentual de 0 a 100.

En la tabla 35 se detallan las respuestas individuales de los 11 participantes para los 10 ítems evaluados, así como el puntaje final calculado para cada uno de ellos.

Tabla 34

Matriz de recolección de datos y cálculo de la métrica SUS Por participante.

ID	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	Puntaje SUS
01	5	2	4	2	4	1	5	2	4	1	85.0
02	4	2	4	2	4	2	4	2	4	1	77.5
03	5	2	5	2	4	1	5	2	5	1	90.0
04	4	2	5	2	4	1	4	2	4	1	82.5
05	4	2	4	2	3	2	4	1	4	2	75.0
06	5	1	4	2	5	2	5	2	4	1	87.5
07	4	2	4	1	4	2	5	2	3	1	80.0
08	5	1	5	2	5	2	5	1	4	1	92.5
09	4	2	4	1	4	2	5	2	3	1	80.0
10	4	2	4	2	4	1	3	2	4	1	77.5
11	3	2	4	2	5	2	4	2	4	1	77.5
<i>Puntaje</i>											82.3

Nota. Elaboración propia en base a datos recolectados

5.1.4. Discusión de Resultados

Dentro del análisis cuantitativo de la muestra obtenida nos arrojó un puntaje global de 82.3 sobre 100. En el contexto de la ingeniería de Software, la literatura académica establece que un Puntaje SUS Superior a 68 indica un nivel de usabilidad aceptable, mientras que puntajes por encima de 80.3 sitúan al sistema en el percentil superior, otorgándole una calificación de categoría “Excelente”.

Relacionando este puntaje numérico con las tareas asignadas a los usuarios durante el escenario de prueba, se extraen los siguientes hallazgos técnicos:

- **Baja Carga Cognitiva y Curva de Aprendizaje:** El ítem 7 del cuestionario mencionaba la rapidez con la que las personas aprenderían a usar el sistema, este ítem tuvo una valoración sobresaliente, con respuestas mayoritariamente ubicadas en los valores de 4 y 5. Se correlaciona directamente con el ítem 10 “Necesitaría aprender muchas cosas antes de poder usar este sistema”, el cual obtuvo calificaciones mínimas casi todas en valores de 1. Desde la perspectiva del desarrollo, demuestra que la abstracción de los datos crudos hacia la escala de colores semántica de la OMS fue un éxito, permitiendo que ciudadanos sin formación técnica puedan interpretar la calidad del aire de manera instantánea y sin requerir manuales de usuario.
- **Navegabilidad y ausencia de fricción técnica:** El ítem 8, encargado de evaluar si el sistema resulta complejo de utilizar, obtuvo puntajes muy bajos con el 1 y 2 como predominantes. Este comportamiento en la muestra nos confirma que la arquitectura de información aplicada y el diseño estructurado en React.js facilitan una interacción fluida. La decisión de omitir métricas técnicas innecesarias y centrarse en variables ambientales evitó la sobrecarga visual, manteniendo al usuario enfocado en la información de valor.

- Alta aceptación y pertinencia de Uso: El ítem 1, referente al deseo de utilizar el sistema frecuentemente, presentó una tendencia altamente positiva entre 4 y 5. Valida que la Plataforma HealthIoT no solamente cumple con un propósito informativo, sino que genera un compromiso con el usuario final. Al implementar tecnologías de tiempo real, el ciudadano percibe que el dashboard es una herramienta confiable y actualizada para la toma de decisiones preventivas sobre su salud respiratoria.

En síntesis, el puntaje global de 82.3 certifica que las decisiones tomadas en el frontend como la renderización condicional, diseño adaptable y la integración de mapas interactivos lograron mitigar eficazmente la brecha entre la complejidad de la infraestructura IoT subyacente y la experiencia del usuario final. El sistema cumple a cabalidad con los atributos de calidad de software correspondientes a la usabilidad, operabilidad y protección contra errores de usuario.

5.2. Pruebas de Accesibilidad

Para complementar la validación de accesibilidad, se ejecutó un análisis estático automatizado utilizando Lighthouse en su versión 13.0.1, a través de la plataforma AChecker. La auditoría fue capturada el 4 de agosto del 2026 sobre el entorno de producción de la aplicación, alojado en la ruta: <https://tesis-iot-ambiental.vercel.app/>. A nivel técnico, las pruebas se simularon bajo un entorno de escritorio emulado, aplicando custom throttling para la red y utilizando el motor Chromium 116.0.0 con Node en una sesión de página única.

El motor de auditoría arrojó resultados sobresalientes, situando a la plataforma en un percentil de alto rendimiento en todas las categorías evaluadas:

- Accesibilidad: 96 sobre 100.
- Mejores Prácticas: 96 sobre 100.
- Optimización para Motores de Búsqueda: 91 sobre 100.

5.3.1. Análisis de Hallazgos y Oportunidades de Mejora

A pesar de haber alcanzado calificaciones de excelencia. La detección automática identificó advertencias específicas que representan oportunidades de optimización para el código base de la página web:

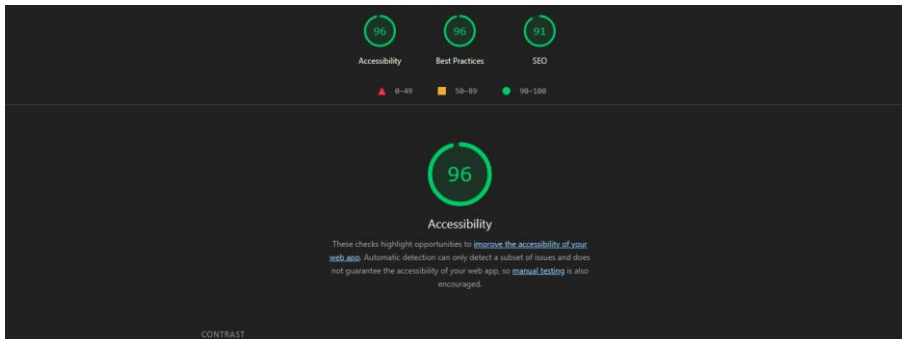
hablando de accesibilidad, el reporte levantó una alerta en la categoría de contraste, lo que indica que los colores de fondo y primer plano no poseen un ratio de contraste suficiente. En el contexto del dashboard, esto sugiere que ciertas paletas de colores semánticos, utilizadas para indicar la calidad del aire, requerirán en un futuro ajustes precisos en los valores hexadecimales para cumplir con mayor rigor los criterios visuales de las pautas WCAG.

En el apartado de mejores prácticas de contenido, la herramienta identificó la ausencia de una etiqueta de descripción. Desde la perspectiva de desarrollo web, el formato del HTML debe permitir que los rastreadores comprendan mejor el contenido de la aplicación, la solución a esto es incluyendo una etiqueta donde exista la descripción del contenido dentro del bloque head del DOM.

En resumen, los altos puntajes obtenidos como se ven en la figura 44 dan certeza de que la arquitectura del proyecto es inclusiva, segura a nivel estructural y de un buen rendimiento, al mismo tiempo, el análisis de las advertencias reportadas por Lighthouse proporciona evidencia de un proceso transparente de evaluación y nos da una hoja de ruta clara para escalar dicho sistema.

Figura 44

Resultado de Accesibilidad proporcionado por Lighthouse



Nota. Captura tomada desde Firefox

5.3. Pruebas de Seguridad y Análisis de Vulnerabilidades

En el diseño de arquitecturas IoT orientadas a la telemetría ciudadana, la superficie de ataque abarca tanto los dispositivos físicos como las interfaces de consumo de datos. Para garantizar la confidencialidad y la integridad de la plataforma, se ejecutaron pruebas de seguridad dinámicas enfocadas en las vulnerabilidades más críticas descritas por el proyecto OWASP, prestando especial atención a las inyecciones de código y la exposición de datos sensibles en la capa de presentación.

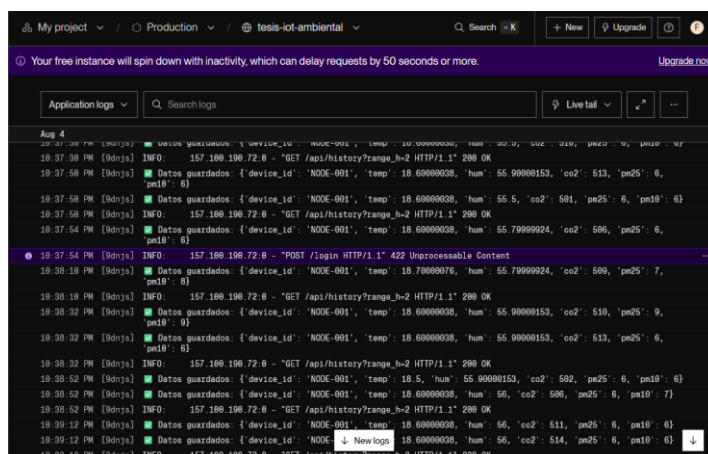
5.5.1. Prevención de inyecciones de código

En las arquitecturas web tradicionales, la inyección SQL representa uno de los vectores de ataque más severos. Sin embargo, debido a que el ecosistema de este proyecto implementa un modelo de persistencia híbrido basado en repositorios no relacionales y de series temporales, el riesgo de inyecciones basadas en sentencias estructuradas se encuentra mitigado desde el diseño arquitectónico.

No obstante, para certificar la inmunidad del sistema ante ataques de Inyección NoSQL, se diseñaron escenarios de prueba enviando cargas útiles maliciosas a través de los endpoints de la API REST. Se intentó manipular los operadores lógicos de las consultas con \$gt en los campos de autenticación de formato JSON para forzar el acceso no autorizado. Las pruebas confirmaron que el backend es completamente resiliente a estos ataques. Esta protección técnica se logra gracias a la implementación de modelos de tipado estricto con la librería Pydantic en FastAPI, la cual actúa como un filtro de sanitización riguroso que valida, escapa y rechaza cualquier estructura de datos anómala antes de que esta alcance el motor de la base de datos. Dentro de la figura 45 Podemos observar como el ataque con el operador \$gt fue invalido.

Figura 45

Resultado de Inyección al Proceso de Login



Nota. Tomado desde el log de Render

5.5.2. Mitigación de fuga de información y vulnerabilidades XSS

El segundo vector de auditoria se enfocó en la capa del navegador (frontend) para prevenir la fuga de información sensible. Uno de los riesgos principales en aplicaciones web interactivas es el Cross-Site Scripting (XSS), donde un atacante intenta inyectar scripts maliciosos en la vista de otros usuarios para robar credenciales o interceptar la sesión.

Las pruebas de intrusión en la interfaz consistieron en intentar incrustar etiquetas script manipuladas dentro de los campos de texto durante el registro de nuevos nodos de monitoreo. Los resultados evidenciaron que la plataforma bloquea satisfactoriamente la ejecución de código de terceros. A nivel de ingeniería de TI, esto es el resultado de haber construido la vista utilizando React.js. El DOM virtual de React realiza un proceso automático de escape de variables antes de renderizar cualquier entrada de texto, neutralizando las inyecciones de XSS por defecto, en la figura 46 podemos ver este ataque siendo interceptado.

Figura 46

Resultado de Inyección XSS



Nota. Tomado desde el Módulo de gestión de nodos

Para validar la seguridad de la capa lógica de negocio y garantizar que la telemetría no pueda ser alterada por agentes externos, se ejecutó una prueba de omisión de credenciales sobre la API REST. Utilizando la interfaz de documentación Swagger UI, se generó una petición de tipo POST hacia el *endpoint* de registro de dispositivos (/nodos), omitiendo intencionalmente la cabecera de Autorización. El *payload* inyectado simuló la creación de un dispositivo arbitrario, dentro de la figura 47 se puede ver reflejado.

Figura 46

Resultado de Acceso por Swagger



```

Curl
curl -X POST \
  https://tesis-iot-ambiental.orender.com/nodos \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "id": "string",
    "nombre": "string",
    "estado": "string",
    "email": "50"
  }'

Request URL
https://tesis-iot-ambiental.orender.com/nodos

Server response
Code    Details

```

Nota. Tomado desde la Interfaz con el módulo de prueba

Como resultado de esta transacción, el orquestador desarrollado en FastAPI interceptó la petición en la capa de middleware. Al detectar la ausencia del JSON Web Token (JWT) válido en las cabeceras HTTP, el servidor bloqueó automáticamente el acceso al controlador de la base de datos y retornó un código de estado HTTP 401 Unauthorized. Esta validación empírica demuestra que el sistema implementa un enrutamiento privado estricto, mitigando el riesgo de manipulación de la infraestructura por parte de usuarios no autenticados y garantizando la integridad de la plataforma IoT.

5.5.3. Resultados de Seguridad

La evaluación rigurosa de los vectores de prueba demostró la adherencia del ecosistema a los principios de desarrollo seguro. El contar con el codificado estricto de datos en el backend mediante FastAPI y los mecanismos automáticos de escape en el frontend confiere a la plataforma HealthIoT una alta tolerancia frente a vulnerabilidades por inyección y filtración de información en el cliente.

La fase de validación empírica constato la efectividad, solidez y rendimiento del sistema en sus tres ejes centrales: usabilidad, desempeño web y ciberseguridad. En el ámbito de la interacción humano-computador, la escala de usabilidad del sistema (SUS) alcanzó un puntaje de 82.3/100, ubicando a la solución en un rango de excelencia y evidenciando una rápida asimilación por parte de los usuarios, facilitada por la semaforización de riesgos alineada a los estándares de la OMS.

Paralelamente, las evaluaciones automatizadas con Google Lighthouse confirmaron la calidad del diseño al obtener puntuaciones de 96/100 tanto en accesibilidad como en buenas practicas de desarrollo, asegurando un despliegue responsivo y fluido de la telemetría.

En materia de infraestructura y defensa perimetral, las pruebas dinámicas contra el catalogo de riesgos OWASP ratificaron la robustez del entorno. El orquestador en FastAPI, soportado por la validación de esquemas con Pydantic, neutralizó de forma efectiva intentos de inyección NoSQL, mientras que la capa de presentación en React.js contuvo posibles ataques de Cross-Site Scripting (XSS). De igual forma, la gestión de identidades mediante tokens criptográficos JWT y la autenticación mutua En AWS IoT Core bloquearon accesos no autorizados, garantizando la inmutabilidad de las series temporales y consolidando a la plataforma como un sistema confiable y escalable para la vigilancia inteligente de la calidad del aire.

Conclusiones

Tras el desarrollo, implementación y validación integral de la plataforma de monitoreo ambiental HealthIoT, se desprenden las siguientes conclusiones técnicas y funcionales:

La fase de análisis inicial presentó las restricciones de resolución espacial inherentes a las estaciones meteorológicas fijas convencionales ante la topografía accidentada y densidad del tráfico vehicular urbano. El despliegue focalizado en el sector del barrio “El Rosario” corroboró la necesidad imperativa de registrar métricas continuas a microescala, demostrando que la detección precisa de focos de emisión demanda arquitecturas distribuidas que complementen la red de vigilancia institucional para sustentar intervenciones eficaces en salud pública.

Se consolidó una estación junto al Microcontrolador ESP32 acoplado a sensores de bajo costo y alta fidelidad operativa (PMS5003 para material particulado, MH-Z19C para CO₂ y DHT22 para parámetros termo higrométricos. El uso del protocolo liviano MQTT encapsulado bajo canales con autenticación mutua mTLS en AWS IoT Core evidenció una notable eficiencia energética y tolerancia a fallos durante la transmisión, comprobando la viabilidad técnica del despliegue de mallas sensoriales asequibles en contextos metropolitanos complejos.

Se construyó una infraestructura de software robusta, desacoplada y de alta disponibilidad desplegada en la nube. La orquestación del backend en FastAPI con un esquema de persistencia híbrida (MongoDB para metadatos e InfluxDB para series temporales de alta frecuencia), en conjunto con una interfaz reactiva en React.js y el uso de WebSockets, garantizó la entrega de datos con una latencia mínima, ofreciendo a la ciudadanía una navegación fluida, georreferenciada con Leaflet y con un nivel de usabilidad sobresaliente de 82.3 sobre 100 en la escala SUS.

. Se integró exitosamente un motor analítico en la capa lógica que cruza las mediciones ambientales en tiempo real con los límites permisibles de la Organización Mundial de la Salud (OMS). Esta funcionalidad traduce lecturas cuantitativas complejas en una escala

colorimétrica intuitiva y notificaciones visuales automáticas, dotando a la plataforma de una capacidad reactiva inmediata para advertir a la población sobre episodios de exposición a aire dañino sin requerir intervención manual ni recarga del navegador

En cuanto a la persistencia y análisis de datos, el uso de InfluxDB como repositorio de series temporales permitió consolidar un historial ambiental íntegro y estructurado. La estructuración en lenguaje Flux demostró un rendimiento óptimo en la ejecución de agregaciones matemáticas y filtrados temporales dinámicos, procesando grandes volúmenes de telemetría sin comprometer el consumo de memoria del servidor.

Se diseñó, implementó y validó por completo integralmente la plataforma HealthIoT para la vigilancia inteligente de la calidad del aire en sectores urbanos de Quito. La solución articuló exitosamente la captura sensorial en Edge computing, la ingesta segura sobre la nube y la visualización interactiva orientada al usuario, consolidándose como un ecosistema funcional, escalable y robusto que democratiza el acceso a métricas ambientales y respalda la prevención comunitaria en salud respiratoria.

Recomendaciones

En correspondencia con los resultados alcanzados y proyectando la escalabilidad del sistema, se formulan las siguientes líneas de trabajo e investigación futura:

Para despliegues masivos a escala metropolitana, se sugiere transicionar el alojamiento del backend hacia una arquitectura de microservicios contenerizados y orquestados con Kubernetes. A la par, se recomienda profundizar en la ejecución de algoritmos de procesamiento local en los microcontroladores para realizar el filtrado inicial de lecturas atípicas, reduciendo la transferencia de datos y los costos operativos de la nube.

Con las series temporales que se encuentran almacenadas, se plantea el entrenamiento de modelos de aprendizaje automático basados en redes neuronales recurrentes (LSTM) o modelos autorregresivos integrados de medias móviles (ARIMA). Esta evolución permitirá transitar de un esquema de supervisión reactivo a un paradigma analítico predictivo, capaz de anticipar contingencias ambientales y picos de polución con horas de antelación.

Para fortalecer la apropiación comunitaria de este sistema y extender la accesibilidad ciudadana, se aconseja el desarrollo de un aplicativo móvil nativo o multiplataforma. Este desarrollo consumirá los servicios de la API REST e integrará sistemas de mensajería push geolocalizados, advirtiéndole a la población de forma proactiva ante la superación de umbrales críticos de exposición.

Para generar un perfil más exhaustivo del riesgo respiratorio se recomienda ampliar la matriz del hardware para capturar contaminantes gaseosos adicionales de alta relevancia toxicológica, tales como monóxido de carbono, dióxido de nitrógeno y ozono troposférico. A la par, se recomienda integrar pruebas de seguridad automatizadas dentro del ciclo de integración continua para mantener la robustez del código.

Referencias

- Alsamrai, O., Redel-Macias, M. D., Pinzi, S., & Dorado, M. P. (2024). A Systematic Review for Indoor and Outdoor Air Pollution Monitoring Systems Based on Internet of Things. *Sustainability* 2024, Vol. 16, Page 4353, 16(11), 4353. <https://doi.org/10.3390/SU16114353>
- Anderson, D.J. (2010) *Kanban Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, Washington DC. - References - Scientific Research Publishing. (n.d.). Retrieved January 14, 2026, from <https://www.scirp.org/reference/referencespapers?referenceid=2444636>
- Argyropoulos, C., Pana, Z. D., Lin, C., Tariq, H., Touati, F., Crescini, D., & Mnaouer, A. Ben. (2024a). State-of-the-Art Low-Cost Air Quality Sensors, Assemblies, Calibration and Evaluation for Respiration-Associated Diseases: A Systematic Review. *Atmosphere* 2024, Vol. 15, Page 471, 15(4), 471. <https://doi.org/10.3390/ATMOS15040471>
- Argyropoulos, C., Pana, Z. D., Lin, C., Tariq, H., Touati, F., Crescini, D., & Mnaouer, A. Ben. (2024b). State-of-the-Art Low-Cost Air Quality Sensors, Assemblies, Calibration and Evaluation for Respiration-Associated Diseases: A Systematic Review. *Atmosphere* 2024, Vol. 15, Page 471, 15(4), 471. <https://doi.org/10.3390/ATMOS15040471>
- Balagopal, G., Wijeratne, L., Waczak, J., Hathurusinghe, P., Iqbal, M., Kiv, D., Aker, A., Lee, S., Agnihotri, V., Simmons, C., & Lary, D. J. (2025). Calibration of Low-Cost LoRaWAN-Based IoT Air Quality Monitors Using the Super Learner Ensemble: A Case Study for Accurate Particulate Matter Measurement. *Sensors* 2025, Vol. 25, Page 1614, 25(5), 1614. <https://doi.org/10.3390/S25051614>
- Dubey, R., Telles, A., Nikkel, J., Cao, C., Gewirtzman, J., Raymond, P. A., & Lee, X. (2024). Low-Cost CO2 NDIR Sensors: Performance Evaluation and Calibration Using Machine Learning Techniques. *Sensors*, 24(17), 5675. <https://doi.org/10.3390/S24175675/S1>
- Felici-Castell, S., Segura-Garcia, J., Perez-Solano, J. J., Fayos-Jordan, R., Soriano-Asensi, A., & Alcaraz-Calero, J. M. (2023). AI-IoT Low-Cost Pollution-Monitoring Sensor Network to Assist Citizens with Respiratory Problems. *Sensors*, 23(23), 9585. <https://doi.org/10.3390/S23239585/S1>
- Forouzan, B. A. (2012). *Data Communications and Networking* (Marty Lange, Ed.). McGraw-Hill.
- Guerrón, R. A., D'Amore, F., Bencardino, M., Lamonaca, F., Colaprico, A., Lanuzza, M., Taco, R., & Carnì, D. L. (2023). IoT sensor nodes for air pollution monitoring: A review. *Acta IMEKO*, 12(4), 1–10. <https://doi.org/10.21014/ACTAIMEKO.V12I4.1676>
- Hassan, A. K. ;, Saraya, A.-E., Abdelsalam, A. M. T. ;, Hassan, A. K., Saraya, M. S., Ali-Eldin, A. M. T., & Abdelsalam, M. M. (2024). Low-Cost IoT Air Quality Monitoring Station Using Cloud Platform and Blockchain Technology. *Applied Sciences* 2024, Vol. 14, Page 5774, 14(13), 5774. <https://doi.org/10.3390/APP14135774>
- Lewis, J. (2014, March 25). *Microservices*. <https://martinfowler.com/articles/microservices.html>
- Light, R. A. (2017). Mosquitto: server and client implementation of the MQTT protocol. *Journal of Open Source Software*, 2(13), 265. <https://doi.org/10.21105/JOSS.00265>
- Mell, P. M., & Grance, T. (2011). *The NIST definition of cloud computing*. <https://doi.org/10.6028/NIST.SP.800-145>
- MQTT Version 3.1.1*. (n.d.). Retrieved January 5, 2026, from <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>
- Newman, S. (2021). *Building Microservices* (2da ed.). O'Reilly.

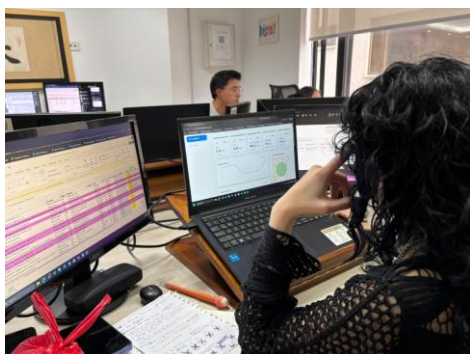
- Organización Mundial de la Salud (OMS). (2021). WHO global air quality guidelines. *Particulate Matter (PM_{2.5} and PM₁₀), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*, 1–360.
- Schwaber, K., Sutherland, J., & Definitiva, L. G. (2020). *La Guía Scrum*.
- Sommerville, I. (2011). *Ingeniería de Software* (9na ed.).
- Stallings, S., & Stallings, W. (n.d.). *Comunicaciones y Redes de Computadores Comunicaciones y Redes de Computadores Comunicaciones y Redes de Computadores*. Retrieved January 19, 2026, from www.pearsoneducacion.com
- Wetherall D., & Tanenbaum A. (2012). *Redes de Computadoras* (Luis M. Cruz Castillo, Bernardino Gutiérrez Hernández, & Juan José García Guzmán, Eds.; 5ta ed.). Pearson. www.pearsoneducacion.net
- WHO global air quality guidelines: particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide: executive summary. (n.d.). Retrieved July 27, 2026, from <https://iris.who.int/items/2f8fec42-5636-4506-9b44-e0c91c678484>
- Google. (2026). *Gemini* [Modelo de lenguaje grande]. <https://gemini.google.com>
- Mermaid. (s.f.). *Mermaid Live Editor* [Aplicación web]. <https://mermaid.live/>

Apéndice

Apéndice 1. Usuarios en Pruebas de Usabilidad

Figura A1

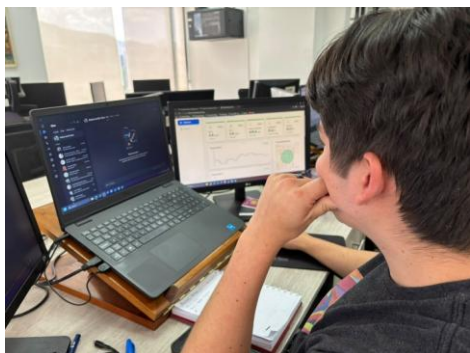
Demostración de plataforma a ciudadano



Nota. Tomado durante la explicación de la plataforma

Figura A2

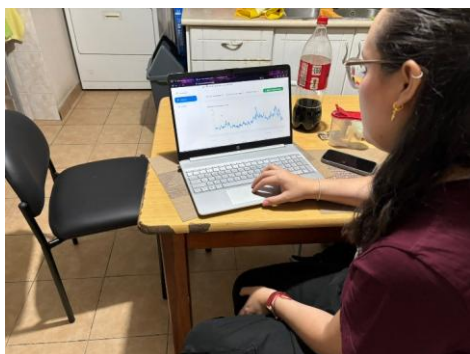
Demostración de plataforma a ciudadano



Nota. Tomado durante la explicación de la plataforma

Figura A3

Demostración de plataforma a ciudadano



Nota. Tomado durante la explicación de la plataforma